Programmering nivå 1 med Python

Den här boken introducerar programmering med Python och följer det centrala innehållet i kursen Programmering 1. Målet är att ge eleverna både förståelse för hur datorer fungerar och praktiska färdigheter i att skriva och analysera program.

Alla övningar finns som kod som du kan ladda ner från denna sida:
https://github.com/paul-pauliskolan/Programmering-1-Python

1- Vad är programmering
- 1.1 Vad är ett program
- 1.2 Vad är programmering
- 1.3 Datorns funktionssätt och programmeringens möjligheter
  - Datorn följer instruktioner
  - Möjligheter
  - Begränsningar
  - Sammanfattning
- 1.4 Programspråk
  - Varför Python
- 1.5 Hur ett program körs
  - Från kod till körning
  - Exekveringsmiljö
  - Operativsystemets roll
  - Hårdvarans roll
  - Så går det till
  - Exempel
  - Varför detta är viktigt att förstå
- 1.6 Kompilerade och interpreterade programspråk
  - Kompilerade programspråk
  - Interpreterade programspråk
  - Python och tolkning
  - Sammanfattning
- 1.7 Installera Python på macOS
  - Steg 1 – Ladda ner Python
  - Steg 2 – Installera Python
  - Steg 3 – Kontrollera installationen
  - Vanliga problem
  - Nästa steg
- 1.8 Skriva och köra program i IDLE
  - Starta IDLE

Python Shell
Skapa ett Pythonprogram
Skriv ditt program
Spara programmet
Kör programmet
Ändra programmet
Sammanfattning
1.9 Installera Visual Studio Code
Installera VS Code
1.10 Programmera i VS Code med Python
Installera Python-tillägget
Välj Python-version
Kör programmet
Använda extentions
1.11 Syntax och noggrannhet i kod
Vad är syntax
Indrag i Python
Vanliga misstag
Felmeddelanden
Var noggrann när du skriver kod
Sammanfattning
1.12 Programmering i vardagen
Programmering i appar och webbtjänster
Programmering i spel
Programmering i teknik och maskiner
Programmering och automatisering
Programmering och samhället
Sammanfattning
1.13 Att tänka som en programmerare
Dela upp problem
Tänk steg för steg
Testa och förbättra
Vara noggrann
Sammanfattning
1.14 Övningar
Övning 1 – Skriv ut text
Övning 2 – Flera rader
Övning 3 – Ändra texten
Övning 4 – Favoritsak
Övning 5 – Använd flera print
Övning 6 – Gör ett fel

Övning 7 – Rätta felet
1.15 Reflektionsfrågor
1 - Vad är ett program? Beskriv med egna ord.
2 - Varför måste man vara noggrann när man skriver kod?
3 - Vad menas med syntax i ett programspråk?
4 - Vad är skillnaden mellan ett kompilerat och ett interpreterat programspråk?
5 - Ge tre exempel på var programmering används i vardagen.
6 - Varför är det viktigt att läsa felmeddelanden när ett program inte fungerar?
7 - Vad menas med att tänka steg för steg när man programmerar?
8 - Varför är det viktigt att förstå koden även om man använder AI-verktyg?
9 -Vilka saker har du lärt dig i detta kapitel som du inte visste innan?
2 - Variabler, datatyper och input
2.1 Vad är en variabel
2.2 Tilldela värden
2.3 Datatyper
Övning 2
Övning 2
2.4 Skriva ut variabler
Skriva ut text och variabler tillsammans
Skriva ut flera variabler
F-strängar
Övning 1 – F-strängar
Övning 2 – F-strängar
Extra uppgift
2.5 Input från användaren
Använda input tillsammans med print
Flera frågor
Viktigt att veta
Testa själv
Övning
2.6 Typomvandling
Omvandla text till tal
Exempel med heltal
Exempel med decimaltal
Kontrollera datatypen
Testa själv
Övning
2.7 Vanliga nybörjarfel
Glömda citattecken
Felstavade kommandon
Saknade parenteser

Fel datatyp
Felmeddelanden
Sammanfattning
2.8 Övningar
Övning 1 – Presentera dig
Övning 2 – Datatyper
Övning 3 – Fråga efter namn
Övning 4 – Ålder
Övning 5 – Längd
Övning 6 – Favoritfilm
Övning 7 – Kontrollera datatyper
Övning 8 – Tre frågor
2.9 Reflektionsfrågor
3 - Problemlösning i programmering
3.1 Vad är ett programmeringsproblem
3.2 Algoritmer
Exempel på en enkel algoritm
Varför algoritmer är viktiga
Egenskaper hos en bra algoritm
Sammanfattning
3.3 Dela upp problem
Exempel
Mindre delar i program
Exempel i Python
Sammanfattning
3.4 Pseudokod
Exempel
Varför pseudokod är användbart
Sammanfattning
3.5 Flödesscheman
Sekvens
Villkor
Iteration
Exempel 1 – Hälsningsprogram
Exempel 2 – Räkna ut summan av två tal
Exempel 3 – Stort eller litet tal
Exempel 4 – Ålder om fem år
Exempel 5 – Enkel meny
start
sätt räknare = 3
medan räknare > 0

skriv ut räknare
räknare = räknare - 1
skriv ut "Klar!"
slut
Sammanfattning
3.6 Från idé till program
Steg 1 Förstå problemet
Steg 2 Planera lösningen
Steg 3 Beskriva lösningen
Steg 4 Skriva programmet
Steg 5 Testa programmet
Sammanfattning
3.7 Exempel: temperaturprogram
Steg 1 Förstå problemet
Steg 2 Algoritm
Steg 3 Pseudokod
Steg 4 Pythonprogram
Hur programmet fungerar
Sammanfattning
3.8 Övningar
Övning 1 Hälsningsprogram
Övning 2 Ålder om tio år
Övning 3 Summan av två tal
Övning 4 Temperaturprogram
Övning 5 Medelvärde
Övning 6 Största talet
3.9 Reflektionsfrågor
4 - Operatorer och beräkningar
4.1 Aritmetiska operatorer
Addition
Subtraktion
Multiplikation
Division
Heltalsdivision
Operatorer och variabler
Sammanfattning
Övningar 4.1 Aritmetiska operatorer
Övning 1 Addition
Övning 2 Subtraktion
Övning 3 Multiplikation
Övning 4 Division
Övning 5 Heltalsdivision

Övning 6 Eget räkneprogram
4.2 Jämförelseoperatorer
Lika med
Inte lika med
Mindre än
Större än
Jämförelser i variabler
Sammanfattning
Övning 1 Lika eller inte
Övning 2 Olika tal
Övning 3 Mindre än
Övning 4 Större än
Övning 5 Jämför två tal från användaren
Övning 6 Testa olika värden
4.3 Logiska operatorer
Operatorn and
Operatorn or
Operatorn not
Exempel med flera villkor
Sammanfattning
Övningar 4.3 Logiska operatorer
Övning 1 Operatorn and
Övning 2 Operatorn or
Övning 3 Operatorn not
Övning 4 Flera villkor
Övning 5 Jämför två tal
Övning 6 Ett av två villkor
Övning 7 Vänd ett värde
4.4 Matematiska uttryck
Flera operatorer i samma uttryck
Använda parenteser
Matematiska uttryck med variabler
Exempel med flera beräkningar
Sammanfattning
Övning 1 Enkel beräkning
Övning 2 Flera operatorer
Övning 3 Parenteser
Övning 4 Beräkning med variabler
Övning 5 Medelvärde
Övning 6 Räkna ut area
4.5 Beräkningsprogram

- Exempel 1 Enkel miniräknare
- Exempel 2 Beräkna pris
- Exempel 3 Temperaturberäkning
- Hur man bygger ett beräkningsprogram
- Sammanfattning

4.6 Vanliga fel
- Glömda parenteser
- Fel operator
- Förväxla = och ==
- Fel datatyp
- Fel ordning i beräkningar
- Sammanfattning

4.7 Övningar
- Övning 1 Enkel beräkning
- Övning 2 Medelvärde
- Övning 3 Räkna ut area
- Övning 4 Temperatur
- Övning 5 Dubbel kontroll
- Övning 6 Största talet

4.8 Reflektionsfrågor

5 - Villkor

5.1 Beslut i program
- Exempel på beslut i program
- Exempel
- Vad betyder if
- Villkor ger True eller False
- Indrag i Python
- Sammanfattning
- Enkel övning

5.2 if
- Grundstruktur
- Exempel
- Hur programmet fungerar
- Exempel på körning
- Villkor i if
- Viktigt med kolon och indrag
- Flera rader kod
- Övning

5.3 if else
- Grundstruktur
- Exempel

- Hur programmet fungerar
- Exempel på körning
- Indrag i if else
- Exempel
- Övning

5.4 elif
- Grundstruktur
- Exempel
- Hur programmet fungerar
- Exempel på körning
- Viktigt att tänka på
- Övning

5.5 Nästlade villkor
- Grundstruktur
- Exempel
- Hur programmet fungerar
- Exempel på körning
- Viktigt att tänka på
- Övning

5.6 Logiska uttryck
- Operatorn and
- Operatorn or
- Operatorn not
- Kombinera logiska operatorer
- Viktigt att tänka på
- Övning

5.7 Program med beslut
- Exempel: biljettpris
- Hur programmet fungerar
- Exempel: jämnt eller udda tal
- Exempel: tillgång till ett konto
- Varför beslut är viktiga
- Övning

5.8 Övningar
- Övning 1
- Övning 2
- Övning 3
- Övning 4
- Övning 5
- Övning 6
- Övning 7

Övning 8
5.9 Reflektionsfrågor
Kapitel 6 Loopar
6.1 Varför loopar behövs
6.2 while-loop
Exempel utan loop
Exempel med loop
Varför loopar är viktiga
Två vanliga loopar i Python
6.3 for-loop
Hur programmet fungerar
Viktigt att tänka på
6.4 range()
Enkelt exempel
Start och stopp
Steg i range()
6.5 Oändliga loopar
Oändlig loop av misstag
Rätt sätt
6.6 Loopar och villkor
Exempel
Hur programmet fungerar
Exempel med användarens inmatning
Varför detta är viktigt
6.7 Program med upprepningar
Exempel: skriva ut flera tal
Exempel: summera tal
Hur programmet fungerar
Exempel: räkna jämna tal
Varför upprepningar är viktiga
6.8 Break
Exempel
Hur programmet fungerar
Exempel på körning
Varför break används
6.9 Övningar
Övning 1
Övning 2
Övning 3
Övning 4
Övning 5

- Övning 6
- Övning 7
- Övning 8
- Vad är FizzBuzz?
- Bakgrund
- 6.10 Reflektionsfrågor

Kapitel 7 Listor

- 7.1 Vad är en lista
  - Skriva ut en lista
  - Loop genom en lista
  - 7.2 Index
  - Hämta ett värde med index
  - Viktigt att tänka på
  - Övning
- 7.2 Index
  - Hämta ett värde med index
  - Ändra ett värde i en lista
  - Viktigt att tänka på
  - Övning
- 7.3 Loopar över listor
  - Exempel
  - Hur det fungerar
  - Exempel med beräkning
  - Varför detta är användbart
  - Övning
- 7.4 Lägga till värden
  - Exempel
  - Exempel med flera värden
  - Exempel med användarens inmatning
  - Övning
- 7.5 Bearbeta data
  - Exempel: summa
  - Exempel: medelvärde
  - Exempel: största värdet
  - Varför detta är viktigt
  - Övning
- 7.6 Program med listor
  - Exempel: samla in tal
  - Exempel: hitta största talet
  - Exempel: räkna jämna tal
  - Varför listor är viktiga

- Övning
- 7.7 Övningar
  - Övning 1
  - Övning 2
  - Övning 3
  - Övning 4
  - Övning 5
  - Övning 6
  - Övning 7
- 7.8 Reflektionsfrågor

Kapitel 8 Funktioner

- 8.1 Varför funktioner används
  - Exempel utan funktion
  - Exempel med funktion
  - Varför funktioner är bra
  - Viktigt att tänka på
  - Övning
- 8.2 Definiera funktioner
  - Exempel
  - Hur programmet fungerar
  - Exempel med f-sträng
  - Övning
- 8.3 Parametrar
  - Exempel
  - Flera parametrar
  - Hur parametrar fungerar
  - Övning
- 8.4 Return
  - Exempel
  - Vad som händer i programmet
  - Varför return används
  - Exempel
  - Övning
- 8.5 Strukturera program
  - Exempel
  - Varför struktur är viktigt
  - Bra vanor i Python
  - Övning
- 8.6 Större program
  - Exempel
  - Hur programmet är uppbyggt

- Fördelar med denna struktur
- Övning
- 8.7 Övningar
  - Övning 1
  - Övning 2
  - Övning 3
  - Övning 4
  - Övning 5
  - Övning 6
- 8.8 Reflektionsfrågor

Kapitel 9 Felsökning

- 9.1 Syntaxfel
  - Exempel på syntaxfel
  - Exempel med saknat kolon
  - Exempel med stavfel
  - Hur man hittar syntaxfel
  - Övning
- 9.2 Logiska fel
  - Exempel på logiskt fel
  - Ett annat exempel
  - Hur man hittar logiska fel
  - Övning
- 9.3 Exekveringsfel
  - Exempel: fel datatyp
  - Exempel: division med noll
  - Exempel: variabel som inte finns
  - Hur man hittar exekveringsfel
  - Övning
- 9.4 Läsa felmeddelanden
  - Exempel
  - Exempel med division med noll
  - Vad man ska göra när ett fel uppstår
  - Övning
- 9.5 Systematisk felsökning
  - Ett bra arbetssätt
  - Använd print vid felsökning
  - Exempel
  - Testa små delar
  - Sammanfattning
  - Övning
- 9.6 Testa program

- Testa med olika värden
- Testa gränsvärden
- Testa felaktig indata
- Övning
- 9.7 Övningar
  - Övning 1
  - Övning 2
  - Övning 3
  - Övning 4
  - Övning 5
  - Övning 6
- 9.8 Reflektionsfrågor

Kapitel 10 Undantagshantering
- 10.1 Vad är undantag
  - Varför använda undantag
  - Exempel utan undantag
  - Exempel med undantag
  - Hur det fungerar
  - Övning
- 10.2 Hantera felaktig input
  - Problem utan felhantering
  - Lösning med try except
  - Be användaren försöka igen
  - Varför detta är viktigt
  - Övning
- 10.3 Robusta program
  - Exempel på icke robust program
  - Exempel på robust program
  - Vad gör ett program robust
  - Varför detta är viktigt
  - Övning
- 10.4 Övningar
  - Övning 1
  - Övning 2
  - Övning 3
  - Övning 4
  - Övning 5
  - Övning 6
- 10.5 Reflektionsfrågor

Kapitel 11 Struktur och läsbar kod
- 11.1 Namngivning1

- Dåliga namn
- Bra namn
- Namn på funktioner
- Regler i Python
- Bra vanor
- Sammanfattning
- Övning

11.2 Kommentarer
- Exempel
- När man ska använda kommentarer
- När man inte ska använda kommentarer
- Bättre kod istället för kommentarer
- Kommentarer i funktioner
- Sammanfattning
- Övning

11.3 Strukturera kod
- Dela upp i funktioner
- Använd tydliga steg
- Undvik onödigt upprepad kod
- Sammanfattning
- Övning

11.4 Dela upp program och läslig kod
- Dela upp program i delar
- Läslig kod
- Exempel på oläslig kod
- Exempel på läsbar kod
- Tänk på
- Sammanfattning
- Övning

11.5 Övningar
- Övning 1
- Övning 2
- Övning 3
- Övning 4
- Övning 5

11.6 Reflektionsfrågor

Kapitel 12 Användargränssnitt

12.1 Program och användare
- Olika typer av användargränssnitt
- Exempel
- Vad är ett bra användargränssnitt

- Exempel på tydligt program
- Exempel på otydligt program
- Tänk på
- Sammanfattning

12.2 Textbaserade menyer
- Exempel på meny
- Använda val med if
- Göra menyn återkommande
- Varför använda menyer
- Tänk på
- Övning

12.3 Introduktion till Tkinter
- Vad är Tkinter
- Skapa ett enkelt fönster
- Exempel: hela programmet
- Lägga till text
- Lägga till en knapp
- Viktiga delar
- Sammanfattning
- Övning

12.4 TKinter input och uppdatera text
- Ta emot input i GUI
- Hämta värdet från inputfältet
- Uppdatera text i fönstret
- Sammanfattning
- Övningar

12.5 Tkinter Input och output, widgets, ändra utseende
- Input och output i GUI
- Flera widgets i samma program
- Ändra utseende
- Rensa inputfält
- Kontrollera input (enkel validering)
- Placering med grid istället för pack
- Sammanfattning
- Övningar
- Radioknappar (Radiobutton)
- Dropdown (OptionMenu)
- Kryssruta (Checkbutton)
- Textfält (flera rader)
- Sammanfattning
- Exempel 1 Filmapp

- Exempel 2 Temperaturomvandling
- Exempel 3 Enkel budgetapp
- Vad är viktigt att förstå
- 12.7 Reflektionsfrågor
- Widgets du behöver kunna
  - Tk (huvudfönster)
  - Label
  - Entry
  - Button
  - Frame
- Input & val
  - Checkbutton (ja/nej)
  - Radiobutton (välja ett alternativ)
  - Text (flera rader)
  - Spinbox (välja tal)
- Listor & dropdown
  - Listbox
  - ttk.Combobox (dropdown)
- Layout & struktur
  - LabelFrame
  - Scrollbar
- Extra
  - Toplevel (nytt fönster)
  - Menu
  - ttk.Notebook (flikar)
  - ttk.Treeview (tabell)
  - Övning – Bygg ett filmregister

Kapitel 13 Programmering, användare och framtid
- 13.1 Vanliga programspråk
- 13.3 Anpassning till användare och tillgänglighet

Kapitel 14 Projekt
- 14.1 Planera ett program
  - Vad innebär planering
  - Exempel
  - En enkel plan
  - Viktiga frågor att ställa
  - Tips
  - Koppling till kursen
  - Övning
- 14.2 Kravspecifikation (VAD)
  - Varför behövs en kravspecifikation

- Vad innehåller en kravspecifikation
- Funktionella krav
- Krav på användaren
- Krav på beteende
- Exempel
- Tips
- Koppling till kursen
- Övning

14.3 Design (HUR)
- Vad innebär design
- Dela upp programmet
- Exempel
- Pseudokod
- En enkel struktur
- Tänk på detta
- Koppling till kursen
- Övning

14.4 Implementering
- Vad innebär implementering
- Arbeta steg för steg
- Exempel
- Testa under tiden
- Vanliga fel
- Tips
- Koppling till kursen
- Övning

Övning
- Krav
- Tips
- Extra utmaning

14.5 Testning
- Varför är testning viktigt
- Vad ska man testa
- Exempel
- Testfall
- Exempel på testfall
- Dokumentera testningen
- Tips
- Koppling till kursen
- Övning

14.6 Dokumentation

Varför behövs dokumentation
Vad kan dokumenteras
Kommentarer i kod
Beskrivning av programmet
Beskrivning av viktiga delar
Vad bra dokumentation kännetecknas av
Tips
Koppling till kursen
Övning
14.7 Redovisning
Varför redovisning är viktigt
Vad en redovisning kan innehålla
Beskriv projektet
Beskriv din planering
Beskriv hur programmet fungerar
Beskriv problem och lösningar
Beskriv testningen
Visa förbättringar
Tips inför redovisning
Koppling till kursen
Övning
Kapitel 15 - Programmering i samhället
Programmering och arbetsliv
Automatisering
Programmering i samhället
Koppling till kursen
15.1 AI och programmering
AI finns redan i samhället
AI och framtidens IT-yrken
Hur IT-yrken kan förändras
Yrken som kan växa
Osäkerhet och försiktighet
Vad som kan bli viktigt framöver
Koppling till kursen
15.2 Etiska frågor
Varför etik är viktigt
Integritet
Rättvisa och bias
Ansvar
Säkerhet
AI och etik

Det är inte alltid enkelt
Exempel
Koppling till kursen
Kapitel 16 Versionshantering och GitHub
16.1 Vad är versionshantering
Commit-kommentarer på engelska
Exempel på bra commit-kommentarer
Viktigt att tänka på
Undvik sådana kommentarer
En enkel modell
16.2 Ladda upp Pythonfiler till GitHub
Vad du behöver
Skapa ett repository
Ladda upp en Pythonfil
Exempel
Ladda upp en ny version av samma fil
Redigera filer direkt på GitHub
Vad du ska ladda upp
Arbeta steg för steg
Vanliga misstag
README.md
Vad betyder README
Vad betyder .md
Varför README är viktig
README ska skrivas på engelska
Vad kan stå i en enkel README
Exempel på innehåll
Vad som passar i den här kursen
Tips
17 Gör en Tkinter-app till en Mac-app med PyInstaller
1. Kontrollera att programmet fungerar
2. Öppna Terminal
3. Installera PyInstaller
4. Kontrollera att PyInstaller är installerat
5. Bygg appen
6. Hitta appen
7. Starta appen
8. Bygg med eget namn
9. Lägg till ikon
10. Viktigt för macOS
11. Använd inte detta som första val på Mac

12. Om appen inte fungerar
13. Om programmet använder bilder eller andra filer
14. Enkel standardmodell
15. Sammanfattning
Exempel
Bra att veta
18 Arbetssätt i projekt kopplat till Teknik 2
Innan projektet
Under projektet
Efter projektet
Kort checklista
Appendix - förslag till projekt

1- Vad är programmering

Programmering innebär att skriva instruktioner som en dator kan följa. Dessa instruktioner kallas **program**. Ett program består av kod som talar om för datorn vad den ska göra och i vilken ordning saker ska göras.

I detta kapitel introduceras:

- vad ett program är
- hur datorer kör instruktioner
- hur programmering används i vardagen
- hur man skriver sitt första program i Python

1.1 Vad är ett program

Ett **program** är en serie instruktioner som en dator följer steg för steg.

När en dator kör ett program gör den exakt det som instruktionerna säger. Datorn kan inte gissa vad programmeraren menade. Den gör bara det som står i koden.

Program finns överallt i samhället. Några exempel är:

- mobilappar
- spel
- bankappar
- webbplatser
- bilar
- trafiksystem

- medicinsk utrustning

Alla dessa system innehåller program som styr hur de fungerar.

Ett program kan vara mycket enkelt eller mycket avancerat. I början av kursen kommer vi att skriva små program som löser enkla uppgifter.

1.2 Vad är programmering

Programmering är processen att skapa program.

Det innebär att man:

1. analyserar ett problem
2. planerar en lösning
3. skriver kod
4. testar programmet
5. förbättrar programmet

En programmerare arbetar alltså både med **problemlösning** och med att **skriva kod**.

Ett viktigt mål i programmering är att skapa program som fungerar korrekt och är lätta att förstå.

1.3 Datorns funktionssätt och programmeringens möjligheter

För att förstå programmering behöver man också förstå hur datorn arbetar.

En dator tänker inte själv.

Den följer instruktioner exakt.

Det betyder att programmering har stora möjligheter, men också tydliga begränsningar.

Datorn följer instruktioner

När du skriver ett program ger du datorn instruktioner.

Datorn utför dem steg för steg.

Om instruktionerna är korrekta fungerar programmet som tänkt.

Om instruktionerna är fel blir resultatet också fel.

Exempel på korrekt skriven kod i Python:

```
tal1 = 5
tal2 = 3
summa = tal1 + tal2
print(summa)
```

Möjligheter

Eftersom datorer är snabba och noggranna kan program användas för att

räkna ut resultat
hantera stora mängder information
styra tekniska system
automatisera uppgifter

Det är därför programmering används i många delar av samhället.

Undervisningen ska också leda till att eleverna utvecklar färdigheter i programvaruutveckling och förståelse för hur programvaror och programmerbara system påverkar sin omgivning.

Det betyder att kursen inte bara handlar om att skriva kod, utan också om att förstå hur program används och vilken påverkan de kan ha.

Begränsningar

Datorn förstår inte vad du menar.

Den förstår bara det du faktiskt skriver.

Om du stavar fel, använder fel variabel eller ger otydliga instruktioner kan programmet ge fel resultat eller krascha.

Det betyder att kod måste vara tydlig, korrekt och testad.

Sammanfattning

Datorn är snabb och noggrann, men den tänker inte själv.

Den följer instruktioner exakt.

Därför kan programmering användas till mycket, men bara om programmet är tydligt och rätt skrivet.

1.4 Programspråk

För att kunna skriva program använder man ett **programspråk**. Ett programspråk är ett språk som är utformat så att både människor och datorer kan arbeta med det.

Människor skriver koden och datorn tolkar instruktionerna och utför dem.

Det finns många olika programspråk. Några vanliga exempel är

- **Python**
- **Java**
- **C#**
- **JavaScript**
- **C++**

Olika programspråk används ofta för olika typer av system. Till exempel används vissa språk ofta för

- webbplatser
- spel
- mobilappar
- dataanalys
- artificiell intelligens

Varför Python

I denna kurs använder vi **Python**.

Python är ett av världens mest använda programspråk och används inom många olika områden.

Python är särskilt bra för nybörjare eftersom

- språket är lätt att läsa
- koden är ofta kort och tydlig
- det finns mycket stöd och många bibliotek

Python används i dag inom bland annat

- webbutveckling
- dataanalys

- artificiell intelligens
- automatisering
- forskning

I denna kurs kommer du att använda Python för att skriva program som löser olika problem och för att lära dig grunderna i programmering.

1.5 Hur ett program körs

När du skriver kod i Python händer flera saker innan resultatet visas på skärmen.

Det kan kännas som att datorn bara “kör programmet direkt”, men i själva verket samarbetar flera delar.

Programmet, Python, operativsystemet och datorns hårdvara arbetar tillsammans.

Från kod till körning

När du skriver ett program i Python sparas koden i en fil, till exempel `hej.py`. Filen innehåller text som människor kan läsa. Datorns processor kan däremot inte köra vanlig Pythonkod direkt. Därför behövs ett program som kan läsa och köra koden. I Python är det Python-tolken som gör detta.

Exekveringsmiljö

Den miljö där ett program körs kallas exekveringsmiljö. För Python består exekveringsmiljön till exempel av

- Python-tolken
- bibliotek som programmet använder
- inställningar i datorn
- de filer som programmet behöver

Exekveringsmiljön är alltså det sammanhang där programmet körs. Om något saknas i miljön kanske programmet inte fungerar.

Operativsystemets roll

Operativsystemet är det system som hanterar datorns grundläggande funktioner. Exempel på operativsystem är Windows, macOS och Linux. När du startar ett Pythonprogram hjälper operativsystemet till med att

- öppna programmet
- hantera minne
- läsa filer
- visa fönster
- ta emot inmatning från tangentbord och mus

Operativsystemet fungerar alltså som en mellanhand mellan programmet och datorns hårdvara.

Hårdvarans roll

Hårdvara är de fysiska delarna i datorn.

Det kan till exempel vara

- processor
- arbetsminne
- skärm
- tangentbord
- mus

När programmet körs är det hårdvaran som faktiskt utför arbetet. Processorn räknar och följer instruktioner. Minnet lagrar data medan programmet körs. Skärmen visar resultatet. Tangentbordet och musen används för inmatning.

Så går det till

1. När du kör ett Pythonprogram händer detta i förenklad form:
2. Du skriver kod i en fil
3. Operativsystemet startar Python
4. Python läser koden
5. Instruktionerna körs steg för steg
6. Operativsystemet hjälper till med filer, fönster och inmatning
7. Hårdvaran utför det praktiska arbetet
8. Resultatet visas för användaren

Exempel

Titta på detta program:

```
namn = input("Vad heter du? ")
print(f"Hej {namn}")
```

1. När programmet körs händer bland annat detta:
2. Python läser raden med `input()`
3. Operativsystemet visar texten på skärmen

4. Användaren skriver på tangentbordet
5. Operativsystemet skickar inmatningen vidare
6. Python sparar texten i variabeln `namn`
7. Sedan körs `print()` och resultatet visas på skärmen

Varför detta är viktigt att förstå

Det är bra att förstå hur ett program körs, eftersom det hjälper dig att förstå varför problem kan uppstå.

Ett program kan till exempel sluta fungera om

- Python inte är installerat
- en fil saknas
- ett bibliotek inte finns
- operativsystemet inte hittar rätt fil
- programmet får fel inmatning

Problemet behöver alltså inte alltid finnas i själva koden. Ibland handlar det om miljön runt programmet.

1.6 Kompilerade och interpreterade programspråk

Programsprog fungerar inte alltid på samma sätt när ett program körs. En viktig skillnad mellan programspråk är **hur koden översätts till något datorn kan förstå**.

Datorns processor kan bara köra **maskinkod**, alltså mycket enkla instruktioner som består av ettor och nollor. Därför måste koden i ett programspråk först översättas till något datorn kan köra.

Det finns två vanliga sätt att göra detta.

- kompilerade programspråk
- interpreterade programspråk

Kompilerade programspråk

I ett **kompilerat programspråk** översätts hela programmet till maskinkod innan programmet körs.

Detta görs av ett program som kallas **kompilator**.

Processen fungerar ungefär så här

1. Programmeraren skriver koden
2. En kompilator översätter hela programmet
3. Resultatet blir ett körbart program
4. Programmet kan sedan startas

Exempel på kompilerade programspråk är

- **C**
- **C++**
- **Rust**
- **Go**

En fördel med kompilerade språk är att program ofta körs **mycket snabbt**.

En nackdel är att man ofta måste **kompilera programmet igen** varje gång man ändrar något i koden.

Interpreterade programspråk

I ett **interpreterat programspråk** översätts och körs koden **rad för rad** när programmet startas.

Det görs av ett program som kallas **interpreter**.

Processen fungerar ungefär så här

1. Programmeraren skriver koden
2. Programmet startas
3. Interpretern läser en rad kod
4. Raden översätts och körs direkt

Exempel på interpreterade språk är

- **Python**
- **JavaScript**
- **Ruby**

En fördel med interpreterade språk är att det går **snabbt att testa och ändra program**.

En nackdel är att program ibland kan bli **lite långsammare** än kompilerade program.

Python och tolkning

Python är ett **interpreterat programspråk**.

Det betyder att när du kör ett Pythonprogram läser Python-programmet (interpretern) din kod och kör den steg för steg.

Till exempel

```
print("Hej världen")
```

När programmet körs läser Python instruktionen och utför den direkt.

Sammanfattning

Kompilerade språk

- hela programmet översätts först
- resultatet blir ett körbart program
- program körs ofta mycket snabbt

Interpreterade språk

- koden körs rad för rad
- ingen separat kompilering behövs
- lätt att testa och ändra kod

I denna kurs använder vi **Python**, som är ett interpreterat programspråk.

1.7 Installera Python på macOS

För att kunna skriva och köra Pythonprogram behöver Python vara installerat på datorn. De flesta Mac-datorer har redan en version av Python installerad, men ofta är den äldre och används av operativsystemet. I denna kurs använder vi **Python 3**, därför installerar vi den senaste versionen.

Steg 1 – Ladda ner Python

1. Öppna en webbläsare.
2. Gå till adressen

https://www.python.org

3. Klicka på **Downloads**.
4. Klicka på knappen **Download Python 3**.X.X

   Python uppdateras hela tiden så när du läser detta så kan det vara en senare version

Webbplatsen väljer automatiskt rätt version för macOS.

Steg 2 – Installera Python

1. Öppna filen som laddades ner.
2. Starta installationsprogrammet.
3. Klicka på **Continue** i varje steg.
4. Slutför installationen.

När installationen är klar finns Python installerat på datorn. Samtidigt installeras även programmet **IDLE**, som är en enkel miljö för att skriva och köra Pythonkod.

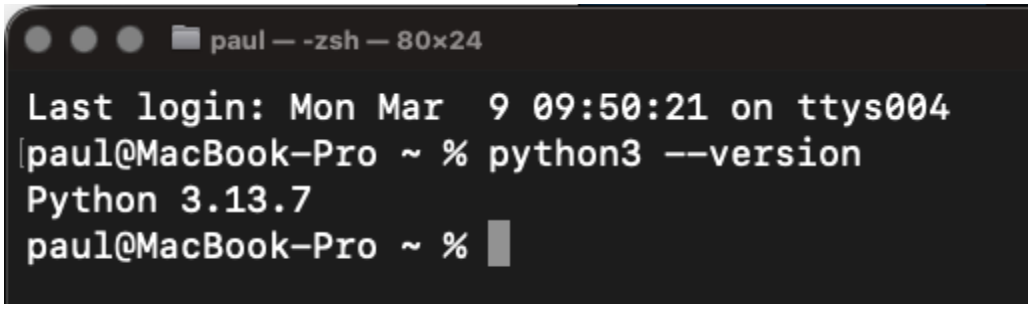
Steg 3 – Kontrollera installationen

Det är bra att kontrollera att Python fungerar.

1. Öppna programmet **Terminal**.
2. Skriv följande kommando

python3 --version

3. Tryck på **Enter**.



Python uppdateras hela tiden så när du läser detta så kan det vara en senare version

Om installationen fungerar visas ett meddelande som liknar. Det kan vara en senare version när du instalelrar det.

```
Python 3.14.3
```

Det betyder att Python är installerat och fungerar korrekt.

Hitta Python på din Mac

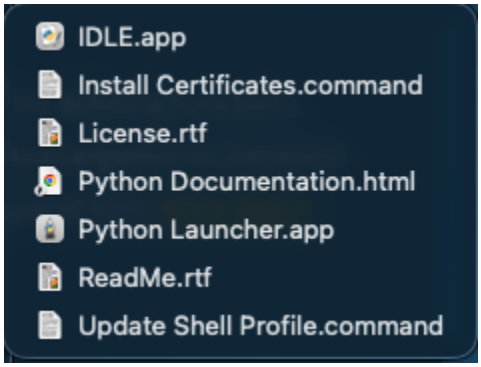
När Python installeras skapas flera filer på datorn. Ibland kan det vara bra att veta var Python finns installerat.

Det enklaste sättet är att använda **Finder**.

1. Öppna **Finder**.
2. Gå till mappen **Applications (Program)**.
3. Leta efter mappen **Python 3.14.3 (eller senare version)**.

I denna mapp finns bland annat

- **IDLE**
- **Python Launcher**
- **Install Certificates.command**
- dokumentation för Python



Härifrån kan du starta **IDLE** direkt.

Om du installerat Python från python.org finns programmen vanligtvis i mappen

Applications → Python 3.x

Du kan också hitta Python genom att använda **Spotlight**.

1. Tryck **Cmd + mellanslag**.
2. Skriv **Python** eller **IDLE**.
3. Starta programmet därifrån.

Vanliga problem

Om Terminalen visar ett felmeddelande kan det bero på att Python inte installerades korrekt. I så fall kan du försöka installera programmet igen från python.org.

Nästa steg

Nu när Python är installerat ska vi lära oss att skriva och köra Pythonprogram i programmet **IDLE**. Det gör vi i nästa avsnitt.

1.8 Skriva och köra program i IDLE

När Python installeras följer programmet **IDLE** med. IDLE är en enkel utvecklingsmiljö där du kan skriva och köra Pythonkod. Den är särskilt bra när man börjar lära sig programmering.

I detta avsnitt lär du dig

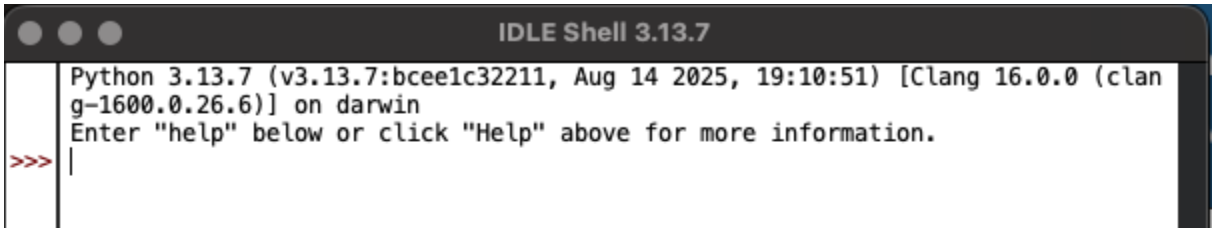
- hur du startar IDLE
- hur du skriver ett Pythonprogram
- hur du sparar programmet
- hur du kör programmet

Starta IDLE

För att starta IDLE på macOS gör du så här

1. Öppna **Spotlight** genom att trycka på **Cmd + mellanslag**.
2. Skriv **IDLE**.
3. Starta programmet **IDLE (Python 3)**.

När IDLE startar öppnas ett fönster som kallas **Python Shell**.



I detta fönster kan man skriva Pythonkod direkt och köra den direkt.

Python Shell

Python Shell fungerar ungefär som en räknare för Python. Du skriver en instruktion och Python kör den direkt.

Prova att skriva

```
print("Hej")
```

Tryck sedan på **Enter**.

Python skriver då ut

```
Hej
```

Detta visar att Python fungerar.

Skapa ett Pythonprogram

Även om man kan skriva kod direkt i Python Shell är det vanligaste att man skriver program i en fil.

Gör så här

1. Klicka på **File** i menyn.
2. Välj **New File**.

Ett nytt fönster öppnas. Detta fönster används för att skriva programkod.

Skriv ditt program

Skriv följande kod i det nya fönstret

```
print("Hej världen")
```

Detta är ett mycket enkelt Pythonprogram som skriver ut en text på skärmen.

Spara programmet

Innan programmet kan köras måste det sparas.

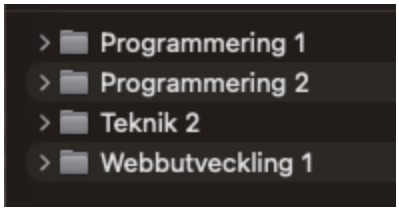
1. Klicka på **File**.
2. Välj **Save**.
3. Ge filen ett namn, till exempel

`hej.py`

OBS En kort paus om mappstruktur. Kanske du redan har en mapp i andra kurser.

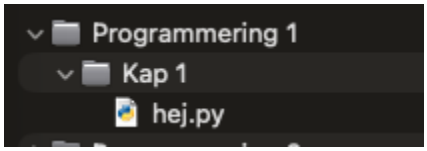
För att lyckas hålla reda på flera kurser och alla olika projekt i varje kurs ska du skapa mappar för varje kurs under Dokument på din Mac.

Har du inte gjort det innan så blir det en mapp som heter Programmering 1 under Dokument-mappen



I mappen Programmering 1 skapa en ny mapp som heter “kap 1”

När du är klar ska det se ut så här



Pythonprogram sparas alltid med filändelsen **.py**.

Kör programmet

När programmet är sparat kan det köras.

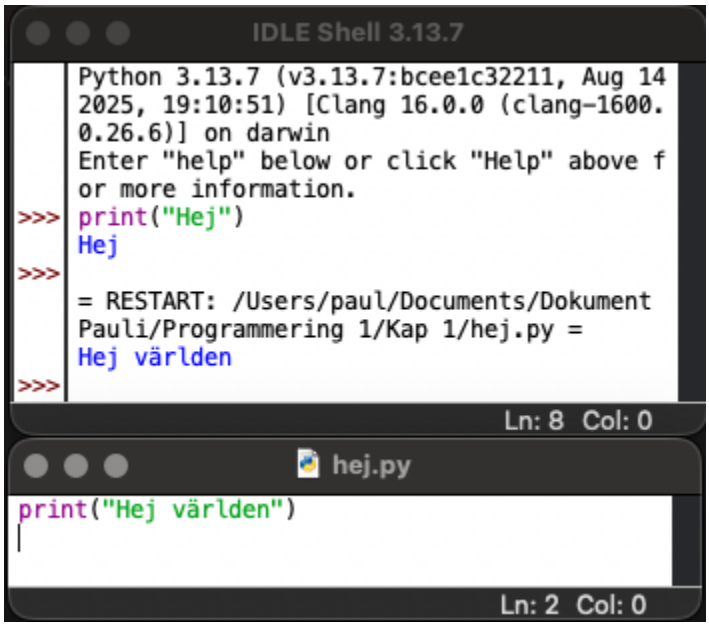
1. Klicka på **Run** i menyn.
2. Välj **Run Module**.

Du kan också använda kortkommandot **F5**. OBS på Mac måste du hålla ned fn-tangetn för att F5 ska kungera

Programmet körs då och resultatet visas i **Python Shell**.

Du ska nu se

```
Hej världen
```



Ändra programmet

Prova att ändra texten i programmet.

Till exempel

```
print("Jag lär mig Python")
```

Spara programmet igen och kör det en gång till.

Sammanfattning

I detta avsnitt har du lärt dig att

- starta IDLE
- skriva Pythonkod
- spara ett program
- köra ett program

I nästa avsnitt ska vi skriva vårt första riktiga Pythonprogram och börja förstå hur kod fungerar.

1.9 Installera Visual Studio Code

Visual Studio Code (VS Code) är ett program där man kan skriva och köra kod. Det används av många programmerare eftersom det är snabbt, enkelt att använda och fungerar med många olika programspråk.

Om du **redan har VS Code installerat på din dator behöver du inte installera det igen**. Du kan då gå vidare till nästa avsnitt.

Installera VS Code

1. Öppna en webbläsare.
2. Gå till adressen

https://code.visualstudio.com

3. Klicka på **Download for macOS**.
4. När filen laddats ner, öppna den.
5. Dra **Visual Studio Code** till mappen **Applications (Program)**.

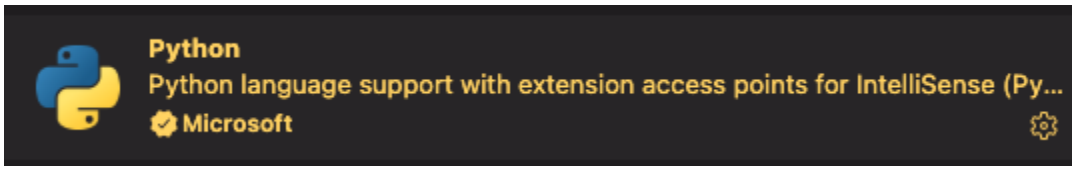
Starta sedan programmet från **Applications** eller genom **Spotlight** (Cmd + mellanslag och skriv VS Code).

1.10 Programmera i VS Code med Python

För att kunna skriva Pythonkod i VS Code behöver man installera ett **Python-tillägg**.

Installera Python-tillägget

1. Starta **VS Code**.
2. Klicka på **Extensions** i vänstermenyn (ikonen med fyra rutor).
3. Skriv **Python** i sökfältet.
4. Installera tillägget **Python** från Microsoft.



När tillägget är installerat kan VS Code arbeta med Pythonprogram.

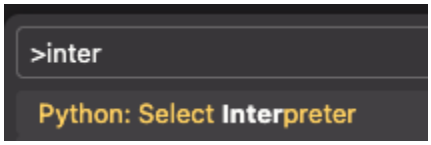
Välj Python-version

VS Code behöver veta vilken Python-installation som ska användas.

1. Tryck **Cmd + Shift + P**.

2. Skriv

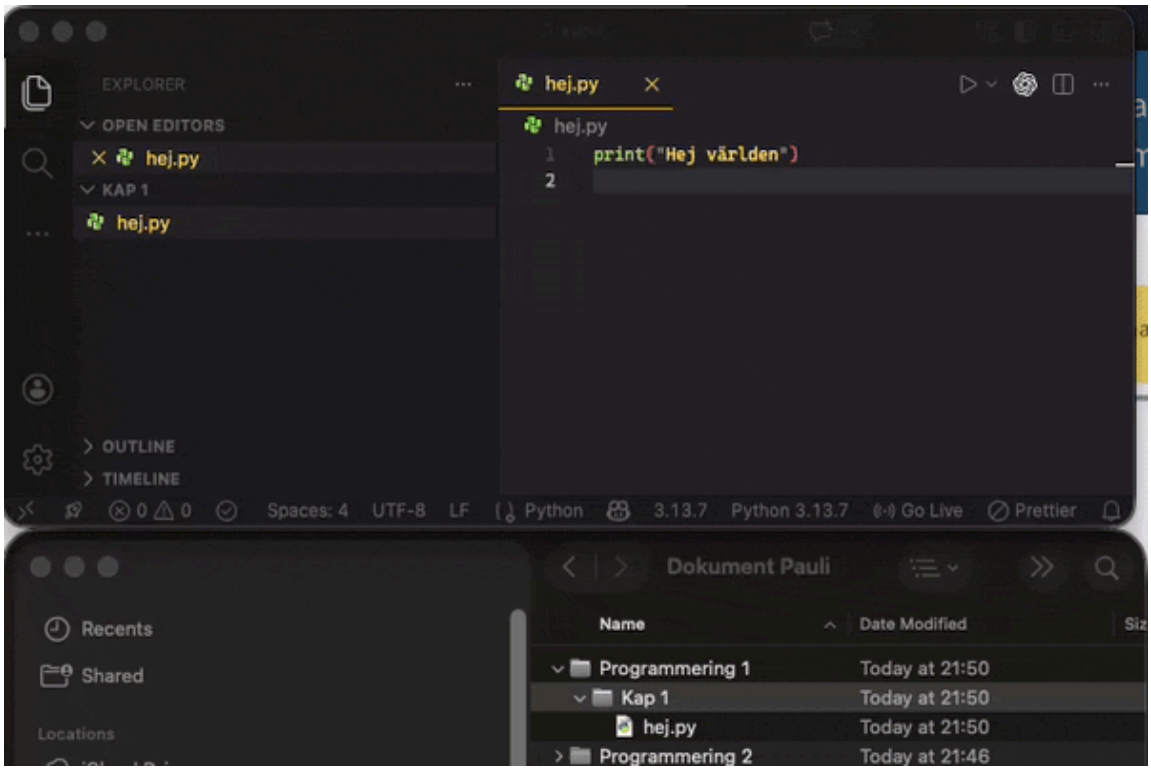
Python: Select Interpreter



3. Välj den Python-version som installerades tidigare, till exempel

Python 3.14.3

1. Dra mappen Kap 1 från Finder in i VS Code-fönstret.



Kör programmet

Det finns flera sätt att köra programmet.

Det enklaste är att klicka på **Run**-knappen uppe till höger i VS Code.

Programmet körs då i terminalen och resultatet visas längst ner i fönstret.

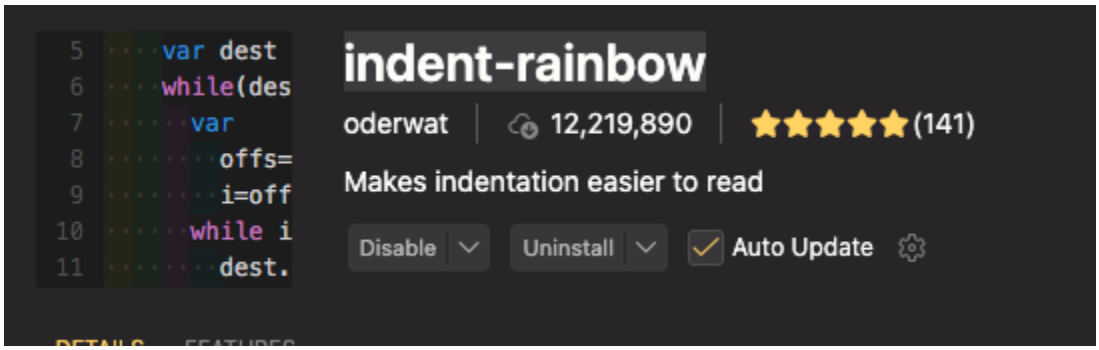
Du ska då se

```
Hej världen
```

Nu kan du skriva och köra Pythonprogram direkt i VS Code.

Använda extentions

Jag rekomenderar att man installerar detta tillägg
indent-rainbow



Det gör indragen med tab tydliga

1.11 Syntax och noggrannhet i kod

När man programmerar är det viktigt att skriva koden **precis rätt**. Datorn kan inte tolka vad vi menar om koden är felaktig. Den följer bara de instruktioner som står i programmet.

Därför måste programmerare vara **noggranna** när de skriver kod.

Vad är syntax

Syntax är reglerna för hur kod ska skrivas i ett programspråk. Varje programspråk har sina egna regler.

Om syntaxen är fel kan programmet inte köras.

Till exempel är detta korrekt Pythonkod

```
print("Hej världen")
```

Om man glömmer citattecken blir det fel

```
print(Hej världen)
```

Python förstår då inte vad texten betyder och visar ett felmeddelande.

```
paul@MacBook-Pro Kap 1 % /usr/local/bin/python3 "/Users/paul/Documents/Dokument
 Pauli/Programmering 1/Kap 1/hej.py"
  File "/Users/paul/Documents/Dokument Pauli/Programmering 1/Kap 1/hej.py", lin
e 1
    print(Hej världen)
          ^^^^^^^^^^^
SyntaxError: invalid syntax. Perhaps you forgot a comma?
```

Indrag i Python

I många programspråk används klamrar för att visa vilka instruktioner som hör ihop. I Python används istället **indrag**.

Till exempel

```
if True:
      print("Detta skrivs ut")
```

Indraget visar att raden hör till instruktionen ovanför.

Gör indraget med tab-tangenten. Det är mycker viktigt att du kommer i håg indraget med tab på nästa rad efter if.

Om indraget saknas eller är fel kan Python visa ett felmeddelande.

```
if True:
      print("Detta skrivs ut")
```

```
    print("Detta skrivs ut")
    ^^^^^
IndentationError: expected an indented block after 'if' statement on line 1
```

Vanliga misstag

Små misstag kan göra att programmet inte fungerar. Några vanliga exempel är

- glömda citattecken
- felstavade kommandon
- saknade parenteser
- felaktiga indrag

Till exempel

```
prin("Hej")
```

```
NameError: name 'prin' is not defined. Did you mean: 'print'?
```

Här är kommandot **print** felstavat och programmet fungerar inte.

Felmeddelanden

När något är fel i koden visar Python ett **felmeddelande**.

Felmeddelandet hjälper dig att hitta problemet. Det visar ofta

- vilken rad som innehåller felet
- vilken typ av fel det är

Det är därför viktigt att **läsa felmeddelanden noggrant**.

Var noggrann när du skriver kod

När du programmerar är det viktigt att

- skriva kommandon korrekt
- använda rätt citattecken och parenteser
- använda rätt indrag
- läsa felmeddelanden

Att vara noggrann gör det mycket lättare att hitta och lösa problem i program.

Sammanfattning

Syntax är reglerna för hur kod ska skrivas i ett programspråk. Om syntaxen är fel kan programmet inte köras.

I Python är det särskilt viktigt att

- skriva kommandon korrekt
- använda rätt parenteser och citattecken
- använda rätt indrag

Genom att läsa felmeddelanden och kontrollera koden noggrant kan man hitta och rätta fel.

1.12 Text, radbrytningar och specialtecken i Python

När man skriver text i Python använder man oftast print(). Det gör att texten visas i terminalen.

```
print("Hej världen")
```

Det som står mellan citattecknen skrivs ut exakt som text.

Vanliga specialtecken

I Python finns några tecken som används för att påverka **formateringen** av text.
De kallas ofta **escape-tecken** eller **specialtecken**.

`\n` – ny rad

`\n` betyder att texten fortsätter på nästa rad.

Exempel:

```
print("Hej\nvärlden")
```

Det skrivs ut så här:

Hej
världen

`\t` – tab

`\t` betyder **tabbtecken**.
Det används för att skapa ett mellanrum som är större än ett vanligt mellanslag.

Exempel:

```
print("Namn\tÅlder")
print("Anna\t16")
```

Det kan skrivas ut ungefär så här:

```
Namn        Ålder
Anna        16
```

Det är användbart när man vill göra enkel uppradning av text.

`\"` – citattecken i text

Om du vill skriva ut citattecken inne i en textsträng kan du använda `\"`.

Exempel:

```
print("Han sa: \"Hej!\"")
```

Utskrift:

```
Han sa: "Hej!"
```

`\\` – backslash

Om du vill skriva ut ett vanligt omvänt snedstreck använder du `\\`.

Exempel:

```
print("C:\\Python\\MinaFiler")
```

Utskrift:

```
C:\Python\MinaFiler
```

Exempel med flera specialtecken

```
print("Övning 1 – Skriv ut text")
print("Hej världen")
print("Hej\nigen")
print("Namn\tKlass")
print("Paul\tTE23")
print("Han sa: \"Python är kul\"")
```

Viktigt att tänka på

Specialtecken fungerar bara **inne i textsträngar**, alltså mellan citattecken.
Om du skriver fel kan programmet ge ett felmeddelande eller skriva ut något annat än du tänkt.

Sammanfattning

Några vanliga specialtecken i Python är:

- `\n` = ny rad
- `\t` = tab
- `\"` = citattecken

- `\\` = backslash

De används för att styra hur text visas när programmet körs.

1.13 Programmering i vardagen

Programmering finns i många delar av samhället och i många av de saker vi använder varje dag. Många av de tjänster och appar som människor använder bygger på program som någon har skrivit.

Programmering används till exempel i

- mobilappar
- webbsidor
- datorspel
- banktjänster
- sociala medier
- navigationssystem
- smarta hem

När du använder en app på din telefon eller en tjänst på internet är det alltid ett program som ligger bakom.

Programmering i appar och webbtjänster

Många av de appar vi använder varje dag är byggda med hjälp av programmering. Till exempel behöver en app kunna

- visa information
- ta emot inmatning från användaren
- spara data
- kommunicera med servrar

Till exempel använder en väderapp program som hämtar data från vädertjänster och visar prognoser.

Programmering i spel

Datorspel är också ett tydligt exempel på programmering. I ett spel används programkod för att styra

- hur figurer rör sig
- hur poäng räknas
- hur spelvärlden fungerar

- hur spelaren kan interagera med spelet

Många elever möter programmering första gången genom spelutveckling.

Programmering i teknik och maskiner

Programmering används också i många tekniska system.

Exempel är

- robotar i fabriker
- trafikljus
- bilar
- medicinsk utrustning
- flygplan

I dessa system används program för att styra hur maskiner ska bete sig.

Programmering och automatisering

Ett viktigt användningsområde för programmering är **automatisering**. Det innebär att datorer utför uppgifter automatiskt istället för att människor gör dem manuellt.

Exempel på automatisering är

- sortering av paket
- analys av data
- bokningssystem
- tradingbotar
- rekommendationer i streamingtjänster

Automatisering kan göra arbete snabbare och mer effektivt.

Programmering och samhället

Programmering påverkar också samhället på många sätt. Algoritmer används till exempel för att

- rekommendera filmer och musik
- visa annonser
- sortera information
- analysera stora datamängder

Detta kan ha både positiva och negativa effekter. Därför är det viktigt att förstå hur programmerade system fungerar.

Sammanfattning

Programmering används i många delar av vardagen. Den finns i appar, spel, webbtjänster och tekniska system.

Genom att lära sig programmering får man en bättre förståelse för hur många av de digitala system som används i samhället fungerar.

1.14 Att tänka som en programmerare

Att programmera handlar inte bara om att skriva kod. Det handlar också om att **tänka på ett särskilt sätt** när man löser problem. Programmerare måste kunna analysera problem, dela upp dem i mindre delar och beskriva lösningar steg för steg.

Detta sätt att tänka kallas ibland **algoritmiskt tänkande**.

Dela upp problem

Stora problem kan ofta vara svåra att lösa direkt. Därför brukar programmerare dela upp problemet i **mindre delar**.

Till exempel kan ett program som räknar ut ett medelvärde delas upp i flera steg

1. läsa in tal
2. lägga ihop talen
3. räkna hur många tal som finns
4. dela summan med antalet

Genom att dela upp problemet blir lösningen lättare att förstå.

Tänk steg för steg

En dator kan inte gissa vad vi menar. Därför måste varje steg i ett program vara tydligt.

Programmerare måste därför tänka

- vad ska hända först
- vad ska hända sedan
- vad ska hända sist

Till exempel

1. fråga användaren efter ett namn
2. spara namnet i en variabel
3. skriva ut en hälsning

Testa och förbättra

När man programmerar fungerar inte allt rätt direkt. Det är normalt att behöva testa sitt program och ändra koden flera gånger.

Programmerare arbetar därför ofta i en process där man

- skriver kod
- testar programmet
- hittar fel
- förbättrar lösningen

Vara noggrann

Eftersom datorer följer instruktioner exakt måste programmerare vara noggranna. Små misstag kan göra att programmet inte fungerar.

Det är därför viktigt att

- skriva kod tydligt
- använda bra namn på variabler
- testa programmet ofta

Sammanfattning

Att tänka som en programmerare innebär att

- dela upp problem i mindre delar
- tänka steg för steg
- testa och förbättra lösningar
- vara noggrann när man skriver kod

Dessa arbetssätt hjälper programmerare att skapa program som fungerar och är lätta att förstå.

1.15 Övningar

I dessa övningar ska du skriva enkla Pythonprogram. Målet är att öva på att skriva kod, köra program och vara noggrann med syntax.

Skriv varje program i en egen **.py-fil** i din projektmapp.

Övning 1 – Skriv ut text

Skriv ett program som skriver ut texten

Hej världen

Kör programmet och kontrollera att texten visas i terminalen.

Övning 2 – Flera rader

Skriv ett program som skriver ut tre rader

Hej
Jag lär mig Python
Det är roligt att programmera

Övning 3 – Ändra texten

Ändra programmet så att det skriver ut

Hej jag heter [ditt namn]

Till exempel

Hej jag heter Anna

Övning 4 – Favoritsak

Skriv ett program som skriver ut tre saker du tycker om.

Till exempel

Jag tycker om programmering
Jag tycker om musik
Jag tycker om spel

Övning 5 – Använd flera print

Skriv ett program som använder flera **print()** för att skriva ut följande

Python
är
Roligt

Övning 6 – Gör ett fel

Skriv följande kod

```
print("Hej världen"
```

Kör programmet.

Vad händer?

Läs felmeddelandet och fundera på vad som är fel.

Övning 7 – Rätta felet

Rätta felet i programmet från övning 6 så att programmet fungerar igen.

1.15 Reflektionsfrågor

Reflektera över frågorna nedan. Diskutera gärna med en klasskamrat eller skriv ner dina svar.

1 - Vad är ett program? Beskriv med egna ord.

2 - Varför måste man vara noggrann när man skriver kod?

3 - Vad menas med syntax i ett programspråk?

4 - Vad är skillnaden mellan ett kompilerat och ett interpreterat programspråk?

5 - Ge tre exempel på var programmering används i vardagen.

6 - Varför är det viktigt att läsa felmeddelanden när ett program inte fungerar?

7 - Vad menas med att tänka steg för steg när man programmerar?

8 - Varför är det viktigt att förstå koden även om man använder AI-verktyg?

9 -Vilka saker har du lärt dig i detta kapitel som du inte visste innan?

2 - Variabler, datatyper och input

I detta kapitel lär du dig hur program lagrar information och hur program kan ta emot information från användaren. Detta är grunden för nästan all programmering.

2.1 Vad är en variabel

När ett program körs behöver det kunna lagra information. Den informationen sparas i något som kallas variabler.

En variabel är en plats i datorns minne där ett värde lagras. Variabeln har ett namn så att programmet kan använda värdet senare.

Man kan tänka på en variabel som en låda med en etikett. Etiketten är variabelns namn och i lådan ligger värdet.

```
name = "Anna"
```

Här skapas en variabel som heter `name`. Värdet som lagras i variabeln är texten `"Anna"`.

Programmet kan sedan använda variabeln senare i koden.

Variabler gör det möjligt att

- lagra information
- ändra värden
- använda värden flera gånger i ett program

2.2 Tilldela värden

När man ger en variabel ett värde kallas det att tilldela ett värde.

Det görs med likhetstecknet =.

```
name = "Anna"
age = 17
```

Här skapas två variabler.

Variabel	Värde
`name`	`Anna`
`age`	`17`

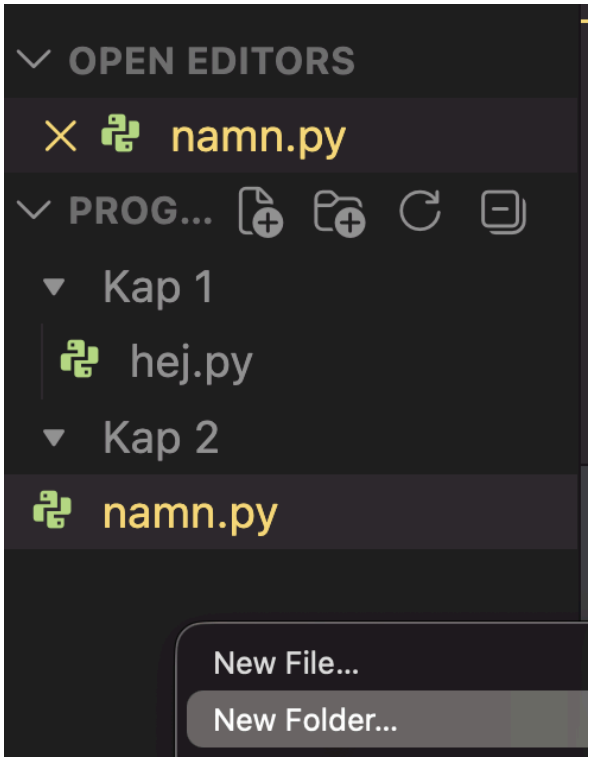
Programmet kan nu använda dessa värden i senare instruktioner.

Det är viktigt att komma ihåg att = i programmering betyder tilldela ett värde, inte matematiskt lika med.

Testa nu att skriva in dessa koder i IDLE eller VS Code. I fortsättningen kommer jag skriva din kodeditor, editor eller IDE.

Skapa en ny mapp som heter kap 2 och i den skapa en fil du kallar namn.py

En ny mapp gör du genom att två fingerklicka på macplattan eller högerhögerklicka på datormusen. På svenska heter det sekundärklick på engelska *context click.*



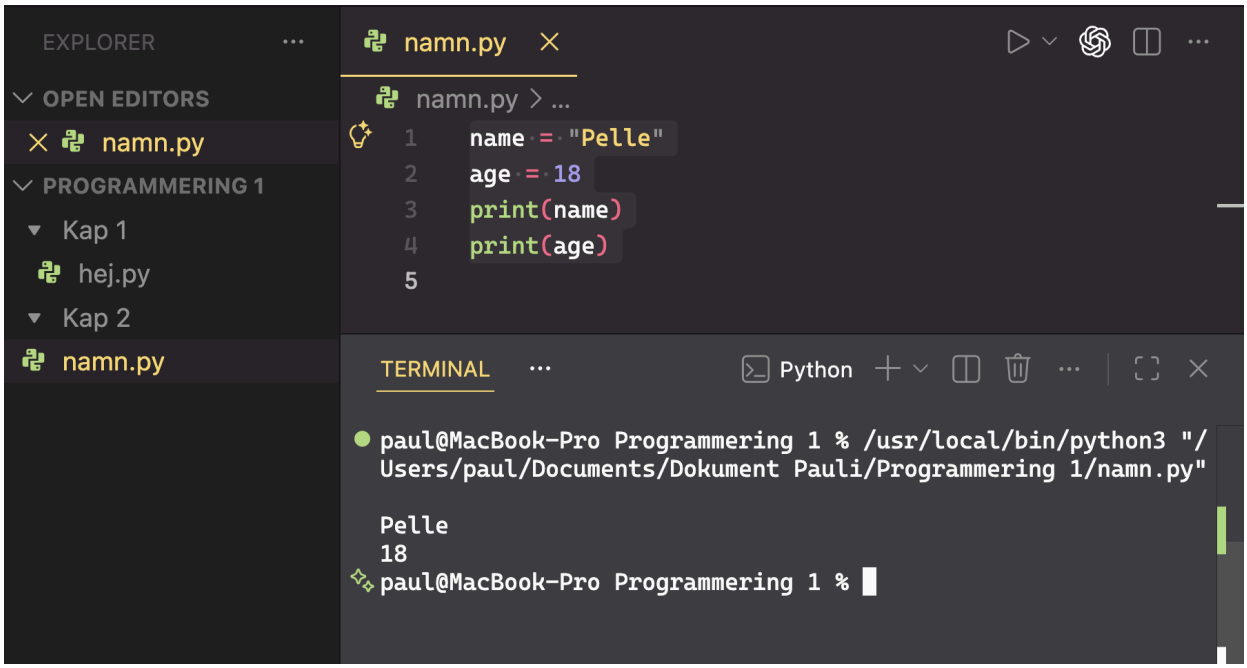
I den skriver du tex

```
name = "Pelle"
age = 18
print(name)
```

```
print(age)
```

Och kör programmet

För att rensa i Terminalen skriv clear och klicka på Enter



2.3 Datatyper

Information som lagras i variabler kan ha olika **datatyper**. Datatypen beskriver vilken typ av värde som lagras.

De vanligaste datatyperna i Python är:

`int` som är heltal

```
age = 17
```

`float` som är decimaltal

```
temperature = 21.5
```

`str` som är text

```
name = "Anna"
```

`bool` som är sant eller falskt

```
student = True
```

Datatyper är viktiga eftersom Python behöver veta hur värden ska behandlas.

Övning 1

Skapa en ny fil i din projektmapp.

Filen ska heta - datatyper.py

Skriv följande kod i filen.

```
age = 17
temperature = 21.5
name = "Anna"
student = True


print(type(age))
print(type(temperature))
print(type(name))
print(type(student))
```

Kör programmet.

Du kommer att se vilken datatyp varje variabel har.

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Övning 2

Skapa fyra nya variabler i din fil.

En variabel för

- ditt namn
- din ålder
- din längd
- om du är student (True eller False)

Skriv sedan ut datatyperna med `type()`.

Övning 2

Testa att skriva följande kod

```
print(type(10))
print(type(3.14))
print(type("Hej"))
print(type(True))
```

Vad blir datatyperna?

Som du har sett kan man i Python kan man kontrollera vilken datatyp ett värde har. Detta görs med funktionen type().

```
age = 17
print(type(age))
```

```
<class 'int'>
```

Det betyder att variabeln innehåller ett heltal.

Att använda `type()` är mycket användbart när man lär sig programmering. Det hjälper dig att

- förstå vilken datatyp ett värde har
- kontrollera att ett program fungerar som du tänkt
- hitta fel i din kod
  Till exempel kan man kontrollera datatyper så här

```
print(type(10))
print(type(3.14))
print(type("Hej"))
print(type(True))
```

2.4 Skriva ut variabler

När man programmerar behöver man ofta visa information på skärmen. I Python görs detta med funktionen **`print()`**.

Funktionen `print()` används för att skriva ut text eller värden i terminalen.

Exempel

```
name = "Anna"
print(name)
```

Programmet skriver då ut

Anna

Här händer två saker:

1. Variabeln `name` skapas och får värdet `"Anna"`.
2. Funktionen `print()` skriver ut värdet som finns i variabeln.

Skriva ut text och variabler tillsammans

Man kan också skriva ut text tillsammans med variabler.

Exempel

```
name = "Anna"
print("Hej", name)
```

Programmet skriver då ut

```
Hej Anna
```

Python sätter automatiskt ett mellanslag mellan värdena som skrivs ut.

Skriva ut flera variabler

Man kan skriva ut flera variabler samtidigt.

Exempel

```
name = "Anna"
age = 17
print(name, age)
```

Resultat

```
Anna 17
```

F-strängar

Ett annat sätt att skriva ut variabler tillsammans med text är att använda **f-strängar**.

En f-sträng börjar med bokstaven **f** före citattecknen.

Exempel

```
name = "Anna"
print(f"Hej {name}")
```

Programmet skriver ut

```
Hej Anna
```

Variabeln placeras mellan **{ }** i texten.

Man kan också använda flera variabler.

```
name = "Anna"
age = 17
print(f"{name} är {age} år gammal")
```

Resultat

```
Anna är 17 år gammal
```

F-strängar gör ofta koden **tydligare och lättare att läsa**, därför används de ofta i Python.

Övning 1 – F-strängar

Skapa en ny Pythonfil i din mapp.
Filen kan till exempel heta

```
fstring.py
```

Skriv följande kod

```
name = "Alex"
age = 16
print(f"Hej {name}")
print(f"{name} är {age} år gammal")
```

Kör programmet.

Övning 2 – F-strängar

Ändra programmet så att det använder **dina egna uppgifter**.

Programmet ska skriva ut något i stil med

```
Hej Maria

Maria är 17 år gammal
```

Extra uppgift

Lägg till en ny variabel som innehåller namnet på din favoritmat.

Exempel

```
food = "pizza"
```

Skriv sedan ut en mening med en f-sträng.

Exempel

```
Min favoritmat är pizza
```

2.5 Input från användaren

Hittills har våra program bara skrivit ut information. Men program kan också **ta emot information från användaren**. Detta gör program mer interaktiva.

I Python används funktionen `input()` för att läsa in text från användaren.

Exempel

```
name = input("Vad heter du? ")
```

När programmet körs visas frågan

Vad heter du?

Programmet väntar då på att användaren ska skriva något och trycka på Enter.

Om användaren skriver

Erik

sparas texten **Erik** i variabeln `name`.

Använda input tillsammans med print

När informationen har sparats i en variabel kan den användas senare i programmet.

Exempel

```
name = input("Vad heter du? ")
print(f"Hej {name}")
```

Om användaren skriver

Anna

skriver programmet ut

Hej Anna

Flera frågor

Ett program kan också ställa flera frågor.

Exempel

```
name = input("Vad heter du? ")
city = input("Vilken stad bor du i? ")
print(f"Hej {name}")
print(f"Du bor i {city}")
```

Programmet läser först in namnet och sedan staden.

Viktigt att veta

Information som kommer från `input()` är alltid **text**.
Det betyder att värdet sparas som datatypen **str**.

Senare i kapitlet kommer vi att se hur man kan omvandla text till tal.

Testa själv

Skapa en ny fil i din projektmapp som heter

input_test.py

Skriv följande kod

```
name = input("Vad heter du? ")
age = input("Hur gammal är du? ")
print(f"Hej {name}")
print(f"Du är {age} år gammal")
```

Kör programmet och svara på frågorna.

Övning

Skriv ett program som frågar användaren efter

- namn
- favoritfärg

Programmet ska sedan skriva ut en mening med hjälp av en **f-sträng**.

Exempel på resultat

```
Hej Erik
Din favoritfärg är blå
```

2.6 Typomvandling

I avsnittet om `input()` såg vi att program kan ta emot information från användaren. Men det finns en viktig sak att känna till.

All information som kommer från `input()` sparas som **text**. Datatypen blir alltså **str**.

Exempel

```
age = input("Hur gammal är du? ")
print(type(age))
```

Om användaren skriver

17

kommer programmet att visa

```
<class 'str'>
```

Det betyder att värdet **17 sparas som text**, inte som ett tal.

Det kan skapa problem om man vill göra beräkningar.

Exempel

```
age = input("Hur gammal är du? ")
print(age + 5)
```

Ger felet

```
TypeError: can only concatenate str (not "int") to str
```

Detta fungerar inte eftersom Python inte kan lägga ihop text och tal.

Omvandla text till tal

För att lösa detta använder man **typomvandling**. Typomvandling betyder att man ändrar en datatyp till en annan.

I Python finns funktioner för detta.

De vanligaste är

`int()` omvandlar till heltal

`float()` omvandlar till decimaltal

`str()` omvandlar till text

Exempel med heltal

```
age = int(input("Hur gammal är du? "))
print(age + 5)
```

Här händer tre saker

1. Programmet läser in text från användaren
2. Texten omvandlas till ett heltal med `int()`
3. Värdet sparas i variabeln `age`

Nu fungerar beräkningen.

Om användaren skriver

17

skriver programmet ut

22

Exempel med decimaltal

```
temperature = float(input("Vad är temperaturen? "))
print(temperature + 1)
```

Här omvandlas texten till ett **decimaltal**.

Kontrollera datatypen

Man kan använda `type()` för att kontrollera datatypen.

Exempel

```
age = int(input("Hur gammal är du? "))
print(type(age))
```

Resultatet blir

```
<class 'int'>
```

Testa själv

Skapa en ny fil som heter

`typomvandling.py`

Skriv följande kod

```
age = int(input("Hur gammal är du? "))
print(f"Om fem år är du {age + 5} år gammal")
```

Kör programmet och testa olika värden.

Övning

Skriv ett program som frågar användaren efter

    längd i centimeter

Programmet ska sedan skriva ut

```
Din längd är ___ cm
Om du växer 10 cm blir din längd ___ cm
```

Tips: använd `int()` för att omvandla värdet till ett tal.

2.7 Vanliga nybörjarfel

Vi har redan tagit upp delar av detta, Men det är så viktigt att vi repterar lite igen och tar med några nya saker att tänka på

När man börjar programmera är det vanligt att göra små misstag i koden. Det är helt normalt. Alla programmerare gör fel ibland. Det viktiga är att lära sig **hur man hittar och förstår felen**.

Här är några vanliga nybörjarfel i Python.

Glömda citattecken

Text i Python måste stå inom citattecken.

Rätt kod

```
print("Hej världen")
```

Fel kod

```
print(Hej världen)
```

I den felaktiga koden förstår Python inte vad texten betyder eftersom citattecknen saknas.

Felstavade kommandon

Python känner bara igen kommandon som är **stavade exakt rätt**.

Rätt kod

```
print("Hej")
```

Fel kod

```
prin("Hej")
```

Om ett kommando är felstavat visar Python ett felmeddelande.

Saknade parenteser

Många funktioner i Python använder parenteser.

Rätt kod

```
print("Hej")
```

Fel kod

```
print "Hej"
```

Python kräver parenteser runt det som ska skrivas ut.

Fel datatyp

Ett annat vanligt fel är att försöka använda text som ett tal.

Fel kod

```
age = input("Hur gammal är du? ")
print(age + 5)
```

Här sparas värdet från `input()` som text. Python kan därför inte lägga till talet 5.

Rätt kod

```
age = int(input("Hur gammal är du? "))
print(age + 5)
```

Här omvandlas texten till ett heltal.

Felmeddelanden

När något är fel i ett program visar Python ett **felmeddelande**. Felmeddelandet berättar ofta

- vilken rad felet finns på
- vilken typ av fel det är

Det är därför viktigt att **läsa felmeddelandet noggrant**.

Sammanfattning

Några vanliga nybörjarfel är

- glömda citattecken
- felstavade kommandon
- saknade parenteser
- fel datatyp

Genom att läsa felmeddelanden och kontrollera koden noggrant kan man oftast hitta och rätta felet snabbt.

2.8 Övningar

I dessa övningar ska du använda det du har lärt dig i kapitel 1 och kapitel 2.

Du kommer att öva på

- `print()`
- variabler
- datatyper
- `type()`
- `input()`

- typomvandling
- f-strängar

Skriv varje program i en egen `.py`-fil i din projektmapp.

Övning 1 – Presentera dig

Skriv ett program som skrivovaer ut

Hej jag heter ditt namn

Skapa först en variabel med ditt namn och använd sedan `print()`.

Exempel

```
name = "Anna"
print(f"Hej jag heter {name}")
```

Övning 2 – Datatyper

Skapa fyra variabler

namn
ålder
längd
student (True eller False)

Skriv sedan ut datatypen för varje variabel med `type()`.

Exempel

```
name = "Anna"
age = 17
height = 170
student = True
print(type(name))
print(type(age))
print(type(height))
print(type(student))
```

Övning 3 – Fråga efter namn

Skriv ett program som frågar användaren efter sitt namn och skriver ut en hälsning.

Programmet ska skriva ut

Hej ditt namn

Övning 4 – Ålder

Skriv ett program som frågar användaren efter sin ålder.

Programmet ska skriva ut

Du är ett antal år gammal
Om fem år är du ett antal år gammal

Tips: använd `int()`.

Övning 5 – Längd

Skriv ett program som frågar användaren efter sin längd i centimeter.

Programmet ska skriva ut

Din längd är ett antal centimeter
Om du växer tio centimeter blir din längd ett antal centimeter

Övning 6 – Favoritfilm

Skriv ett program som frågar användaren efter

namn
favoritfilm

Programmet ska skriva ut

Hej ditt namn
Din favoritfilm är filmens namn

Övning 7 – Kontrollera datatyper

Skriv ett program som innehåller följande värden

10
3.14
"Hej"
True

Använd `type()` för att skriva ut vilken datatyp varje värde har.

Övning 8 – Tre frågor

Skriv ett program som frågar användaren efter

namn
 stad
 favoritmat

Programmet ska sedan skriva ut en kort presentation.

Exempel

Hej Erik
 Du bor i Malmö
 Din favoritmat är pizza

2.9 Reflektionsfrågor

Fundera på frågorna nedan. Du kan diskutera dem med en klasskamrat eller skriva ner dina svar.

1 Vad är en variabel? Förklara med egna ord.

2 Varför använder man variabler i program?

3 Vad är en datatyp?

4 Ge exempel på fyra vanliga datatyper i Python.

5 Vad gör funktionen `type()`?

6 Vad gör funktionen `input()`?

7 Varför behöver man ibland använda `int()` när man använder `input()`?

8 Vad är en f-sträng och varför är den användbar?9
 Ge ett exempel på ett enkelt program som använder variabler.

10 Varför är det viktigt att läsa felmeddelanden när ett program inte fungerar?

3 - Problemlösning i programmering

När man programmerar handlar det inte bara om att skriva kod. Det handlar också om att **lösa problem**.

En programmerare måste först förstå vilket problem som ska lösas och sedan skapa ett program som löser problemet steg för steg.

I detta kapitel kommer du att lära dig hur programmerare arbetar när de löser problem. Du kommer att lära dig att

- förstå ett problem
- dela upp problemet i mindre delar
- planera en lösning
- skriva kod som löser problemet

Detta sätt att arbeta kallas ofta **problemlösning i programmering**.

3.1 Vad är ett programmeringsproblem

Ett programmeringsproblem är en uppgift som kan lösas med hjälp av ett program.

Programmet ska ta emot information, göra något med informationen och sedan ge ett resultat.

Ett enkelt exempel kan vara ett program som räknar ut hur gammal en person är om fem år.

Problemet kan beskrivas så här (pseudokod)

```
Fråga användaren efter sin ålder
Räkna ut åldern om fem år
Skriv ut resultatet
```

Detta problem kan lösas med ett program.

```
age = int(input("Hur gammal är du? "))
print(f"Om fem år är du {age + 5} år gammal")
```

Programmeringsproblem finns i många olika områden.

Exempel

- beräkna resultat i ett spel
- sortera information i en databas

- räkna ut priser i en webbshop
- analysera data

När programmerare arbetar börjar de nästan alltid med att **förstå problemet** innan de börjar skriva kod.

Ett bra sätt att tänka är att ställa frågor som

Vad ska programmet göra?
Vilken information behövs?
Vad ska programmet skriva ut?

Genom att förstå problemet tydligt blir det mycket lättare att skriva ett program som fungerar.

3.2 Algoritmer

När man löser ett programmeringsproblem behöver man bestämma **vilka steg som ska utföras** för att nå en lösning. Denna stegvisa beskrivning kallas en **algoritm**.

En algoritm är alltså en **tydlig serie instruktioner som beskriver hur ett problem ska lösas**.

Algoritmer används inte bara i programmering. De finns även i vardagliga situationer där man följer en serie steg.

Exempel är

- ett recept i matlagning
- instruktioner för att bygga en möbel
- vägbeskrivningar

Alla dessa beskriver **vilka steg som ska göras och i vilken ordning**.

Exempel på en enkel algoritm

Tänk dig ett program som ska räkna ut hur gammal en person är om fem år.

Algoritmen kan beskrivas så här

fråga användaren efter ålder
spara åldern i en variabel
lägg till 5
skriv ut resultatet

När algoritmen är tydlig kan den enkelt omvandlas till ett program.

```
age = int(input("Hur gammal är du? "))
print(f"Om fem år är du {age + 5} år gammal")
```

Varför algoritmer är viktiga

Algoritmer hjälper programmerare att

- planera en lösning innan de skriver kod
- förstå hur ett problem ska lösas
- strukturera sina program

Om man börjar skriva kod direkt utan att tänka igenom algoritmen kan programmet lätt bli svårt att förstå eller innehålla fel.

Egenskaper hos en bra algoritm

En bra algoritm ska vara

tydlig
stegvis
logisk
möjlig att följa

Varje steg ska vara enkelt att förstå och utföra.

Sammanfattning

En algoritm är en **steg-för-steg-beskrivning av hur ett problem ska lösas**.

Genom att först formulera en algoritm blir det mycket lättare att skriva ett fungerande program.

3.3 Dela upp problem

Många programmeringsproblem kan vara svåra att lösa direkt. Ett vanligt arbetssätt i programmering är därför att **dela upp problemet i mindre delar**.

När ett stort problem delas upp i mindre delar blir det lättare att förstå och lösa.

Detta arbetssätt används av nästan alla programmerare.

Exempel

Tänk dig att du ska skriva ett program som räknar ut medelvärdet av flera tal.

Om man försöker lösa hela problemet direkt kan det kännas svårt. Men om man delar upp problemet i mindre steg blir det enklare.

Problemet kan delas upp så här

fråga användaren efter ett tal
fråga efter nästa tal
lägg ihop talen
räkna hur många tal som finns
dela summan med antalet
skriv ut resultatet

När problemet är uppdelat i steg blir det mycket lättare att skriva programmet.

Mindre delar i program

I programmering kan man ofta dela upp problem i delar som

- läsa in information
- göra beräkningar
- fatta beslut
- skriva ut resultat

Om man arbetar steg för steg blir programmet ofta

lättare att förstå
lättare att testa
lättare att ändra

Exempel i Python

Här är ett enkelt program där problemet har delats upp i tydliga steg.

```
number1 = int(input("Skriv ett tal: "))
number2 = int(input("Skriv ett tal till: "))
summa = number1 + number2
print(f"Summan är {summa}")
```

Programmet gör följande

läser in två tal
lägger ihop talen
skriver ut resultatet

Sammanfattning

Att dela upp problem är ett viktigt arbetssätt i programmering.

Genom att dela upp ett problem i mindre delar blir det

lättare att förstå problemet
lättare att skriva programmet
lättare att hitta fel i koden

Detta arbetssätt används ofta när programmerare planerar sina program.

3.4 Pseudokod

Innan man börjar skriva kod kan det vara bra att **planera lösningen**. Ett vanligt sätt att göra detta är att använda **pseudokod**.

Pseudokod är ett sätt att beskriva ett program **med vanliga ord och enkla instruktioner** istället för riktig kod.

Det liknar programmering, men man behöver inte följa exakt syntax som i ett programspråk.

Syftet med pseudokod är att

- planera lösningen på ett problem
- tänka igenom vilka steg programmet ska göra
- göra det lättare att skriva koden senare

Exempel

Anta att vi vill skriva ett program som frågar användaren efter ett tal och säger om talet är stort eller litet.

Pseudokoden kan se ut så här

```
läs in ett tal
om talet är större än 10
skriv ut stort tal
annars
skriv ut litet tal
```

När lösningen är planerad kan man skriva programmet i Python.

```
number = int(input("Skriv ett tal: "))
if number > 10:
      print("Stort tal")
```

```
else:
     print("Litet tal")
```

Gör indraget med tab-tangenten. Det är mycker viktigt att du kommer i håg indraget med tab på nästa rad efter if.

Varför pseudokod är användbart

Pseudokod hjälper programmerare att

- fokusera på lösningen istället för syntax
- förstå programmet innan de skriver koden
- planera större program

Det gör också att andra personer lättare kan förstå hur ett program är tänkt att fungera.

Sammanfattning

Pseudokod är en **enkel beskrivning av ett program i steg**.

Man använder vanliga ord istället för riktig kod, men strukturen liknar ett program. När pseudokoden är klar blir det ofta mycket lättare att skriva själva programmet.

3.5 Flödesscheman

Ett flödesschema är en **grafisk bild av ett program**. Det visar programmets steg med hjälp av symboler och pilar.

Flödesscheman används för att

- planera program
- förstå hur ett program fungerar
- förklara lösningar för andra
- hitta fel i programlogik

Flödesscheman är en form av **diagramteknik för problemlösning**

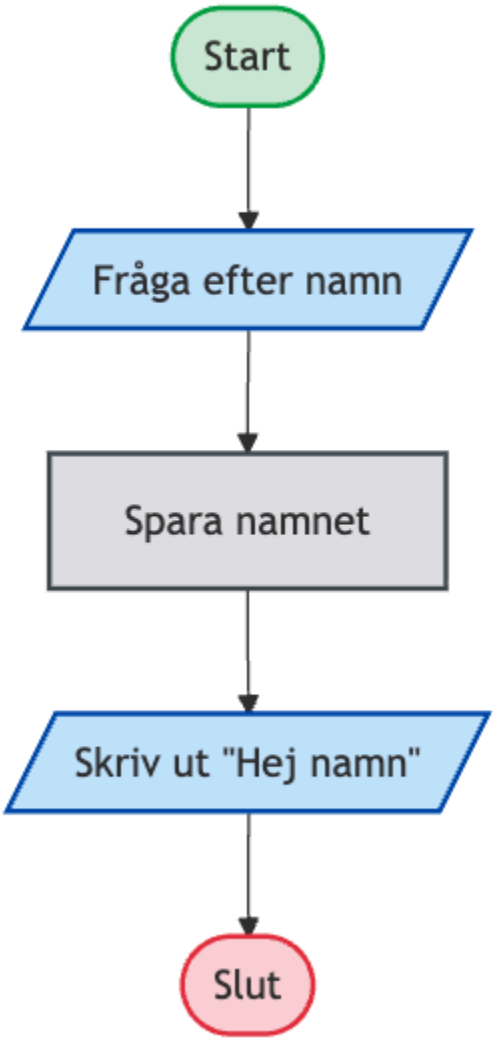
Sekvens

Sekvens betyder att instruktioner utförs i ordning, steg för steg.

Exempel:

läs in namn
skriv ut namn
skriv ut "Välkommen"

I ett flödesschema visas sekvens som flera steg efter varandra med pilar mellan.



Villkor

Villkor betyder att programmet måste göra ett val.

Programmet kontrollerar om något är sant eller falskt och väljer sedan väg.

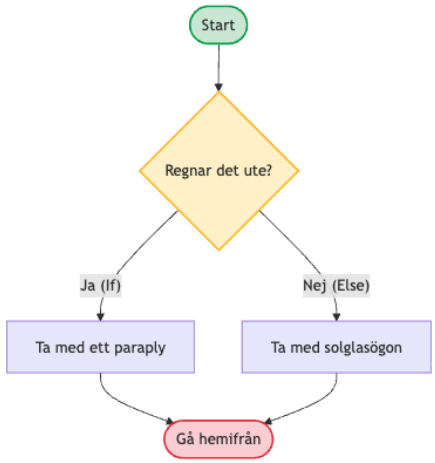
Exempel:

```
om tal > 10
skriv ut "Stort tal"
annars
skriv ut "Litet tal"
```

I ett flödesschema visas detta med en beslutssymbol.

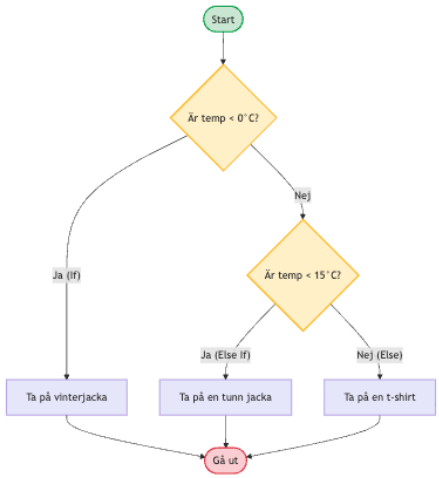
1. Enkel If / Else (Antingen eller)

Här ställer vi en ja/nej-fråga: **Regnar det?** Programmet delar upp sig i två möjliga vägar beroende på svaret.



2. Flera villkor (Else-If)

När ett enkelt ja eller nej inte räcker. Här kontrollerar vi temperaturen i flera steg.



Vad betyder formerna?

- **Gröna/Röda Runda rutor:** Visar var programmet startar och slutar.
- **Gula Romb-former (Diamanter):** Detta är själva `if`-satsen! Här ställs en fråga som programmet måste svara på.
- **Vita Fyrkanter:** Handlingen som utförs (själva koden som körs).

Iteration

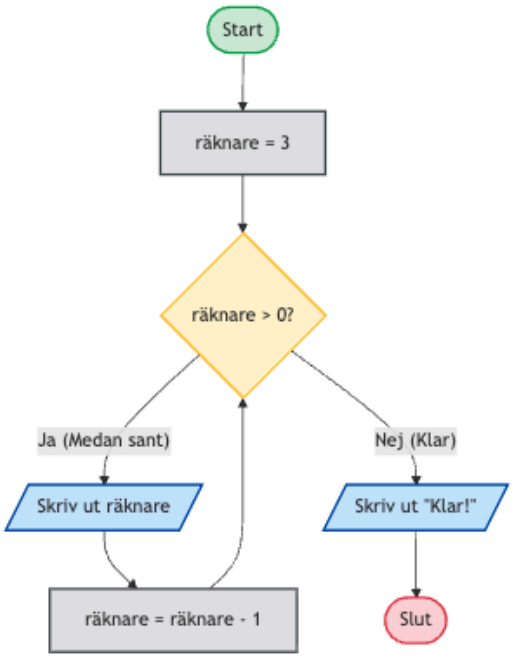
Iteration betyder att något upprepas flera gånger.

Det kan vara en loop som fortsätter tills ett villkor är uppfyllt.

Exempel:

```
så länge talet inte är 0
skriv ut talet
läs in nytt tal
```

I ett flödesschema visas iteration genom att en pil går tillbaka till ett tidigare ste



Ett flödesschema beskriver samma sak som pseudokod, men i **bildform istället för text**.

Vanliga symboler i flödesscheman

Start / Slut
visar var programmet börjar och slutar

Inmatning / Utskrift
visar när programmet läser in eller skriver ut information

Bearbetning
visar beräkningar eller andra steg i programmet

Beslut
visar ett val, till exempel ja eller nej

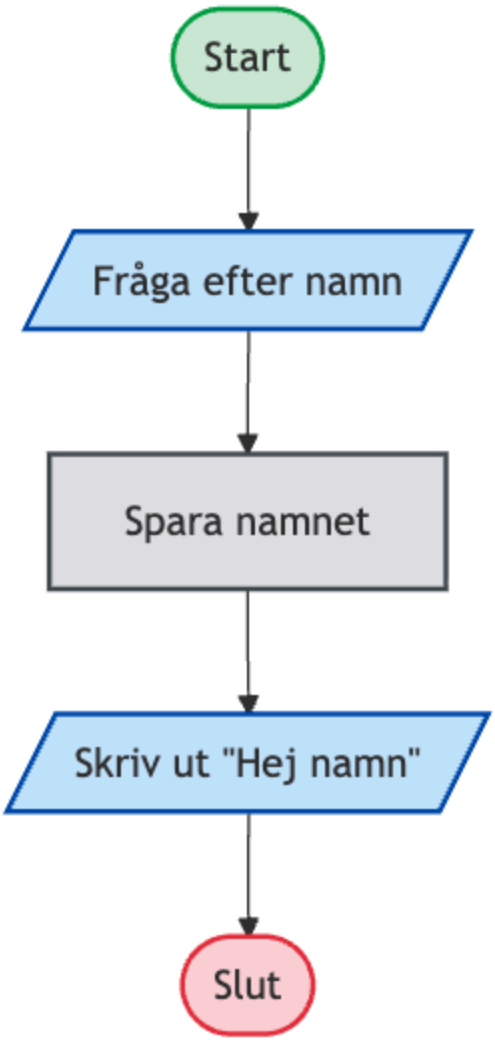
Pilar
visar i vilken ordning stegen utförs

Nedan finns pseudokod som kan omvandlas till flödesscheman.

Exempel 1 – Hälsningsprogram

Pseudokod

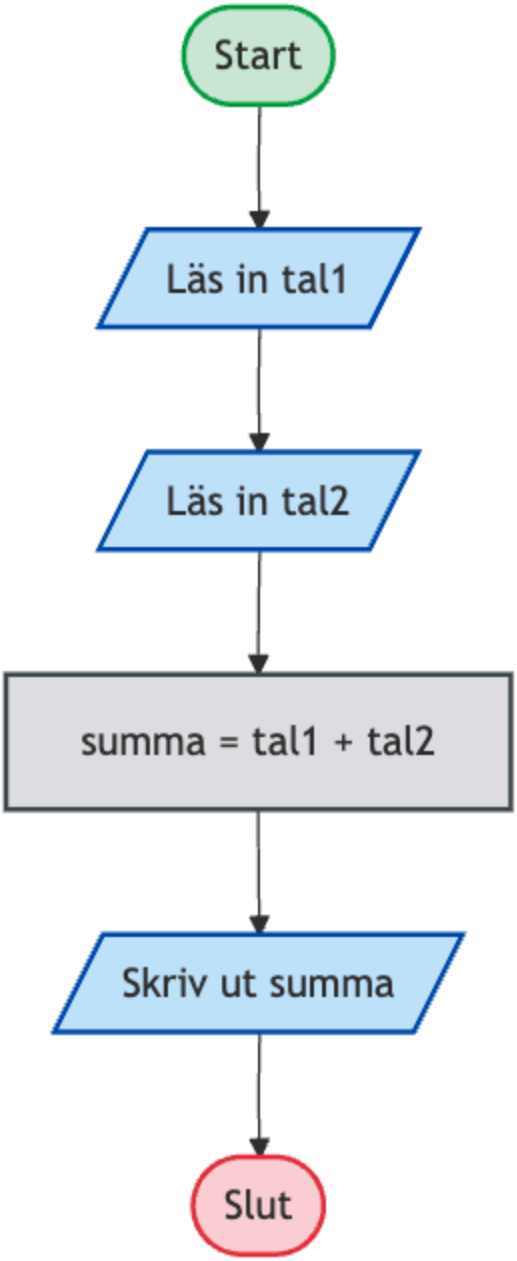
```
start
fråga efter namn
spara namnet
skriv ut "Hej namn"
slut
```



Exempel 2 – Räkna ut summan av två tal

Pseudokod

```
start
läs in tal1
läs in tal2
summa = tal1 + tal2
skriv ut summa
slut
```

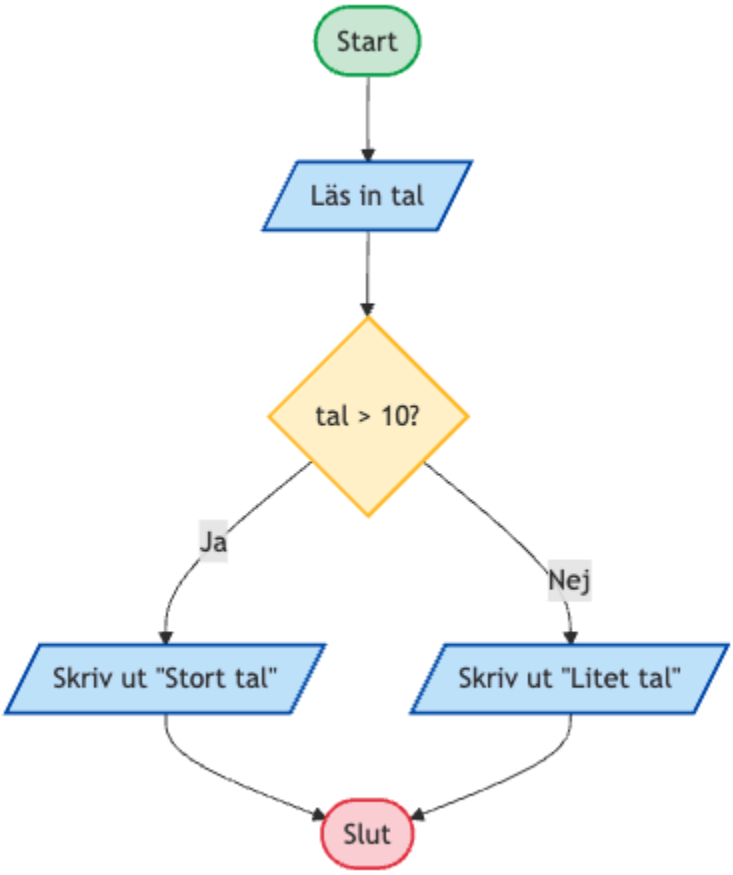


Exempel 3 – Stort eller litet tal

Pseudokod

```
start
läs in tal
```

```
om tal > 10
      skriv ut "Stort tal"
annars
      skriv ut "Litet tal"
slut
```



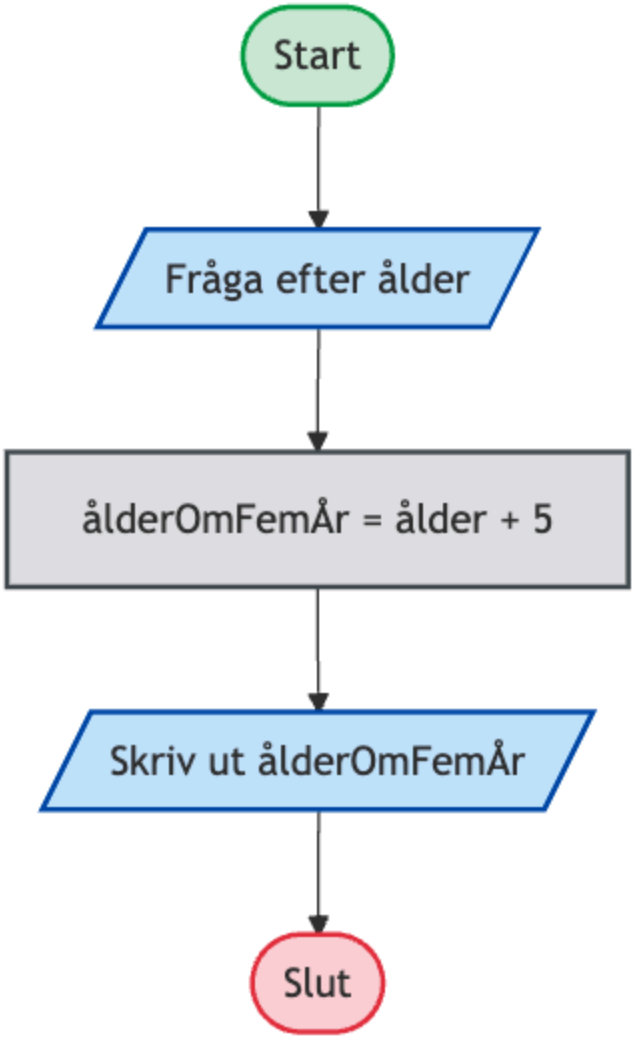
Här används en **beslutspunkt** i flödesschemat.

Exempel 4 – Ålder om fem år

Pseudokod

```
start
fråga efter ålder
ålderOmFemÅr = ålder + 5
```

```
skriv ut ålderOmFemÅr
slut
```

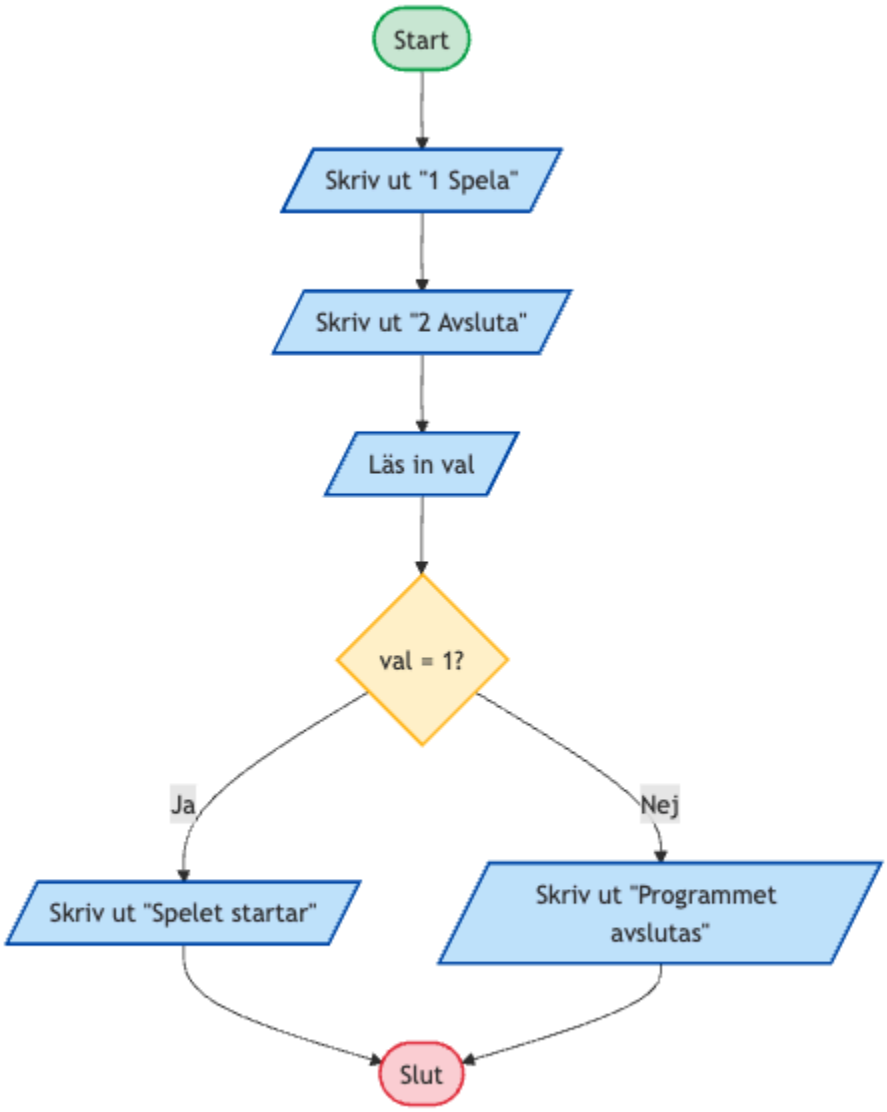


Exempel 5 – Enkel meny

Pseudokod

```
start
skriv ut "1 Spela"
skriv ut "2 Avsluta"
läs in val
om val = 1
      skriv ut "Spelet startar"
```

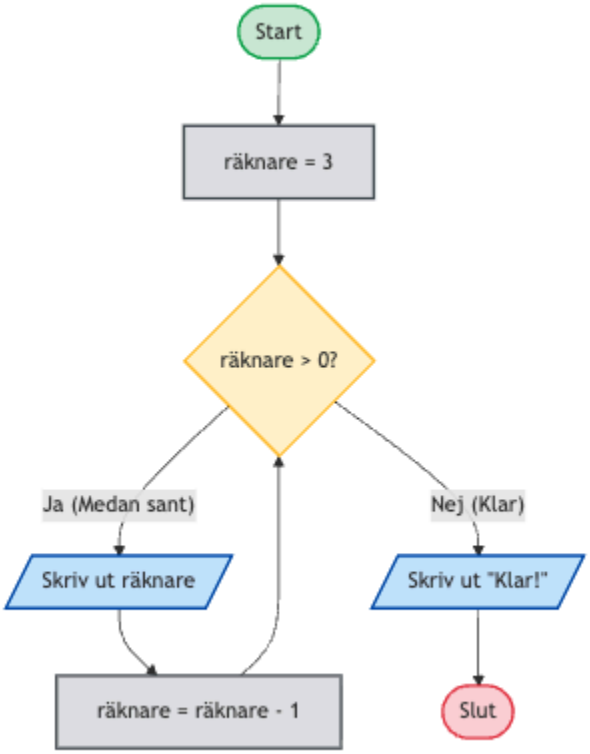
```
annars
      skriv ut "Programmet avslutas"
slut
```



Exempel 6 – Iteration (Loop / Nedräkning)

```
start
sätt räknare = 3
```

```
medan räknare > 0
      skriv ut räknare
räknare = räknare - 1
skriv ut "Klar!"
slut
```



Sammanfattning

Flödesscheman hjälper programmerare att

planera program
förstå programlogik
visualisera algoritmer

I praktiken gör man ofta först

1. Problem
2. Algoritm
3. Pseudokod
4. Flödesschema
5. Kod

Det gör det lättare att skapa program som fungerar korrekt.

3.6 Från idé till program

När programmerare skapar ett program börjar de sällan direkt med att skriva kod. Istället arbetar de steg för steg från en idé till ett färdigt program.

Detta arbetssätt gör det lättare att förstå problemet och skapa ett program som fungerar.

Ett vanligt arbetssätt kan beskrivas i följande steg.

Förstå problemet
Planera lösningen
Beskriva lösningen
Skriva programmet
Testa programmet

Steg 1 Förstå problemet

Först måste man förstå vad programmet ska göra.

Exempel

Ett program ska fråga användaren efter sin ålder och sedan skriva ut hur gammal personen är om tio år.

Frågor man kan ställa är

Vilken information behöver programmet
Vad ska programmet räkna ut
Vad ska programmet skriva ut

Steg 2 Planera lösningen

Nästa steg är att planera hur problemet ska lösas.

Man kan till exempel skriva ner de viktigaste stegen.

```
fråga efter ålder
spara åldern
lägg till 10
skriv ut resultatet
```

Steg 3 Beskriva lösningen

Nu kan man beskriva lösningen mer tydligt med pseudokod.

Pseudokod

```
start
läs in ålder
ålderOmTioÅr = ålder + 10
skriv ut ålderOmTioÅr
slut
```

Man skulle också kunna rita ett flödesschema som visar samma steg.

Steg 4 Skriva programmet

När lösningen är planerad kan man skriva programmet i Python.

```
age = int(input("Hur gammal är du? "))
future_age = age + 10
print(f"Om tio år är du {future_age} år gammal")
```

Steg 5 Testa programmet

När programmet är skrivet måste det testas.

Man kan till exempel testa med olika värden.

Om användaren skriver

15

ska programmet skriva ut

Om tio år är du 25 år gammal

Om något inte fungerar måste programmet ändras tills det fungerar korrekt.

Sammanfattning

Program utvecklas ofta i flera steg.

idé
problem
algoritm
pseudokod

program
testning

Genom att arbeta på detta sätt blir det lättare att skapa program som fungerar och är lätta att förstå.

3.7 Exempel: temperaturprogram

Nu ska vi se ett exempel på hur man kan gå från ett problem till ett färdigt program.

Problemet är att skapa ett program som frågar användaren efter en temperatur i grader Celsius och sedan skriver ut temperaturen i grader Fahrenheit.

Steg 1 Förstå problemet

Programmet ska

fråga efter en temperatur i Celsius
räkna om temperaturen till Fahrenheit
skriva ut resultatet

Formeln för att omvandla Celsius till Fahrenheit är

$F = C \times 9 / 5 + 32$

Steg 2 Algoritm

Algoritmen kan beskrivas så här

läs in temperaturen i Celsius
räkna ut temperaturen i Fahrenheit
skriv ut resultatet

Steg 3 Pseudokod

```
start
läs in temperatur i Celsius
fahrenheit = celsius × 9 / 5 + 32
skriv ut temperaturen i Fahrenheit
slut
```

Steg 4 Pythonprogram

```
celsius = float(input("Skriv en temperatur i Celsius: "))
fahrenheit = celsius * 9 / 5 + 32
print(f"Temperaturen i Fahrenheit är {fahrenheit}")
```

Hur programmet fungerar

Programmet gör följande

läser in temperaturen från användaren
omvandlar temperaturen till ett decimaltal
räknar ut temperaturen i Fahrenheit
skriver ut resultatet

Om användaren skriver

20

ska programmet skriva ut

Temperaturen i Fahrenheit är 68.0

Sammanfattning

I detta exempel har vi gått igenom hela processen

problem
algoritm
pseudokod
program

Detta är ett vanligt arbetssätt när programmerare utvecklar program.

3.8 Övningar

I dessa övningar ska du arbeta med **problemlösning i programmering**. Försök att först tänka igenom lösningen innan du börjar skriva kod.

Arbeta gärna i följande steg

förstå problemet
skriv en enkel algoritm

skriv pseudokod
skriv programmet i Python

Övning 1 Hälsningsprogram

Skriv ett program som frågar användaren efter sitt namn och skriver ut en hälsning.

Programmet ska skriva ut

```
Hej namn
```

Fundera först på

vilken information programmet behöver
vad programmet ska skriva ut

Övning 2 Ålder om tio år

Skriv ett program som

frågar användaren efter sin ålder
räknar ut hur gammal personen är om tio år
skriver ut resultatet

Övning 3 Summan av två tal

Skriv ett program som

läser in två tal från användaren
räknar ut summan
skriver ut resultatet

Exempel på resultat

Summan är 25

Övning 4 Temperaturprogram

Skriv ett program som

läser in en temperatur i Celsius
räknar om temperaturen till Fahrenheit
skriver ut resultatet

Tips
Formeln är

$F = C \times 9 / 5 + 32$

Övning 5 Medelvärde

Skriv ett program som

läser in tre tal
räknar ut medelvärdet
skriver ut resultatet

Tips
Medelvärde = summan av talen delat med antalet tal

Övning 6 Största talet

Skriv ett program som

läser in två tal
avgör vilket tal som är störst
skriver ut resultatet

Tips
Du kan använda `if` i programmet.

3.9 Reflektionsfrågor

1 - Vad är ett programmeringsproblem?

2 - Varför är det viktigt att förstå problemet innan man börjar skriva kod?

3 - Vad är en algoritm?

4 - Ge ett exempel på en algoritm i vardagen.

5 - Varför är det ofta bra att dela upp ett stort problem i mindre delar?

6 - Vad är pseudokod?

7 - Varför kan pseudokod göra det lättare att skriva ett program?

8 - Vad är ett flödesschema?

9 - Vilken skillnad finns mellan pseudokod och ett flödesschema?

10 - Vilka steg brukar programmerare följa när de går från en idé till ett färdigt program?

4 - Operatorer och beräkningar

Program behöver ofta **räkna ut saker**. I Python görs beräkningar med hjälp av **operatorer**.

Operatorer används för att

göra matematiska beräkningar
jämföra värden
arbeta med variabler

I detta kapitel börjar vi med **aritmetiska operatorer**, alltså operatorer som används för matematik.

4.1 Aritmetiska operatorer

Aritmetiska operatorer används för vanliga matematiska beräkningar.

De vanligaste operatorerna är

`+` addition
`-` subtraktion
`*` multiplikation
`/` division
`//` heltalsdivision

Addition

Addition används för att lägga ihop tal.

```
a = 5
b = 3
result = a + b
print(result)
```

Programmet skriver ut

Subtraktion

Subtraktion används för att dra bort ett tal från ett annat.

```
 a = 10
b = 4
result = a - b
print(result)
```

Programmet skriver ut

6

Multiplikation

Multiplikation används för att multiplicera tal.

```
a = 6
b = 7
result = a * b
print(result)
```

Programmet skriver ut

42

Division

Division används för att dela ett tal med ett annat.

```
a = 10
b = 2
result = a / b
print(result)
```

Programmet skriver ut

5.0

När man använder `/` i Python blir resultatet ofta ett **decimaltal**, även om divisionen går jämnt ut.

Heltalsdivision

Heltalsdivision används när man vill ha ett heltal som resultat.

```
a = 10
b = 3
result = a // b
print(result)
```

Programmet skriver ut

3

Här delas 10 med 3. Svaret blir egentligen 3.333..., men med `//` tar Python bara heltalsdelen.

Operatorer och variabler

Operatorer används ofta tillsammans med variabler.

```
price = 50
amount = 3
total = price * amount
print(f"Totalt pris är {total}")
```

Programmet räknar ut det totala priset genom att multiplicera priset med antalet.

Sammanfattning

Aritmetiska operatorer används för matematiska beräkningar i program.

De vanligaste är

`+` addition
`-` subtraktion
`*` multiplikation
`/` division
`//` heltalsdivision

Genom att kombinera operatorer och variabler kan program göra beräkningar och lösa olika problem.

Övningar 4.1 Aritmetiska operatorer

Skriv varje program i en egen `.py`-fil eller fortsätt i samma fil.

Övning 1 Addition

Skapa två variabler med talen 8 och 5.

Räkna ut summan och skriv ut resultatet.

Exempel på resultat

Summan är 13

Övning 2 Subtraktion

Skapa två variabler med talen 20 och 7.

Räkna ut skillnaden mellan talen och skriv ut resultatet.

Övning 3 Multiplikation

Skapa två variabler

```
price = 25
amount = 4
```

Räkna ut det totala priset och skriv ut resultatet.

Exempel på resultat

Totalt pris är 100

Övning 4 Division

Skapa två variabler

```
a = 20
b = 4
```

Dela talen med operatorn `/` och skriv ut resultatet.

Fundera på varför resultatet blir ett decimaltal.

Övning 5 Heltalsdivision

Skapa två variabler

```
a = 17
b = 5
```

Använd operatorn `//` och skriv ut resultatet.

Övning 6 Eget räkneprogram

Skriv ett program som frågar användaren efter två tal.

Programmet ska skriva ut

summan
skillnaden
produkten

Tips

Använd `int(input())` för att läsa in talen.

4.2 Jämförelseoperatorer

I programmering behöver man ofta **jämföra värden**. Python använder då något som kallas **jämförelseoperatorer**.

En jämförelseoperator jämför två värden och resultatet blir antingen

`True`
eller
`False`

Det betyder att uttrycket är **sant eller falskt**.

De vanligaste jämförelseoperatorerna är

`==` lika med
`!=` inte lika med
`<` mindre än
`>` större än

Lika med

Operatorn `==` används för att kontrollera om två värden är lika.

```
a = 5
b = 5
print(a == b)
```

Programmet skriver ut

True

Eftersom värdena är lika.

Inte lika med

Operatorn `!=` används för att kontrollera om två värden är olika.

```
a = 5
b = 3
print(a != b)
```

Programmet skriver ut

True

Eftersom värdena är olika.

Mindre än

Operatorn `<` används för att kontrollera om ett värde är mindre än ett annat.

```
a = 4
b = 10
print(a < b)
```

Programmet skriver ut

True

Eftersom 4 är mindre än 10.

Större än

Operatorn `>` används för att kontrollera om ett värde är större än ett annat.

```
a = 12
b = 7
print(a > b)
```

Programmet skriver ut

True

Eftersom 12 är större än 7.

Jämförelser i variabler

Man kan också spara resultatet av en jämförelse i en variabel.

```
age = 18
adult = age >= 18
print(adult)
```

Programmet skriver ut

True

Eftersom åldern är minst 18.

Sammanfattning

Jämförelseoperatorer används för att jämföra värden.

Resultatet blir alltid

`True`
eller
`False`

De operatorer du har lärt dig är

`==` lika med
`!=` inte lika med
`<` mindre än
`>` större än

Dessa operatorer används ofta tillsammans med **if-satser**, som vi kommer att arbeta mer med i nästa kapitel.

Skriv programmen i Python och kör dem för att se vad som händer.

Övning 1 Lika eller inte

Skapa två variabler

```
a = 10
b = 10
```

Skriv ut resultatet av jämförelsen

```
a == b
```

Fundera på varför resultatet blir det som det blir.

Övning 2 Olika tal

Skapa två variabler

```
a = 7
b = 3
```

Skriv ut resultatet av

```
a != b
```

Övning 3 Mindre än

Skapa två variabler

```
x = 4
y = 9
```

Skriv ut resultatet av

```
x < y
```

Övning 4 Större än

Skapa två variabler

```
x = 15
y = 8
```

Skriv ut resultatet av

```
x > y
```

Övning 5 Jämför två tal från användaren

Skriv ett program som

läser in två tal från användaren
jämför talen
skriver ut resultatet av jämförelsen

Programmet ska skriva ut

är talen lika
är talen olika
är det första talet större än det andra

Tips

Använd `int(input())` för att läsa in talen.

Övning 6 Testa olika värden

Skriv ett program där du jämför följande värden. Tänk först vad det blir innan du testar

```
5 == 5
5 != 5
3 < 7
10 > 20
```

Skriv ut resultatet av varje jämförelse.

4.3 Logiska operatorer

I programmering behöver man ibland **kombinera flera villkor**. För att göra detta används **logiska operatorer**.

Logiska operatorer arbetar med värden som är

`True`
eller
`False`

De tre vanligaste logiska operatorerna i Python är

```
and
or
not
```

Operatorn and

Operatorn `and` används när **båda villkoren måste vara sanna**.

```
age = 20
print(age > 18 and age < 30)
```

Programmet skriver ut

True

Det beror på att båda villkoren är sanna.

`age > 18` är sant
`age < 30` är också sant

Om ett av villkoren är falskt blir hela uttrycket falskt.

Operatorn or

Operatorn `or` används när **minst ett av villkoren måste vara sant**.

```
age = 15
print(age < 18 or age > 65)
```

Programmet skriver ut

True

Eftersom personen är yngre än 18.

Om minst ett villkor är sant blir resultatet sant.

Operatorn not

Operatorn `not` används för att **vända ett värde**.

Om något är sant blir det falskt, och om något är falskt blir det sant.

```
is_student = True print(not is_student)
```

Programmet skriver ut

False

Exempel med flera villkor

Logiska operatorer används ofta när program behöver kontrollera flera saker samtidigt.

```
age = 22
has_ticket = True
print(age >= 18 and has_ticket)
```

Programmet kontrollerar om personen

är minst 18 år
och har biljett

Om båda villkoren är sanna blir resultatet sant.

Sammanfattning

Logiska operatorer används för att kombinera villkor.

`and` båda villkoren måste vara sanna
`or` minst ett villkor måste vara sant
`not` vänder ett villkor

De används ofta tillsammans med **jämförelseoperatorer och if-satser**.

Övningar 4.3 Logiska operatorer

Skriv programmen i Python och testa vad som skrivs ut.

Övning 1 Operatorn and

Tänk på att tänka igenom vad resultatet blir innan du testar i dessa övningar

Skapa en variabel

```
age = 20
```

Skriv ut resultatet av följande jämförelse

```
age > 18 and age < 30
```

Fundera på varför resultatet blir sant eller falskt.

Övning 2 Operatorn or

Skapa en variabel

```
age = 15
```

Skriv ut resultatet av

```
age < 18 or age > 65
```

Fundera på varför resultatet blir sant eller falskt.

Övning 3 Operatorn not

Skapa en variabel

```
is_student = True
```

Skriv ut resultatet av

```
not is_student
```

Övning 4 Flera villkor

Skapa två variabler

```
age = 25
has_ticket = True
```

Skriv ut resultatet av

```
age >= 18 and has_ticket
```

Övning 5 Jämför två tal

Skriv ett program som frågar användaren efter två tal.

Programmet ska skriva ut

om båda talen är större än 10

Tips

Använd `and`.

Övning 6 Ett av två villkor

Skriv ett program som frågar användaren efter sin ålder.

Programmet ska kontrollera om personen är

`yngre` än 18
eller
äldre än 65

Skriv ut resultatet av jämförelsen.

Tips

Använd `or`.

Övning 7 Vänd ett värde

Skapa en variabel

```
logged_in = False
```

Skriv ut resultatet av

```
not logged_in
```

Fundera på varför resultatet blir det som det blir.

4.4 Matematiska uttryck

I Python kan man kombinera **tal, variabler och operatorer** för att skapa matematiska uttryck.

Ett matematiskt uttryck fungerar på samma sätt som i vanlig matematik. Python räknar då ut ett resultat.

Ett enkelt exempel är

```
result = 5 + 3
print(result)
```

Programmet skriver ut

8

Matematiska uttryck används ofta tillsammans med variabler.

```
a = 10
b = 4
```

```
result = a + b
print(result)
```

Flera operatorer i samma uttryck

Man kan använda flera operatorer i samma uttryck.

```
result = 5 + 3 * 2
print(result)
```

Python följer samma **prioriteringsregler** som i matematik.

Multiplikation och division räknas före addition och subtraktion.

I detta fall blir resultatet

11

Använda parenteser

Man kan använda parenteser för att bestämma vilken beräkning som ska göras först.

```
result = (5 + 3) * 2
print(result)
```

Nu räknas additionen först.

Resultatet blir

16

Matematiska uttryck med variabler

Matematiska uttryck används ofta tillsammans med variabler.

```
price = 50
amount = 3
total = price * amount
print(total)
```

Programmet räknar ut det totala priset.

Exempel med flera beräkningar

```
a = 10
b = 5
c = 2
result = a + b * c
print(result)
```

Python räknar först multiplikationen och sedan additionen.

Sammanfattning

Matematiska uttryck består av

tal
variabler
operatorer

Python följer samma prioriteringsregler som i matematik.

multiplikation och division räknas först
addition och subtraktion räknas efteråt

Med parenteser kan man ändra ordningen på beräkningarna.

Skriv programmen i Python och kör dem för att se resultatet.

Övning 1 Enkel beräkning

Skapa ett program som räknar ut

```
5 + 7
```

Skriv ut resultatet.

Övning 2 Flera operatorer

Skriv ett program som räknar ut

```
10 + 4 * 3
```

Skriv ut resultatet och fundera på varför resultatet blir som det blir.

Övning 3 Parenteser

Skriv ett program som räknar ut

```
(10 + 4) * 3
```

Jämför resultatet med övning 2.

Övning 4 Beräkning med variabler

Skapa tre variabler

```
a = 8
b = 6
c = 2
```

Beräkna

```
a + b * c
```

Skriv ut resultatet.

Övning 5 Medelvärde

Skapa tre variabler

```
x = 10
y = 20
z = 30
```

Räkna ut medelvärdet.

Tips
Medelvärde = summan av talen delat med antalet tal.

Övning 6 Räkna ut area

Skriv ett program som räknar ut arean av en rektangel.

Skapa två variabler

```
length = 8
width = 5
```

Räkna ut arean och skriv ut resultatet.

Formel

area = längd * bredd

4.5 Beräkningsprogram

Nu ska vi kombinera det vi har lärt oss i detta kapitel. Med variabler, operatorer och matematiska uttryck kan vi skapa **enkla beräkningsprogram**.

Ett beräkningsprogram är ett program som

läser in värden
gör beräkningar
skriver ut resultatet

Sådana program används överallt, till exempel i

kalkylatorer
spel
ekonomiprogram
vetenskapliga beräkningar

Exempel 1 Enkel miniräknare

Detta program läser in två tal och räknar ut summan.

```
number1 = float(input("Skriv första talet: "))
number2 = float(input("Skriv andra talet: "))
summa = number1 + number2
print(f"Summan är {summa}")
```

Programmet gör följande

läser in två tal
lägger ihop talen
skriver ut resultatet

Exempel 2 Beräkna pris

Ett program kan också räkna ut totala kostnader.

```
price = float(input("Pris per vara: "))
amount = int(input("Antal varor: "))
total = price * amount
print(f"Totalt pris är {total}")
```

Programmet multiplicerar priset med antalet varor.

Exempel 3 Temperaturberäkning

Program kan också användas för att omvandla värden.

```
celsius = float(input("Temperatur i Celsius: "))
fahrenheit = celsius * 9 / 5 + 32
print(f"Temperaturen i Fahrenheit är {fahrenheit}")
```

Hur man bygger ett beräkningsprogram

När man skriver ett beräkningsprogram är det vanligt att följa dessa steg

läs in värden från användaren
spara värdena i variabler
gör beräkningen
skriv ut resultatet

Detta arbetssätt gör programmen tydliga och lätta att förstå.

Sammanfattning

Beräkningsprogram använder

variabler
operatorer
matematiska uttryck

Genom att kombinera dessa kan man skapa program som löser många olika typer av problem.

4.6 Vanliga fel

När man arbetar med operatorer och beräkningar i Python är det vanligt att göra vissa misstag. Dessa fel är normala när man lär sig programmera.

Genom att känna till de vanligaste felen blir det lättare att hitta och rätta dem.

Glömda parenteser

En vanlig miss är att glömma parenteser i funktioner.

Fel kod

```
print 5 + 3
```

Rätt kod

```
print(5 + 3)
```

I Python måste `print()` alltid ha parenteser.

Fel operator

Ett annat vanligt misstag är att använda fel operator.

Fel kod

```
result = 5 x 3
```

Rätt kod

```
result = 5 * 3
```

I Python används `*` för multiplikation.

Förväxla = och ==

Många nybörjare blandar ihop

`=`
och
`==`

`=` används för att tilldela ett värde till en variabel.

`==` används för att jämföra två värden.

Exempel

```
a = 5
print(a == 5)
```

Fel datatyp

När man använder `input()` får man alltid text. Om man försöker göra en beräkning direkt kan det bli fel.

Fel kod

```
age = input("Hur gammal är du? ")
print(age + 5)
```

Här försöker programmet lägga ihop text och tal.

Rätt kod

```
age = int(input("Hur gammal är du? "))
print(age + 5)
```

Fel ordning i beräkningar

Om man inte använder parenteser kan resultatet bli annorlunda än man tänkt.

Exempel

```
result = 10 + 4 * 3
print(result)
```

Python räknar först multiplikationen.

Om man vill räkna additionen först måste man använda parenteser.

```
result = (10 + 4) * 3
print(result)
```

Sammanfattning

Vanliga fel i beräkningsprogram är

glömda parenteser
fel operator
förväxling av `=` och `==`
fel datatyp från `input()`
fel ordning i matematiska uttryck

Genom att läsa felmeddelanden och kontrollera koden noggrant kan man oftast snabbt hitta problemet.

4.7 Övningar

I dessa övningar ska du använda operatorer och matematiska uttryck för att skapa olika beräkningsprogram.

Övning 1 Enkel beräkning

Skriv ett program som läser in två tal från användaren.

Programmet ska skriva ut

summan
skillnaden
produkten

Tips

Använd `int(input())`.

Övning 2 Medelvärde

Skriv ett program som läser in tre tal från användaren.

Programmet ska räkna ut medelvärdet och skriva ut resultatet.

Tips

Medelvärde = summan av talen delat med antalet tal.

Övning 3 Räkna ut area

Skriv ett program som frågar efter

längd
bredd

Programmet ska räkna ut arean av en rektangel.

Formel

area = längd * bredd

Övning 4 Temperatur

Skriv ett program som läser in en temperatur i Celsius.

Programmet ska räkna om temperaturen till Fahrenheit.

Formel

F = C * 9 / 5 + 32

Övning 5 Dubbel kontroll

Skriv ett program som frågar efter ett tal.

Programmet ska skriva ut

om talet är större än 10
och
mindre än 100

Tips

Använd `and`.

Övning 6 Största talet

Skriv ett program som läser in två tal.

Programmet ska skriva ut vilket tal som är störst.

Tips

Använd `if`.

4.8 Reflektionsfrågor

1 - Vad är en operator i programmering?

2 - Vad används aritmetiska operatorer till?

3 - Ge exempel på tre aritmetiska operatorer i Python.

4 - Vad är skillnaden mellan `/` och `//` i Python?

5 - Vad gör en jämförelseoperator?

6 - Vad blir resultatet av en jämförelse i Python?

7 - När använder man logiska operatorer som `and`, `or` och `not`?

8 - Varför är parenteser ibland viktiga i matematiska uttryck?

9 - Vad gör funktionen `input()` i ett beräkningsprogram?

10 - Varför är det viktigt att kontrollera datatyper när man gör beräkningar i Python?

Gör nu inlämningsuppgifter till kapitel 1-4

5 - Villkor

5.1 Beslut i program

Hittills har våra program utfört instruktioner **rad för rad uppifrån och ner**.
Alla instruktioner har körts varje gång programmet startar.

I många program behöver datorn däremot **fatta beslut**. Programmet måste kunna göra olika saker beroende på vad användaren skriver in eller vilka värden som finns i programmet.

Detta kallas **villkor**.

Ett villkor gör att programmet kan kontrollera något och sedan välja vad som ska hända.

Exempel på beslut i program

Ett program kan till exempel behöva kontrollera

om en person är myndig
om ett tal är större än ett annat
om en användare har skrivit rätt lösenord

Programmet kan då fatta ett beslut och göra olika saker beroende på resultatet.

Exempel

Ett program kan kontrollera en persons ålder.

```
age = int(input("Hur gammal är du? "))

if age >= 18:
     print("Du är myndig")
```

Gör indraget med tab-tangenten. Det är mycker viktigt att du kommer i håg indraget med tab på nästa rad efter if.

TIPS: tryck Enter efter : så att VS Code gör indraget direkt.

I detta exempel händer följande

1 Programmet läser in åldern från användaren
2 Programmet kontrollerar om åldern är större än eller lika med 18
3 Om villkoret är sant skrivs ett meddelande ut

Vad betyder if

Ordet **if** betyder **om**.

Programmet tolkar koden ungefär så här

Om åldern är minst 18
 då skriv ut texten

```
Du är myndig
```

Villkor ger True eller False

I tidigare kapitel har vi arbetat med **jämförelser**. Dessa jämförelser ger alltid resultatet

True
 eller
False

Exempel

```
age = 20
print(age >= 18)
```

Resultat

True

När vi använder **if** kontrollerar Python om uttrycket är **True**.

Om det är True körs koden som hör till villkoret.

Indrag i Python

Python använder **indrag** för att visa vilken kod som hör till ett villkor.

```
age = int(input("Hur gammal är du? "))

if age >= 18:
      print("Du är myndig")
```

Raden med `print` är indragen.
Det betyder att den bara körs om villkoret är sant.

Om indraget saknas kommer Python att ge ett fel.

Sammanfattning

Villkor gör att program kan **fatta beslut**.

Med hjälp av `if` kan ett program

kontrollera ett villkor
avgöra om något är sant eller falskt
köra kod bara när villkoret är sant

Detta gör program mycket mer användbara eftersom de kan reagera på olika situationer.

Enkel övning

Skriv ett program som

frågar användaren efter ett tal
skriver ut ett meddelande om talet är större än 10

Exempel

```
number = int(input("Skriv ett tal: "))
if number > 10:
      print("Talet är större än 10")
```

5.2 if

I Python används **if** för att kontrollera ett villkor och fatta ett beslut i ett program.

Ordet **if** betyder **om**. Programmet kontrollerar om ett villkor är sant. Om villkoret är sant körs koden som hör till villkoret.

Grundstruktur

Ett if-villkor skrivs på detta sätt

```
if villkor:
     kod som ska köras
```

Efter villkoret finns ett kolon `:`.
 På nästa rad skrivs koden som ska köras. Den raden måste vara **indragen**.

Exempel

```
age = int(input("Hur gammal är du? "))
if age >= 18:
     print("Du är myndig")
```

Hur programmet fungerar

1 Programmet frågar användaren efter ålder.
2 Åldern sparas i variabeln `age`.
3 Programmet kontrollerar villkoret `age >= 18`.
4 Om villkoret är sant skrivs texten ut.

Exempel på körning

Hur gammal är du? 20
 Du är myndig

Om användaren istället skriver

15

kommer inget meddelande att skrivas ut eftersom villkoret inte är sant.

Villkor i if

Villkoret i ett if-uttryck är ofta en **jämförelse**.

Exempel

```
age >= 18
score > 50
temperature < 0
```

Dessa uttryck ger alltid resultatet

True
 eller
False

Python kontrollerar om resultatet är **True**.

Viktigt med kolon och indrag

Ett if-uttryck måste innehålla

ett kolon efter villkoret
indrag på raden under

Exempel

```
if age >= 18:
     print("Du är myndig")
```

Om kolon eller indrag saknas kommer Python att visa ett felmeddelande.

Flera rader kod

Ett if-villkor kan innehålla flera rader kod.

```
age = int(input("Hur gammal är du? "))
if age >= 18:
     print("Du är myndig")
     print("Du får rösta i Sverige")
```

Alla indragna rader körs om villkoret är sant.

Övning

Skriv ett program som

frågar användaren efter ett tal
skriver ut ett meddelande om talet är större än 100

Exempel på lösning

```
number = int(input("Skriv ett tal: "))
if number > 100:
     print("Talet är större än 100")
```

5.3 if else

Med **if** kan ett program kontrollera om ett villkor är sant.
Men ibland vill man också göra något **om villkoret inte är sant**.

Då använder man **else**.

Ordet **else** betyder **annars**.

Med **if else** kan programmet välja mellan två olika alternativ.

Grundstruktur

```
if villkor:
      kod som körs om villkoret är sant
else:
      kod som körs om villkoret är falskt
```

Programmet kontrollerar först villkoret.

Om villkoret är **True** körs koden under `if`.
Om villkoret är **False** körs koden under `else`.

Exempel

```
age = int(input("Hur gammal är du? "))

if age >= 18:
      print("Du är myndig")
else:
      print("Du är inte myndig")
```

Hur programmet fungerar

1 Programmet frågar efter ålder.
2 Åldern sparas i variabeln `age`.
3 Programmet kontrollerar villkoret `age >= 18`.
4 Om villkoret är sant skrivs ett meddelande ut.
5 Annars skrivs ett annat meddelande ut.

Exempel på körning

Hur gammal är du? 20
Du är myndig

Hur gammal är du? 15
Du är inte myndig

Indrag i if else

Python använder **indrag** för att visa vilken kod som hör till varje del.

```
number = int(input("Skriv ett tal: "))
if number > 0:
     print("Talet är positivt")
else:
     print("Talet är noll eller negativt")
```

Alla rader som är indragna efter `if` körs om villkoret är sant.
Alla rader som är indragna efter `else` körs annars.

Operatorn %

I Python finns operatorn `%`. Den kallas **modulooperatorn**.

Modulo används för att räkna ut **resten vid en division**.

Exempel

print(10 % 3)

Resultat

1

Förklaring

10 delat med 3 blir 3 med **resten 1**.
Modulooperatorn ger alltså **resten**.

Därför används % ofta för att kontrollera om ett tal är jämnt eller udda.

Exempel

```
print(8 % 2)
```

Resultat

0

Eftersom 8 är jämnt delbart med 2 blir resten **0**.

Övning

Skriv ett program som

frågar användaren efter ett tal
skriver ut om talet är **jämnt** eller **udda**

Tips

Operatorn `%` räknar ut resten i en division.

Exempel

```
number = int(input("Skriv ett tal: "))

if number % 2 == 0:
     print("Talet är jämnt")
else:
     print("Talet är udda")
```

5.4 elif

Ibland behöver ett program kontrollera **flera olika villkor**.
Det räcker då inte med bara `if` och `else`.

Då används **elif**.

Ordet **elif** betyder **annars om**.
Programmet kontrollerar villkoren i tur och ordning tills ett av dem är sant.

Grundstruktur

Ett program med flera villkor kan skrivas så här

```
if villkor:
     kod som ska köras
```

```
elif annat villkor:
      kod som ska köras
      else:
kod som körs om inget villkor är sant
```

Programmet börjar alltid med att kontrollera `if`.
Om villkoret är sant körs den koden och resten av villkoren kontrolleras inte.

Om villkoret inte är sant går programmet vidare till `elif`.

Exempel

Programmet frågar efter en temperatur och skriver ut hur vädret känns.

```
temperature = int(input("Ange temperatur: "))
if temperature < 10:
      print("Det är kallt")
elif temperature <= 20:
      print("Det är lagom")
else:
      print("Det är varmt")
```

Hur programmet fungerar

1 Programmet frågar användaren efter temperatur.
2 Temperaturen sparas i variabeln `temperature`.
3 Programmet kontrollerar först om temperaturen är under 10.
4 Om inte, kontrolleras om temperaturen är 20 eller lägre.
5 Om inget av villkoren är sant körs koden under `else`.

Exempel på körning

Ange temperatur: 5
Det är kallt

Ange temperatur: 15
Det är lagom

Ange temperatur: 27
Det är varmt

Viktigt att tänka på

Villkoren kontrolleras **uppifrån och ner**.
Därför är ordningen viktig.

```
score = int(input("Hur många poäng fick du? "))
if score >= 80:
     print("Du fick ett högt resultat")
elif score >= 50:
     print("Du klarade provet")
else:
     print("Du klarade inte provet")
```

Om poängen är **85** körs bara den första delen eftersom villkoret redan är sant.

Övning

Skriv ett program som frågar efter en persons ålder.

Programmet ska skriva ut

Barn om åldern är under 13
Tonåring om åldern är mellan 13 och 19
Vuxen om åldern är 20 eller mer

Tips: använd `if`, `elif` och `else`.

5.5 Nästlade villkor

Ibland behöver ett program kontrollera **ett villkor inuti ett annat villkor**.
Detta kallas **nästlade villkor**.

Ordet *nästlad* betyder att något ligger **inuti något annat**.

I programmering betyder det att en `if`-sats ligger **inuti en annan `if`-sats**.

Grundstruktur

Ett nästlat villkor kan skrivas så här

```
if villkor1:
     if villkor2:
          kod som ska köras
```

Programmet kontrollerar först det första villkoret.
Om det är sant kontrolleras det andra villkoret.

Exempel

Programmet kontrollerar först om en person är vuxen.
Om personen är vuxen kontrolleras också om personen får rösta.

```
age = int(input("Hur gammal är du? "))
citizen = input("Är du svensk medborgare (ja/nej)? ")

if age >= 18:
      if citizen == "ja":
            print("Du får rösta i svenska val")
      else:
            print("Du är myndig men får inte rösta")
      else:
            print("Du är inte myndig ännu")
```

Hur programmet fungerar

1 Programmet frågar efter ålder.
2 Programmet frågar om personen är svensk medborgare.
3 Programmet kontrollerar först om åldern är minst 18.
4 Om personen är myndig kontrolleras också medborgarskapet.
5 Programmet skriver ut ett meddelande beroende på resultatet.

Exempel på körning

Hur gammal är du? 20
Är du svensk medborgare (ja/nej)? ja
Du får rösta i svenska val

Hur gammal är du? 20
Är du svensk medborgare (ja/nej)? nej
Du är myndig men får inte rösta

Hur gammal är du? 15
Du är inte myndig ännu

Viktigt att tänka på

När villkor ligger inuti andra villkor blir **indragen extra viktiga**.
Indragen visar vilken kod som hör till vilket villkor.

Varje nytt villkor inuti ett annat villkor behöver **ytterligare ett indrag**.

Övning

Skriv ett program som frågar efter

ålder
om personen har körkort (ja eller nej)

Programmet ska skriva ut

Du får köra bil om personen är minst 18 **och** har körkort
Du har inte körkort om personen är minst 18 men saknar körkort
Du är för ung för att köra bil om personen är under 18

5.6 Logiska uttryck

I många program behöver man kontrollera **flera villkor samtidigt**.
Då används **logiska operatorer**.

De tre vanligaste logiska operatorerna i Python är

```
and
or
not
```

Med hjälp av dessa kan man skapa **logiska uttryck**.

Operatorn and

Operatorn `and` betyder att **båda villkoren måste vara sanna**.

Exempel

```
age = int(input("Hur gammal är du? "))
if age >= 18 and age <= 65:
      print("Du är i arbetsför ålder")
```

Programmet kontrollerar två saker

åldern är minst 18
åldern är högst 65

Båda måste vara sanna för att koden ska köras.

Operatorn or

Operatorn `or` betyder att **minst ett av villkoren måste vara sant**.

Exempel

```
day = input("Vilken dag är det? ")
if day == "lördag" or day == "söndag":
     print("Det är helg")
```

Programmet skriver ut meddelandet om dagen är **lördag eller söndag**.

Operatorn not

Operatorn `not` betyder **inte**.
Den används för att vända ett villkor.

Exempel

```
logged_in = False
if not logged_in:
     print("Du måste logga in först")
```

Om variabeln `logged_in` är falsk skriver programmet ut meddelandet.

Kombinera logiska operatorer

Man kan också kombinera flera villkor i samma uttryck.

```
age = int(input("Hur gammal är du? "))
has_ticket = input("Har du biljett (ja/nej)? ")

if age >= 18 and has_ticket == "ja":
     print("Du får gå in")
else:
     print("Du får inte gå in")
```

Programmet kräver att

personen är minst 18 år
personen har biljett

Viktigt att tänka på

Logiska operatorer gör det möjligt att skriva **mer avancerade villkor**.

De används ofta när man behöver kontrollera

flera värden samtidigt
flera alternativ
om något **inte** är sant

Övning

Skriv ett program som frågar efter

ålder
om personen har studentkort (ja eller nej)

Programmet ska skriva ut

Rabatt beviljad om personen är under 20 **eller** har studentkort
Ingen rabatt annars

5.7 Program med beslut

Nu har du lärt dig flera sätt att låta ett program fatta beslut.
Med hjälp av `if`, `elif`, `else` och logiska operatorer kan programmet välja **olika vägar beroende på användarens inmatning**.

Program som använder villkor kallas ofta **program med beslut**.

Exempel: biljettpris

I detta program beräknas priset på en biljett beroende på personens ålder.

```
age = int(input("Hur gammal är du? "))
if age < 12:
      print("Biljettpris: 50 kr")
elif age < 18:
      print("Biljettpris: 80 kr")
else:
      print("Biljettpris: 120 kr")
```

Hur programmet fungerar

1 Programmet frågar efter användarens ålder.
2 Åldern sparas i variabeln `age`.
3 Programmet kontrollerar först om åldern är under 12.
4 Om inte, kontrolleras om åldern är under 18.
5 Om inget av dessa villkor är sant används `else`.

Exempel: jämnt eller udda tal

Programmet kan också fatta beslut baserat på matematiska beräkningar.

```
number = int(input("Skriv ett tal: "))
if number % 2 == 0:
     print("Talet är jämnt")
else:
     print("Talet är udda")
```

Operatorn `%` kallas **modulo** och används för att ta fram **resten i en division**.

Om resten blir **0** är talet jämnt.

Exempel: tillgång till ett konto

```
password = input("Skriv lösenord: ")
if password == "python123":
     print("Inloggning lyckades")
else:
     print("Fel lösenord")
```

Programmet kontrollerar om användarens inmatning är samma som det sparade lösenordet.

Varför beslut är viktiga

Utan villkor skulle alla program göra **samma sak varje gång**.
Med beslut kan programmet

anpassa sig efter användaren
hantera olika situationer
lösa mer avancerade problem

Beslut är därför en av de **viktigaste delarna i programmering**.

Övning

Skriv ett program som frågar efter ett tal.

Programmet ska skriva ut

Positivt tal om talet är större än 0
Noll om talet är 0
Negativt tal om talet är mindre än 0

5.8 Övningar

I dessa övningar ska du använda det du har lärt dig i kapitlet om villkor.

Du ska använda

`if`
`elif`
`else`
logiska operatorer
modulo `%`
f-strängar för utskrift

Övning 1

Skriv ett program som frågar efter en persons ålder.

Programmet ska skriva ut

Barn om åldern är under 13
Tonåring om åldern är mellan 13 och 19
Vuxen om åldern är 20 eller mer

Övning 2

Skriv ett program som frågar efter ett tal.

Programmet ska avgöra om talet är **positivt**, **negativt** eller **noll**.

Programmet ska skriva ut

Talet är positivt
Talet är negativt
Talet är noll

Övning 3

Skriv ett program som frågar efter ett tal.

Programmet ska avgöra om talet är **jämnt eller udda**.

Tips: använd operatorn `%`.

Övning 4

Skriv ett program som frågar efter

namn
ålder

Programmet ska skriva ut en hälsning med en **f-sträng**.

Om personen är minst 18 år ska programmet skriva
Du är myndig

Annars ska programmet skriva
Du är inte myndig

Övning 5

Skriv ett program som frågar efter

poäng på ett prov (0–100)

Programmet ska skriva ut

Underkänt om poängen är under 50
Godkänt om poängen är mellan 50 och 79
Mycket bra om poängen är 80 eller mer

Övning 6

Skriv ett program som frågar efter

ålder
om personen har studentkort (ja eller nej)

Programmet ska skriva ut

Rabatt beviljad om personen är under 20 **eller** har studentkort
 Ingen rabatt annars

Övning 7

Skriv ett program som frågar efter ett tal.

Programmet ska skriva ut

Delbart med 5 om talet kan delas med 5 utan rest
 Inte delbart med 5 annars

Tips: använd operatorn `%`.

Övning 8

Skriv ett program som frågar efter en temperatur.

Programmet ska skriva ut

Kallt om temperaturen är under 10
 Lagom om temperaturen är mellan 10 och 20
 Varmt om temperaturen är över 20

Använd `if`, `elif` och `else`.

5.9 Reflektionsfrågor

1 - Vad är syftet med en `if`-sats i ett program?

2 - Vad är skillnaden mellan `if` och `if else`?

3 - När använder man `elif` i ett program?

4 - Varför är ordningen på villkor viktig när man använder `if`, `elif` och `else`?

5 - Vad betyder operatorn `%` i Python och när används den?

6 - Vad gör logiska operatorer som `and`, `or` och `not`?

7 - Vad betyder det att villkor kan vara **nästlade**?

8 - Varför är indrag viktiga i Python när man arbetar med villkor?

9 - Ge ett exempel på ett program i vardagen där datorer behöver fatta beslut.

10 - Varför är beslut (villkor) en viktig del av programmering?

6 - Loopar

6.1 Varför loopar behövs

6.2 while-loop

I många program behöver samma kod köras **flera gånger**.
Om vi skulle skriva samma kod om och om igen skulle programmet bli långt och svårt att läsa.

Därför använder programmerare **loopar**.

En **loop** gör att ett program kan upprepa en kodsektion automatiskt.

Exempel utan loop

Om vi vill skriva ut talen 1 till 5 utan loop måste vi skriva

```
print(1)
print(2)
print(3)
print(4)
print(5)
```

Det fungerar, men koden blir snabbt lång om vi behöver skriva ut många tal.

Exempel med loop

Med en loop kan samma uppgift lösas mycket enklare.

```
for i in range(5):
     print(i + 1)
```

Glöm inte att göra indraget med tab i VS Code

Programmet upprepar utskriften flera gånger automatiskt.

Varför loopar är viktiga

Loopar används när program behöver

upprepa en uppgift många gånger
arbeta med listor av data
räkna upp eller ner
bearbeta många värden

Många program i verkligheten använder loopar, till exempel när

spel uppdaterar skärmen många gånger per sekund
program går igenom en lista med användare
en app bearbetar många dataposter

Två vanliga loopar i Python

I Python finns två vanliga typer av loopar

`for` används när man vet **hur många gånger** något ska upprepas

`while` används när något ska upprepas **så länge ett villkor är sant**

I detta kapitel kommer du att lära dig hur båda dessa fungerar.

6.3 for-loop

n **for-loop** används när man vill upprepa kod ett visst antal gånger.

I Python används ofta funktionen `range()` tillsammans med en for-loop.

Exempel

```
for i in range(5):
     print(i)
```

Hur programmet fungerar

`range(5)` skapar talen **0, 1, 2, 3, 4**.

Variabeln `i` får varje tal i tur och ordning.

Programmet skriver därför ut

```
0
1
2
3
4
```

Viktigt att tänka på

Koden som ska upprepas måste vara **indragen**.

```
for i in range(5):
    print(i)
```

Varje gång loopen körs får variabeln `i` ett nytt värde.

6.4 range()

Funktionen `range()` används ofta tillsammans med **for-loopar**.
Den skapar en serie tal som loopen kan gå igenom.

Enkelt exempel

```
for i in range(5):
    print(i)
```

`range(5)` skapar talen

```
0
1
2
3
4
```

Observera att talet **5 inte ingår**.

Start och stopp

Man kan också ange **startvärde och stoppvärde**.

```
for i in range(1, 6):
     print(i)
```

Programmet skriver då ut

```
1
2
3
4
5
```

Steg i range()

Man kan också ange **hur mycket talet ska öka varje gång**.

```
for i in range(0, 10, 2):
     print(i)
```

Programmet skriver ut

```
0
2
4
6
8
```

Detta gör att loopen hoppar fram **2 steg varje gång**.

6.5 Oändliga loopar

En **oändlig loop** är en loop som aldrig tar slut.
Det händer när villkoret i loopen alltid är sant.

Exempel

```
while True:
     print("Detta program fortsätter för alltid")
```

Eftersom villkoret `True` alltid är sant kommer loopen att fortsätta.

Oändlig loop av misstag

Man kan också råka skapa en oändlig loop.

```
count = 0
      while count < 5:
      print(count)
```

Här ändras aldrig värdet på `count`.
Villkoret är därför alltid sant och loopen slutar aldrig.

Rätt sätt

För att undvika en oändlig loop måste värdet ändras.

```
count = 0
      while count < 5:
      print(count)
      count = count + 1
```

Nu ökar `count` varje gång loopen körs och programmet avslutas när `count` blir 5.

6.6 Loopar och villkor

Loopar och villkor används ofta tillsammans i program.
En loop kan upprepa en uppgift, och ett villkor kan bestämma **vad som ska hända i varje varv av loopen**.

Exempel

Programmet skriver ut talen från 1 till 10 och kontrollerar om talet är jämnt eller udda.

```
for i in range(1, 11):
      if i % 2 == 0:
            print(f"{i} är jämnt")
      else:
            print(f"{i} är udda")
```

Hur programmet fungerar

Loopen går igenom talen **1 till 10**.

För varje tal kontrollerar programmet

om talet är jämnt
eller om talet är udda

Operatorn `%` används för att ta fram **resten vid division**.

Om resten blir **0** är talet jämnt.

Exempel med användarens inmatning

```
number = int(input("Hur många tal vill du skriva ut? "))
for i in range(1, number + 1):
      if i % 2 == 0:
            print(f"{i} är jämnt")
      else:
            print(f"{i} är udda")
```

Programmet skriver ut alla tal upp till det tal användaren anger.

Varför detta är viktigt

Genom att kombinera **loopar och villkor** kan program

bearbeta många värden
analysera data
göra olika saker i varje varv av loopen

Detta används ofta i verkliga program.

6.7 Program med upprepningar

Loopar används ofta när ett program behöver göra samma sak **flera gånger**.
Det kan till exempel handla om att läsa in flera värden, räkna något eller skriva ut många resultat.

Exempel: skriva ut flera tal

```
for i in range(1, 11):
      print(i)
```

Programmet skriver ut talen från **1 till 10**.

Exempel: summera tal

Programmet frågar efter fem tal och räknar ut summan.

```
total = 0
for i in range(5):
      number = int(input("Skriv ett tal: "))
      total = total + number
print(f"Summan är {total}")
```

Hur programmet fungerar

Variabeln `total` börjar med värdet **0**.

Varje gång loopen körs

ett tal läses in
talet läggs till i `total`

När loopen är klar skrivs summan ut.

Exempel: räkna jämna tal

```
count = 0
for i in range(1, 11):
      if i % 2 == 0:
      count = count + 1
print(f"Antal jämna tal är {count}")
```

Programmet går igenom talen **1 till 10** och räknar hur många som är jämna.

Varför upprepningar är viktiga

Loopar gör det möjligt att

bearbeta många värden
upprepa beräkningar
skapa program som arbetar med större datamängder

Många program i verkligheten bygger på **upprepningar**.

6.8 Break

Ibland behöver ett program **avsluta en loop tidigare** än planerat.
Då kan man använda kommandot `break`.

När `break` körs **avslutas loopen direkt**, även om villkoret fortfarande är sant.

Exempel

Programmet frågar efter tal tills användaren skriver **0**.

```
while True:
      number = int(input("Skriv ett tal (0 för att avsluta): "))
      if number == 0:
            break
      print(f"Du skrev talet {number}")
```

Hur programmet fungerar

Programmet använder en **oändlig loop**.

Varje gång loopen körs frågar programmet efter ett tal.

Om användaren skriver **0** körs `break` och loopen avslutas.

Annars fortsätter programmet.

Exempel på körning

Skriv ett tal (0 för att avsluta): 5
Du skrev talet 5

Skriv ett tal (0 för att avsluta): 8
Du skrev talet 8

Skriv ett tal (0 för att avsluta): 0

Programmet avslutas.

Varför break används

`break` används när man vill

avsluta en loop i förtid
stoppa ett program när ett visst villkor uppfylls
avbryta en oändlig loop

Det är ett vanligt sätt att kontrollera när ett program ska sluta köra.

6.9 Övningar

I dessa övningar ska du använda

`for`
`while`
`range()`
villkor (`if`)
`break`

Övning 1

Skriv ett program som skriver ut talen från **1 till 10** med en `for`-loop.

Övning 2

Skriv ett program som skriver ut talen från **1 till 20**.

Programmet ska skriva ut om varje tal är **jämnt eller udda**.

Tips: använd `%`.

Övning 3

Skriv ett program som frågar användaren efter **5 tal**.

Programmet ska räkna ut **summan av talen** och skriva ut resultatet.

Övning 4

Skriv ett program som frågar användaren efter ett tal.

Programmet ska skriva ut **multiplikationstabellen för talet** från 1 till 10.

Exempel om användaren skriver 3

```
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
```

Övning 5

Skriv ett program som frågar användaren efter tal tills användaren skriver **0**.

Programmet ska skriva ut varje tal som användaren skriver.

Använd en `while`-loop.

Övning 6

Skriv ett program som frågar användaren efter tal tills användaren skriver **0**.

Programmet ska räkna ut **summan av alla tal**.

Tips: använd `break`.

Övning 7

Skriv ett program som frågar användaren efter tal tills användaren skriver **0**.

Programmet ska räkna **hur många tal** användaren har skrivit.

Övning 8

Skriv ett program som skriver ut talen från **1 till 50**.

Programmet ska skriva ut

Fizz om talet är delbart med 3
Buzz om talet är delbart med 5
FizzBuzz om talet är delbart med både 3 och 5
annars ska talet skrivas ut.

Vad är FizzBuzz?

FizzBuzz är en klassisk programmeringsövning som ofta används när man lär sig programmering.

Uppgiften används också ibland i **jobbintervjuer för programmerare**. Den testar om en person förstår grundläggande saker i programmering, till exempel

loopar
villkor
operatorer
logiskt tänkande

Bakgrund

Spelet FizzBuzz kommer ursprungligen från ett **barnspel** där deltagare räknar upp tal i tur och ordning.

Reglerna är

säg **Fizz** istället för tal som är delbara med 3
säg **Buzz** istället för tal som är delbara med 5
säg **FizzBuzz** istället för tal som är delbara med både 3 och 5

När uppgiften används i programmering ska datorn göra samma sak.

6.10 Reflektionsfrågor

1 - Vad är en loop i programmering?

2 - Varför används loopar i program?

3 - Vad är skillnaden mellan en `for`-loop och en `while`-loop?

4 - Vad gör funktionen `range()` i en `for`-loop?

5 - Varför är indrag viktiga när man skriver loopar i Python?

6 - Vad är en oändlig loop och hur kan den uppstå?

7 - Hur kan kommandot `break` användas i en loop?

8 - Ge ett exempel på en situation där ett program behöver upprepa något många gånger.

9 - Varför är loopar viktiga när man arbetar med större datamängder?

10 - Hur kan loopar och villkor användas tillsammans i ett program?

7 - Listor

7.1 Vad är en lista

I många program behöver man lagra **flera värden samtidigt**.
Istället för att skapa många variabler kan man använda en **lista**.

En lista är en datastruktur som kan innehålla flera värden.

Exempel

```
numbers = [3,7,2,9]
```

Här skapas en lista som innehåller fyra tal.

Listan sparas i variabeln `numbers`.

Man kan också ha listor med text.

```
names = ["Anna","Erik","Sara"]
```

Listor används ofta när program behöver

lagra många värden
bearbeta data
gå igenom information med loopar

Skriva ut en lista

Man kan skriva ut hela listan.

```
numbers = [3,7,2,9]
print(numbers)
```

Programmet skriver då ut

[3, 7, 2, 9]

Loop genom en lista

Listor används ofta tillsammans med loopar.

```
numbers = [3,7,2,9]
for number in numbers:
     print(number)
```

Programmet skriver ut varje värde i listan.

7.2 Index

Varje värde i en lista har en **position**.
Denna position kallas **index**.

I Python börjar index alltid på **0**.

Exempel

```
numbers = [3,7,2,9]
```

Positionerna i listan är

0 → 3
1 → 7
2 → 2
3 → 9

Hämta ett värde med index

Man kan hämta ett värde från listan genom att använda index.

```
numbers = [3,7,2,9]
print(numbers[0])
```

Programmet skriver ut

3

Ett annat exempel

```
numbers = [3,7,2,9]
print(numbers[2])
```

Programmet skriver ut

2

Viktigt att tänka på

Eftersom index börjar på **0** är det lätt att göra fel.

Om en lista innehåller fyra värden är det sista indexet **3**, inte 4.

Övning

Skapa en lista med fem tal.

```
numbers = [4, 7, 2, 9, 5]
```

Gör sedan följande i ditt program

1 Skriv ut **hela listan**.

2 Skriv ut **det första talet i listan**.

3 Skriv ut **det tredje talet i listan**.

4 Använd en **for-loop** för att skriva ut alla tal i listan.

Tips: använd **index** när du hämtar ett specifikt värde ur listan.

7.2 Index

Varje värde i en lista har en **position**.
Denna position kallas **index**.

I Python börjar index alltid på **0**.

Exempel

```
numbers = [3,7,2,9]
```

Positionerna i listan är

0 → 3
1 → 7
2 → 2
3 → 9

Hämta ett värde med index

Man kan hämta ett värde ur listan genom att använda index.

```
numbers = [3,7,2,9]
print(numbers[0])
```

Programmet skriver ut

3

Ett annat exempel

```
numbers = [3,7,2,9]
print(numbers[2])
```

Programmet skriver ut

2

Ändra ett värde i en lista

Man kan också ändra ett värde i listan genom att använda index.

```
numbers = [3,7,2,9]
numbers[1] = 10
print(numbers)
```

Resultat

[3, 10, 2, 9]

Viktigt att tänka på

Eftersom index börjar på **0** är det lätt att göra fel.

Om en lista innehåller fyra värden är det sista indexet **3**, inte 4.

Övning

Skapa en lista med fyra färger.

```
colors = ["röd","blå","grön","gul"]
```

Gör sedan följande

1 Skriv ut den första färgen.
2 Skriv ut den sista färgen.
3 Ändra den andra färgen till **svart**.
4 Skriv ut hela listan.

7.3 Loopar över listor

Listor används ofta tillsammans med **loopar**.
En loop gör det möjligt att gå igenom alla värden i listan.

Exempel

```
numbers = [3,7,2,9]
for number in numbers:
     print(number)
```

Programmet skriver ut varje värde i listan.

Resultat

```
3
7
2
9
```

Hur det fungerar

Loopen går igenom listan **ett värde i taget**.

För varje varv i loopen sparas värdet i variabeln `number`.

Sedan körs koden inuti loopen.

Exempel med beräkning

```
numbers = [3,7,2,9]
for number in numbers:
     print(number * 2)
```

Programmet skriver ut varje tal **multiplicerat med 2**.

Resultat

```
6
14
4
18
```

Varför detta är användbart

Loopar över listor används när program behöver

bearbeta många värden
göra beräkningar

skriva ut information
analysera data

Övning

Skapa en lista med fem tal.

```
numbers = [2,4,6,8,10]
```

Gör sedan följande

1 Skriv ut alla tal i listan.
2 Skriv ut varje tal multiplicerat med 3.
3 Skriv ut summan av alla tal i listan.

7.4 Lägga till värden

Ibland behöver ett program **lägga till nya värden i en lista**.
Det kan göras med funktionen `append()`.

`append()` lägger till ett nytt värde **i slutet av listan**.

Exempel

```
numbers = [3,7,2,9]
numbers.append(5)
print(numbers)
```

Resultat

[3, 7, 2, 9, 5]

Talet **5** har lagts till sist i listan.

Exempel med flera värden

Man kan lägga till flera värden genom att använda `append()` flera gånger.

```
numbers = [1,2,3]
numbers.append(4)
numbers.append(5)
print(numbers)
```

Resultat

[1, 2, 3, 4, 5]

Exempel med användarens inmatning

Programmet kan också lägga till värden som användaren skriver.

```
numbers = []

number = int(input("Skriv ett tal: "))
numbers.append(number)
print(numbers)
```

Listan var först tom men innehåller nu talet som användaren skrev.

Övning

Skapa en **tom lista**.

```
numbers = []
```

Gör sedan följande

1 Be användaren skriva **tre tal**.
2 Lägg till varje tal i listan med `append()`.
3 Skriv ut hela listan.

7.5 Bearbeta data

När data finns i en lista kan programmet **bearbeta informationen**.
Det kan till exempel handla om att räkna ut

summan av alla tal
medelvärdet
det största eller minsta värdet

Exempel: summa

```
numbers = [3,7,2,9]
total = 0
for number in numbers:
```

```
    total = total + number
print(f"Summan är {total}")
```

Programmet går igenom alla tal i listan och lägger ihop dem.

Exempel: medelvärde

```
numbers = [3,7,2,9]
total = 0
for number in numbers:
    total = total + number
average = total / len(numbers)
print(f"Medelvärdet är {average}")
```

Funktionen `len()` används för att ta reda på **hur många värden som finns i listan**.

Exempel: största värdet

```
numbers = [3,7,2,9]
largest = numbers[0]
for number in numbers:
    if number > largest:
    largest = number
print(f"Största talet är {largest}")
```

Programmet går igenom alla tal i listan och sparar det största.

Varför detta är viktigt

Att bearbeta data i listor används ofta när program ska

analysera information
räkna ut statistik
hantera stora datamängder

Övning

Skapa en lista med fem tal.

```
numbers = [4,6,2,8,10]
```

Gör sedan följande

1 Räkna ut **summan av alla tal i listan**.
2 Räkna ut **medelvärdet**.
3 Skriv ut resultaten.

7.6 Program med listor

istor används ofta i program som behöver **samla in och bearbeta flera värden**.

Genom att kombinera **listor, loopar och villkor** kan program hantera större mängder data.

Exempel: samla in tal

Programmet ber användaren skriva fem tal och sparar dem i en lista.

```
numbers = []
for i in range(5):
      number = int(input("Skriv ett tal: "))
      numbers.append(number)
print(numbers)
```

Programmet skapar först en **tom lista** och lägger sedan till talen som användaren skriver.

Exempel: hitta största talet

```
numbers = [4,7,2,9,5]
largest = numbers[0]
for number in numbers:
      if number > largest:
      largest = number
print(f"Största talet är {largest}")
```

Programmet går igenom alla tal i listan och sparar det största värdet.

Exempel: räkna jämna tal

```
numbers = [4,7,2,9,5]
count = 0
for number in numbers:
      if number % 2 == 0:
```

```
    count = count + 1
print(f"Antal jämna tal är {count}")
```

Programmet kontrollerar varje tal i listan och räknar hur många som är jämna.

Varför listor är viktiga

Listor används ofta när program behöver

lagra många värden
bearbeta data
göra beräkningar
analysera information

Övning

Skriv ett program som

1 skapar en tom lista
2 ber användaren skriva **fem tal**
3 lägger till talen i listan
4 räknar ut **summan av alla tal**
5 skriver ut både **listan och summan**.

7.7 Övningar

I dessa övningar ska du använda

listor
loopar
`append()`
index
beräkningar med data i listor

Övning 1

Skapa en lista med fem tal.

```
numbers = [3,7,2,9,5]
```

Gör sedan följande

1 Skriv ut hela listan.
2 Skriv ut det första talet i listan.
3 Skriv ut det sista talet i listan.

Övning 2

Skapa en lista med fem tal.

Skriv ett program som skriver ut **alla tal i listan** med en `for`-loop.

Övning 3

Skapa en lista

```
numbers = [4,6,2,8,10]
```

Skriv ett program som räknar ut **summan av alla tal i listan**.

Övning 4

Skapa en lista

```
numbers = [4,6,2,8,10]
```

Skriv ett program som räknar ut **medelvärdet** av talen i listan.

Tips: använd `len()`.

Övning 5

Skriv ett program som

1 skapar en tom lista
2 ber användaren skriva **fem tal**
3 lägger till talen i listan med `append()`
4 skriver ut hela listan

Övning 6

Skriv ett program som

1 skapar en lista med tal
2 går igenom listan med en loop
3 skriver ut **alla jämna tal** i listan

Övning 7

Skriv ett program som

1 skapar en lista med tal
2 går igenom listan
3 räknar hur många tal som är **större än 5**

Programmet ska skriva ut resultatet.

7.8 Reflektionsfrågor

1 - Vad är en lista i Python?

2 - Varför används listor i program?

3 - Vad betyder index i en lista?

4 - Varför börjar index på **0** i Python?

5 - Hur kan man gå igenom alla värden i en lista?

6 - Vad gör metoden `append()`?

7 - Hur kan man räkna ut summan av alla tal i en lista?

8 - Vad gör funktionen `len()` när den används på en lista?

9 - Ge ett exempel på när listor kan vara användbara i ett program.

10 - Varför används listor ofta tillsammans med loopar?

Gör nu inlämningsuppgifter till kapitel 1-7

8 - Funktioner

8.1 Varför funktioner används

När program blir större kan koden bli svår att läsa och svår att ändra.

Ett sätt att göra program mer organiserade är att använda **funktioner**.

En funktion är en **namngiven del av ett program** som utför en bestämd uppgift.

I stället för att skriva samma kod flera gånger kan man lägga koden i en funktion och sedan använda den när den behövs.

Exempel utan funktion

Programmet skriver ut samma text flera gånger.

```
print("Hej och välkommen")
print("Hej och välkommen")
print("Hej och välkommen")
```

Det fungerar, men koden blir onödigt lång.

Om man vill ändra texten måste man ändra på flera ställen.

Exempel med funktion

Med en funktion kan man skriva koden en gång och sedan använda den flera gånger.

```
def greet():
     print("Hej och välkommen")
     greet()
     greet()
     greet()
```

Här definieras först funktionen `greet()`.

Sedan anropas funktionen tre gånger.

Varför funktioner är bra

Funktioner gör program

lättare att läsa

lättare att ändra

lättare att återanvända

De hjälper också programmeraren att dela upp ett program i mindre delar.

Det är särskilt bra när program börjar bli längre.

Viktigt att tänka på

I Python börjar en funktion med ordet `def`.

Efter namnet skriver man parenteser och kolon.

Koden inuti funktionen måste vara **indragen**.

Exempel

```
def greet():
     print("Hej och välkommen")
```

Övning

Skriv ett program som innehåller en funktion som skriver ut texten

```
Välkommen till Python
```

Anropa funktionen **tre gånger**.

8.2 Definiera funktioner

I Python skapas funktioner med ordet `def`.

Efter funktionsnamnet skriver man parenteser `()` och ett kolon `:`.

Koden som tillhör funktionen måste skrivas **indragen på nästa rad**.

Grundstruktur

```
def funktion():
     kod som ska köras
```

Exempel

Här definieras en funktion som hälsar på en person.

```
def greet(name):
     print("Hej", name)
```

Funktionen har en **parameter** som heter `name`.

En parameter är ett värde som skickas in till funktionen.

För att använda funktionen måste den **anropas**.

```
greet("Anna")
greet("Erik")
```

Programmet skriver ut

```
Hej Anna
Hej Erik
```

Hur programmet fungerar

1 Programmet definierar funktionen `greet`.
2 Funktionen har parametern `name`.
3 När funktionen anropas skickas ett namn in.
4 Funktionen skriver ut en hälsning.

Exempel med f-sträng

Man kan också använda f-strängar.

```
def greet(name):
     print(f"Hej {name}")
     greet("Sara")
```

Övning

Skapa en funktion som heter `welcome`.

Funktionen ska ta emot ett namn och skriva ut

```
Välkommen {namn}
```

Exempel på användning

```
welcome("Alex")
```

Programmet ska skriva ut

Välkommen Alex

8.3 Parametrar

När en funktion behöver information för att kunna arbeta används **parametrar**.

En parameter är ett värde som skickas in till funktionen när den anropas.

Parametrar gör att samma funktion kan användas med olika värden.

Exempel

Här har funktionen en parameter som heter `name`.

```
def greet(name):
     print(f"Hej {name}")
```

När funktionen anropas skickas ett namn in.

```
greet("Anna")
greet("Erik")
```

Programmet skriver ut

Hej Anna
Hej Erik

Funktionen är densamma, men resultatet blir olika beroende på vilket värde som skickas in.

Flera parametrar

En funktion kan också ha flera parametrar.

```
def introduce(name, age):
     print(f"{name} är {age} år gammal")
     introduce("Sara", 17)
```

Programmet skriver ut

Sara är 17 år gammal

Hur parametrar fungerar

1 Funktionen definieras med parametrar.
2 När funktionen anropas skickas värden in.
3 Funktionen använder värdena i sin kod.

Övning

Skapa en funktion som heter `add`.

Funktionen ska ta emot **två tal** och skriva ut summan.

Exempel

```
add(3, 5)
```

Programmet ska skriva ut

Summan är 8

8.4 Return

Ibland vill man att en funktion **ska ge tillbaka ett värde**.

Då används ordet `return`.

`return` skickar tillbaka ett resultat från funktionen till den plats där funktionen anropas.

Exempel

Här finns en funktion som räknar ut summan av två tal.

```
def add(a, b):
     return a + b
```

När funktionen anropas returneras summan.

```
result = add(3, 5)
print(result)
```

Programmet skriver ut

8

Vad som händer i programmet

1 Funktionen `add` tar emot två tal.
2 Funktionen räknar ut summan.

3 Summan skickas tillbaka med `return`.
4 Resultatet sparas i variabeln `result`.

Varför return används

Med `return` kan funktioner

beräkna värden
skicka tillbaka resultat
användas i andra beräkningar

Exempel

```
def add(a, b):
      return a + b
      total = add(10, 7)
      print(f"Summan är {total}")
```

Programmet skriver ut

Summan är 17

Övning

Skapa en funktion som heter `multiply`.

Funktionen ska

ta emot två tal
returnera produkten av talen

Använd sedan funktionen och skriv ut resultatet.

8.5 Strukturera program

När program blir större är det viktigt att **strukturera koden**.
Det innebär att dela upp programmet i **mindre delar** som är lättare att läsa och förstå.

Funktioner hjälper programmeraren att organisera programmet.

Ett vanligt sätt att strukturera program är

1 funktioner högst upp i programmet
2 huvudprogram längst ner

Exempel

Här finns två funktioner som används i programmet.

```
def greet(name):
     print(f"Hej {name}")

def add(a, b):
     return a + b
```

Sedan används funktionerna i huvudprogrammet.

```
name = input("Vad heter du? ")
greet(name)
result = add(4, 6)
print(f"Summan är {result}")
```

Varför struktur är viktigt

Strukturerad kod gör program

lättare att läsa
lättare att felsöka
lättare att vidareutveckla

Det gör också att funktioner kan **återanvändas flera gånger**.

Bra vanor i Python

När man skriver program är det bra att

använda tydliga funktionsnamn
dela upp program i funktioner
skriva kod som är lätt att läsa

Övning

Skriv ett program med två funktioner.

Den första funktionen ska skriva ut en hälsning.

```
def greet(name):
```

Den andra funktionen ska räkna ut summan av två tal.

```
def add(a, b):
```

Använd funktionerna i programmet.

8.6 Större program

När program växer blir det viktigt att **dela upp programmet i flera funktioner**.

Varje funktion ska göra **en tydlig uppgift**.

Det gör programmet

lättare att läsa
lättare att testa
lättare att ändra

Ett större program kan därför bestå av flera funktioner som samarbetar.

Exempel

Här finns ett program som använder flera funktioner.

```
def greet(name):
     print(f"Hej {name}")

def add(a, b):
     return a + b

def multiply(a, b):
     return a * b
```

Huvudprogrammet använder funktionerna.

```
name = input("Vad heter du? ")
greet(name)
x = int(input("Skriv ett tal: "))
y = int(input("Skriv ett tal: "))
sum_result = add(x, y)
product_result = multiply(x, y)
```

```
print(f"Summan är {sum_result}")
print(f"Produkten är {product_result}")
```

Hur programmet är uppbyggt

Programmet består av tre delar

funktioner
huvudprogram
utskrift av resultat

Varje funktion gör en egen uppgift.

Fördelar med denna struktur

Programmet blir mer organiserat.

Det blir också lättare att

lägga till nya funktioner
ändra delar av programmet
återanvända funktioner i andra program

Övning

Skapa ett program med tre funktioner.

`def greet(name)`
ska skriva ut en hälsning.

`def square(number)`
ska returnera talet multiplicerat med sig självt.

`def double(number)`
ska returnera talet gånger två.

Programmet ska

fråga efter ett namn
fråga efter ett tal
använda funktionerna
skriva ut resultaten.

8.7 Övningar

I dessa övningar ska du använda

funktioner
parametrar
return
variabler
input
f-strängar

Övning 1

Skapa en funktion som heter `greet`.

Funktionen ska ta emot ett namn och skriva ut

Hej {namn}

Exempel

```
greet("Anna")
```

Övning 2

Skapa en funktion som heter `square`.

Funktionen ska ta emot ett tal och returnera talet multiplicerat med sig självt.

Exempel

```
square(4)
```

Resultat

16

Övning 3

Skapa en funktion som heter `add`.

Funktionen ska ta emot två tal och returnera summan.

Programmet ska sedan skriva ut resultatet.

Övning 4

Skapa en funktion som heter `age_in_five_years`.

Funktionen ska ta emot en ålder och returnera hur gammal personen är om fem år.

Programmet ska fråga användaren efter ålder och skriva ut resultatet.

Övning 5

Skapa en funktion som heter `area`.

Funktionen ska ta emot två värden

bredd
höjd

Funktionen ska returnera arean av en rektangel.

Formel

area = bredd * höjd

Programmet ska skriva ut resultatet.

Övning 6

Skapa ett program med två funktioner.

`def greet(name)`
ska skriva ut en hälsning.

`def double(number)`
ska returnera talet gånger två.

Programmet ska

fråga efter ett namn
fråga efter ett tal
använda funktionerna
skriva ut resultatet.

8.8 Reflektionsfrågor

1 - Vad är en funktion i Python?

2 - Varför använder man funktioner i program?

3 - Vad betyder ordet `def` i Python?

4 - Vad är en parameter?

5 - Vad är skillnaden mellan en funktion som **skriver ut ett värde** och en funktion som använder **return**?

6 - Varför är det bra att dela upp större program i flera funktioner?

7 - Vad händer om man anropar en funktion flera gånger i ett program?

8 - När kan det vara bättre att använda `return` istället för `print` i en funktion?

9 - Ge ett exempel på en uppgift där en funktion kan vara användbar.

10 - Varför blir program ofta lättare att läsa när man använder funktioner?

9 - Felsökning

9.1 Syntaxfel

När man programmerar gör man ofta misstag i koden.
Ett vanligt fel är **syntaxfel**.

Ett syntaxfel betyder att **koden inte följer språkets regler**.
Python kan då inte köra programmet.

När ett syntaxfel uppstår visar Python ett **felmeddelande**.

Felmeddelandet talar ofta om

vilken rad felet finns på
vilken typ av fel som uppstod

Exempel på syntaxfel

I Python måste parenteser stängas.

```
print("Hej världen"
```

Här saknas en avslutande parentes.

Programmet kan därför inte köras.

Exempel med saknat kolon

Efter `if` måste man skriva kolon.

Fel kod

```
age = 18
if age >= 18
     print("Du är myndig")
```

Rätt kod

```
age = 18
if age >= 18:
     print("Du är myndig")
```

Exempel med stavfel

Om man skriver fel på ett kommando kan Python inte förstå koden.

Fel kod

```
pritn("Hej")
```

Rätt kod

```
print("Hej")
```

Hur man hittar syntaxfel

När Python visar ett felmeddelande är det bra att

läsa felmeddelandet
 titta på raden som anges
 kontrollera parenteser, kolon och stavning

Övning

Vilka syntaxfel finns i följande kod?

```
print("Hej världen"
age = 18
if age >= 18
print("Du är myndig")
```

9.2 Logiska fel

Ett **logiskt fel** uppstår när programmet kan köras utan felmeddelanden men ändå ger **fel resultat**.

Python hittar inte dessa fel automatiskt eftersom koden är korrekt skriven enligt språkets regler. Problemet är istället **hur programmet är konstruerat**.

Logiska fel är mycket vanliga när man programmerar.

Exempel på logiskt fel

Programmet nedan ska räkna ut summan av två tal.

Fel kod

```
a = 5
b = 3
summa = a - b
print(summa)
```

Programmet körs utan problem, men resultatet blir **fel** eftersom man råkade använda minus istället för plus.

Rätt kod

```
a = 5
b = 3
summa = a + b
print(summa)
```

Ett annat exempel

Programmet ska lägga till 10 cm till en längd.

Fel kod

```
length = 170
length + 10
print(length)
```

Resultatet blir fortfarande **170** eftersom värdet aldrig sparades i variabeln.

Rätt kod

```
length = 170
length = length + 10
print(length)
```

Hur man hittar logiska fel

Logiska fel kan vara svårare att upptäcka än syntaxfel. Några vanliga metoder är

skriva ut variabler med `print()` för att kontrollera värden
testa programmet med olika indata
gå igenom koden rad för rad

Övning

Programmet ska räkna ut medelvärdet av två tal. Koden innehåller ett logiskt fel.

```
a = 10
b = 20
average = a + b / 2
print(average)
```

Frågor

1 - Vad blir resultatet när programmet körs?
2 - Varför blir resultatet fel?
3 - Hur ska koden ändras för att beräkna medelvärdet korrekt?

9.3 Exekveringsfel

Ett **exekveringsfel** uppstår när programmet startar men kraschar medan det körs.

Koden är alltså **syntaktiskt korrekt**, men något händer under körningen som gör att programmet inte kan fortsätta.

Python visar då ett **felmeddelande** som talar om vad som gick fel.

Exekveringsfel kallas ibland **runtime error**.

Vanliga orsaker är

fel typ av data
division med noll
försök att använda en variabel som inte finns

Exempel: fel datatyp

Programmet försöker omvandla text till ett tal.

```
age = int(input("Hur gammal är du? "))
print(age + 5)
```

Om användaren skriver

hej

kan Python inte omvandla texten till ett heltal.
Programmet kraschar då med ett felmeddelande.

Exempel: division med noll

Division med noll är inte tillåten i matematik.

```
number = 10
result = number / 0
print(result)
```

Programmet kraschar eftersom man inte kan dela ett tal med noll.

Exempel: variabel som inte finns

```
name = "Anna"
print(namn)
```

Här finns ett stavfel.
Variabeln `namn` existerar inte.

Python visar då ett felmeddelande.

Hur man hittar exekveringsfel

När ett exekveringsfel uppstår kan du

läsa felmeddelandet noggrant
kontrollera vilken rad felet uppstod på
kontrollera indata från användaren
testa programmet med olika värden

Övning

Koden nedan innehåller ett exekveringsfel.

```
number = int(input("Skriv ett tal: "))
result = 100 / number
print(result)
```

Frågor

1 - Vad händer om användaren skriver **0**?
2 - Varför uppstår ett fel?
3 - Hur skulle man kunna undvika detta problem i ett program?

9.4 Läsa felmeddelanden

När ett program innehåller fel visar Python ett **felmeddelande**.

Felmeddelandet hjälper programmeraren att förstå **vad som gick fel** och **var felet finns i koden**.

Ett felmeddelande innehåller ofta

filens namn
radnumret där felet uppstod
vilken typ av fel det är

Exempel

Om följande kod körs

```
age = int(input("Hur gammal är du? "))
print(age + 5)
```

och användaren skriver

hej

kan Python visa ett felmeddelande som liknar detta

```
ValueError: invalid literal for int()
```

Det betyder att Python försökte omvandla text till ett heltal men misslyckades.

Exempel med division med noll

```
number = 10
result = number / 0
print(result)
```

Python visar då ett felmeddelande som

```
ZeroDivisionError: division by zero
```

Det betyder att programmet försökte dela ett tal med noll.

Vad man ska göra när ett fel uppstår

När ett felmeddelande visas kan du

läsa vilken typ av fel det är
titta på raden där felet uppstod
kontrollera variabler och beräkningar
testa programmet igen efter ändring

Att **lära sig läsa felmeddelanden** är en viktig del av programmering.

Övning

Koden nedan innehåller ett fel.

```
age = input("Hur gammal är du? ")
result = age + 5
print(result)
```

Frågor

1 - Vilken typ av fel uppstår när programmet körs?
2 - Varför uppstår felet?
3 - Hur kan koden ändras så att programmet fungerar?

9.5 Systematisk felsökning

När ett program inte fungerar är det viktigt att **felsöka steg för steg**.

Systematisk felsökning betyder att man undersöker problemet på ett lugnt och ordnat sätt istället för att bara gissa.

Det är ett viktigt arbetssätt i programmering.

Ett bra arbetssätt

När du felsöker kan du göra så här

läs felmeddelandet noggrant
titta på raden där felet finns
kontrollera variabler och värden
testa små delar av programmet
ändra en sak i taget

Om man ändrar för många saker samtidigt blir det svårare att förstå vad som faktiskt löste problemet.

Använd print vid felsökning

Ett enkelt sätt att felsöka är att skriva ut värden med `print()`.

Exempel

```
age = int(input("Hur gammal är du? "))
print(age)
```

Då kan du kontrollera vilket värde variabeln faktiskt har.

Exempel

Programmet ska räkna ut summan av två tal, men resultatet blir fel.

```
a = 10
b = 5
summa = a - b
print(summa)
```

Om du felsöker systematiskt kan du upptäcka att operatorn är fel.

Rätt kod är

```
a = 10
b = 5
summa = a + b
print(summa)
```

Testa små delar

Om ett program är långt kan det vara bra att testa en liten del i taget.

Till exempel kan du först kontrollera att input fungerar.

```
name = input("Vad heter du? ")
print(name)
```

När den delen fungerar kan du gå vidare.

Sammanfattning

Systematisk felsökning betyder att du

arbetar steg för steg
läser felmeddelanden
kontrollerar värden
testar små delar av programmet
ändrar en sak i taget

Det gör det lättare att hitta och rätta fel.

Övning

Programmet nedan ska räkna ut medelvärdet av två tal, men något är fel.

```
a = int(input("Skriv första talet: "))
b = int(input("Skriv andra talet: "))
average = a + b / 2
print(f"Medelvärdet är {average}")
```

Frågor

1 - Vad blir fel i programmet?
2 - Hur kan du använda `print()` för att felsöka?
3 - Hur ska koden ändras för att ge rätt medelvärde?

9.6 Testa program

När man skriver program är det viktigt att **testa att programmet fungerar korrekt**.

Att testa ett program betyder att man kör programmet med olika indata för att kontrollera att resultatet blir rätt.

Testning hjälper programmeraren att upptäcka

logiska fel
felaktiga beräkningar
problem med indata

Testa med olika värden

Ett program bör testas med flera olika värden.

Exempel

```
age = int(input("Hur gammal är du? "))
print(age + 5)
```

Man kan testa programmet med olika åldrar

10
18
30

Då kan man kontrollera att programmet alltid ger rätt resultat.

Testa gränsvärden

Det är också bra att testa **gränsvärden**.

Gränsvärden är värden som ligger nära en gräns i programmet.

Exempel

```
age = int(input("Hur gammal är du? "))
if age >= 18:
     print("Du är myndig")
else:
     print("Du är inte myndig")
```

Bra testvärden kan vara

17
18
19

Då kan man kontrollera att programmet hanterar gränsen korrekt.

Testa felaktig indata

Det är också viktigt att tänka på vad som händer om användaren skriver något oväntat.

Exempel

```
age = int(input("Hur gammal är du? "))
```

Om användaren skriver

hej

kommer programmet att krascha.

I senare kapitel kommer vi att lära oss hur man kan **hantera sådana fel**.

Övning

Programmet nedan räknar ut hur gammal en person är om tio år.

```
age = int(input("Hur gammal är du? "))
future_age = age + 10
print(f"Om tio år är du {future_age} år gammal")
```

Frågor

1 - Vilka olika värden kan du testa i programmet?
2 - Varför är det bra att testa flera olika värden?
3 - Vad händer om användaren skriver text istället för ett tal?

9.7 Övningar

I dessa övningar ska du träna på att **hitta och rätta fel i program**.

Övningarna innehåller olika typer av fel

syntaxfel
logiska fel
exekveringsfel

Övning 1

Programmet ska skriva ut en hälsning, men koden innehåller ett fel.

```
name = input("Vad heter du? ")
print("Hej " name)
```

Frågor

1 - Vilket fel finns i koden?
2 - Hur ska koden ändras för att fungera?

Övning 2

Programmet ska räkna ut summan av två tal, men resultatet blir fel.

```
a = 8
b = 4
summa = a - b
print(summa)
```

Frågor

1 - Vad är felet i programmet?
2 - Hur ska koden ändras?

Övning 3

Programmet kraschar när det körs.

```
number = int(input("Skriv ett tal: "))
result = 50 / number
print(result)
```

Frågor

1 - Vad händer om användaren skriver **0**?
2 - Varför uppstår felet?

Övning 4

Programmet ska räkna ut medelvärdet av två tal.

```
a = 10
b = 20
average = a + b / 2
print(average)
```

Frågor

1 - Varför blir resultatet fel?
2 - Hur ska koden ändras?

Övning 5

Programmet ska använda en funktion, men något saknas.

```
def greet(name):
print(f"Hej {name}")

greet
```

Frågor

1 - Varför skrivs inget ut?
2 - Hur ska funktionen anropas?

Övning 6

Programmet ska skriva ut talen 1 till 5.

```
for i in range(5)
print(i)
```

Frågor

1 - Vilket syntaxfel finns i koden?
2 - Hur ska koden ändras?

9.8 Reflektionsfrågor

1 - Vad är ett syntaxfel?

2 - Vad är ett logiskt fel?

3 - Vad är ett exekveringsfel?

4 - Vilken typ av fel är lättast att upptäcka, och varför?

5 - Hur kan felmeddelanden hjälpa dig när du programmerar?

6 - Varför är det viktigt att testa program med olika värden?

7 - Hur kan `print()` hjälpa dig att felsöka ett program?

8 - Vad betyder det att felsöka systematiskt?

9 - Vad kan hända om man inte kontrollerar användarens inmatning?

10 - Hur kan du bli bättre på att hitta fel i din kod?

10 - Undantagshantering

10.1 Vad är undantag

I tidigare kapitel har du sett att program kan krascha när något går fel.
Till exempel om användaren skriver fel typ av data eller om man delar med noll.

Ett **undantag** är ett fel som uppstår när programmet körs.

Istället för att programmet kraschar kan vi i Python **fånga felet och hantera det**.
Detta kallas **undantagshantering**.

Varför använda undantag

Undantag gör att program kan

fortsätta köra trots fel
ge tydliga felmeddelanden till användaren
hantera oväntad inmatning

Det gör program mer stabila och användarvänliga.

Exempel utan undantag

```
age = int(input("Hur gammal är du? "))
print(age + 5)
```

Om användaren skriver

hej

kraschar programmet.

Exempel med undantag

```
try:
      age = int(input("Hur gammal är du? "))
      print(age + 5)
except:
      print("Du måste skriva ett tal")
```

Nu kraschar inte programmet.
Istället får användaren ett meddelande.

Hur det fungerar

`try` används för kod som kan orsaka fel
`except` körs om ett fel uppstår

Programmet försöker först köra koden i `try`.
Om något går fel hoppar Python till `except`.

Övning

Skriv ett program som

frågar användaren efter ett tal
försöker omvandla det till ett heltal
skriver ut talet plus 10

Om användaren skriver något som inte är ett tal ska programmet skriva ut

Fel inmatning

10.2 Hantera felaktig input

När ett program tar emot indata från användaren finns det alltid en risk att användaren skriver något oväntat.

Det kan till exempel vara

text istället för tal
tom inmatning
fel format

Därför är det viktigt att kunna **hantera felaktig input**.

Problem utan felhantering

```
age = int(input("Hur gammal är du? "))
print(age + 5)
```

Om användaren skriver

hej

kraschar programmet eftersom text inte kan omvandlas till ett tal.

Lösning med try except

```
try:
      age = int(input("Hur gammal är du? "))
      print(age + 5)
except:
      print("Fel inmatning")
```

Programmet kraschar inte längre.
Istället får användaren ett tydligt meddelande.

Be användaren försöka igen

Ibland vill man att användaren ska få flera försök.

Det kan man göra med en loop.

```
while True:
      try:
            age = int(input("Hur gammal är du? "))
            print(age + 5)
            break
except:
      print("Fel inmatning, försök igen")
```

Programmet fortsätter tills användaren skriver ett korrekt värde.

Varför detta är viktigt

Att hantera felaktig input gör program

mer robusta
mer användarvänliga
mer pålitliga

Övning

Skriv ett program som

frågar användaren efter ett tal
fortsätter fråga tills användaren skriver ett giltigt heltal
skriver ut talet multiplicerat med 2

10.3 Robusta program

Ett **robust program** är ett program som klarar av oväntade situationer utan att krascha.

Det betyder att programmet kan hantera

felaktig input
ovanliga värden
oväntade situationer

Robusta program är viktiga eftersom användare inte alltid gör som programmeraren tänkt.

Exempel på icke robust program

```
number = int(input("Skriv ett tal: "))
print(100 / number)
```

Problem

om användaren skriver text kraschar programmet
om användaren skriver 0 kraschar programmet

Exempel på robust program

```
while True:
    try:
        number = int(input("Skriv ett tal: "))
        if number == 0:
            print("Du kan inte dela med 0")
        else:
            print(100 / number)
            break
    except:
        print("Fel inmatning, försök igen")
```

Programmet

hanterar felaktig input
hanterar division med noll
låter användaren försöka igen

Vad gör ett program robust

Ett robust program

kontrollerar användarens inmatning
hanterar fel med try except
ger tydliga meddelanden
låter användaren försöka igen

Varför detta är viktigt

Robusta program är

mer stabila
lättare att använda
mer professionella

Övning

Skriv ett program som

frågar användaren efter två tal
 fortsätter fråga tills båda talen är giltiga heltal
 skriver ut resultatet av divisionen

Programmet ska

 inte krascha
 hantera division med noll
 ge tydliga felmeddelanden

10.4 Övningar

I dessa övningar ska du träna på

try except
 hantera felaktig input
 skapa robusta program

Övning 1

Skriv ett program som

frågar användaren efter ett tal
 försöker omvandla det till ett heltal
 skriver ut talet multiplicerat med 3

Om användaren skriver fel ska programmet skriva

Fel inmatning

Övning 2

Skriv ett program som

frågar användaren efter ett tal
 fortsätter fråga tills användaren skriver ett giltigt heltal
 skriver ut talet plus 10

Övning 3

Skriv ett program som

frågar användaren efter två tal
 försöker omvandla dem till heltal
 skriver ut summan

Om något blir fel ska programmet skriva

Fel inmatning

Övning 4

Skriv ett program som

frågar användaren efter två tal
 fortsätter fråga tills båda talen är giltiga
 skriver ut produkten

Övning 5

Skriv ett program som

frågar användaren efter ett tal
 kontrollerar att talet inte är 0
 skriver ut 100 delat med talet

Programmet ska

hantera felaktig input
 hantera division med noll
 inte krascha

Övning 6

Skriv ett program som

frågar användaren efter ett tal
 fortsätter fråga tills användaren skriver ett giltigt tal större än 0
 skriver ut talet multiplicerat med 5

Programmet ska ge tydliga felmeddelanden om något blir fel.

10.5 Reflektionsfrågor

1 - Vad är ett undantag i Python?

2 - Vad är syftet med `try` och `except`?

3 - Vad händer om ett fel uppstår i ett `try`-block?

4 - Varför är det viktigt att hantera felaktig input?

5 - Vad innebär det att ett program är robust?

6 - Hur kan man göra ett program mer användarvänligt med hjälp av undantag?

7 - Varför är det bättre att få ett felmeddelande än att programmet kraschar?

8 - Hur fungerar en loop tillsammans med `try except`?

9 - Vad kan hända om man inte hanterar division med noll i ett program?

10 - Hur kan du förbättra ett program så att det klarar oväntade situationer?

11 - Struktur och läsbar kod

11.1 Namngivning1

När man programmerar är det viktigt att använda **tydliga namn på variabler och funktioner**.

Bra namn gör koden

lättare att läsa
lättare att förstå
lättare att felsöka

Dåliga namn

```
x = 10
y = 20
z = x + y
```

Det är svårt att förstå vad variablerna betyder.

Bra namn

```
width = 10
height = 20
area = width + height
```

Nu blir det tydligare vad programmet gör.

Namn på funktioner

Funktioner bör ha namn som beskriver vad de gör.

Dåligt exempel

```
def f(a, b):
     return a + b
```

Bra exempel

```
def add(a, b):
     return a + b
```

Regler i Python

Variabelnamn

ska börja med en bokstav eller underscore
får innehålla bokstäver och siffror
får inte vara ett reserverat ord

Bra vanor

Använd engelska namn i kod
Använd små bokstäver
Använd tydliga ord istället för förkortningar

Exempel

```
total_price = 100
user_name = "Anna"
```

Sammanfattning

Bra namngivning gör att

koden blir tydligare
det blir lättare att förstå programmet
det blir enklare att arbeta vidare med koden

Övning

Förbättra namnen i koden nedan

```
a = 5
b = 10
c = a * b
print(c)
```

11.2 Kommentarer

Kommentarer används för att **förklara kod**.

En kommentar skrivs med tecknet # och körs inte av programmet.

Exempel

```
# detta är en kommentar
print("Hej")
```

Kommentarer används för att göra koden lättare att förstå för andra och för dig själv.

När man ska använda kommentarer

Kommentarer är bra när man vill

förklara vad programmet gör
förklara svåra delar av koden
beskriva syftet med variabler eller funktioner

Exempel

```
# fråga användaren efter ålder
age = int(input("Hur gammal är du? "))

# räkna ut ålder om 5 år
future_age = age + 5
```

```
print(f"Om fem år är du {future_age} år gammal")
```

När man inte ska använda kommentarer

Man ska inte skriva kommentarer som bara upprepar vad koden redan visar.

Dåligt exempel

```
# sätt x till 10
x = 10
```

Det är redan tydligt vad koden gör.

Bättre kod istället för kommentarer

Ibland är det bättre att skriva tydligare kod istället för att använda kommentarer.

Dåligt exempel

```
# räkna ut area
a = w * h
```

Bättre exempel

```
area = width * height
```

Nu behövs ingen kommentar eftersom variabelnamnen är tydliga.

Kommentarer i funktioner

Kommentarer är extra viktiga i funktioner.

```
def add(a, b):
# returnerar summan av två tal
     return a + b
```

Sammanfattning

Använd kommentarer för att

förklara varför något görs
förklara svår kod

Undvik kommentarer som förklarar självklar kod. Det beror på vem som ska läsa koden hur utförlig den ska vara.

Övning

Lägg till lämpliga kommentarer i koden

```
name = input("Vad heter du? ")
age = int(input("Hur gammal är du? "))
future_age = age + 10
print(f"Hej {name}")
print(f"Om tio år är du {future_age} år gammal")
```

11.3 Strukturera kod

Att strukturera kod betyder att man organiserar programmet så att det blir **lätt att läsa, förstå och vidareutveckla**.

Bra struktur gör stor skillnad när program blir större.

Dela upp i funktioner

Istället för att skriva all kod i en lång lista kan man dela upp programmet i funktioner.

Dåligt exempel

```
name = input("Vad heter du? ")
age = int(input("Hur gammal är du? "))
future_age = age + 5
print(f"Hej {name}")
print(f"Om fem år är du {future_age} år gammal")
```

Bättre exempel

```
def greet(name):
     print(f"Hej {name}")

def calculate_future_age(age):
     return age + 5

name = input("Vad heter du? ")
age = int(input("Hur gammal är du? "))
greet(name)
```

```
future_age = calculate_future_age(age)
print(f"Om fem år är du {future_age} år gammal")
```

Använd tydliga steg

Ett program blir tydligare om det följer en struktur

input
bearbetning
output

Exempel

```
# input
age = int(input("Hur gammal är du? "))

# bearbetning
future_age = age + 5

# output
print(f"Om fem år är du {future_age} år gammal")
```

Undvik onödigt upprepad kod

Om samma kod används flera gånger bör man använda funktioner.

Dåligt exempel

```
print("Hej Anna")
print("Hej Erik")
print("Hej Sara")
```

Bättre exempel

```
def greet(name):
     print(f"Hej {name}")

greet("Anna")
greet("Erik")
greet("Sara")
```

Sammanfattning

Bra struktur innebär att

program delas upp i funktioner
koden följer tydliga steg
upprepning undviks

Det gör programmet

lättare att läsa
lättare att felsöka
lättare att utveckla

Övning

Gör om koden så att den använder funktioner

```
name = input("Vad heter du? ")
number = int(input("Skriv ett tal: "))
result = number * 2
print(f"Hej {name}")
print(f"Ditt tal gånger två är {result}")
```

11.4 Dela upp program och läslig kod

När program blir större är det viktigt att både **dela upp programmet** och skriva **läsbar kod**.

Det gör att programmet blir lättare att förstå, ändra och felsöka.

Dela upp program i delar

Ett program bör delas upp i mindre delar med hjälp av funktioner.

Varje funktion ska ha en tydlig uppgift.

Exempel

```
def greet(name):
     print(f"Hej {name}")

def double(number):
     return number * 2
```

```
name = input("Vad heter du? ")
number = int(input("Skriv ett tal: "))

greet(name)

result = double(number)
print(f"Resultatet är {result}")
```

Läslig kod

Kod ska vara lätt att läsa, både för dig själv och för andra.

Det uppnås genom

tydliga variabelnamn
tydliga funktioner
bra struktur
korrekta indrag

Exempel på oläslig kod

```
n=input("namn")
a=int(input("ålder"))
print("hej",n,a+5)
```

Exempel på läsbar kod

```
name = input("Vad heter du? ")
age = int(input("Hur gammal är du? "))

future_age = age + 5

print(f"Hej {name}")
print(f"Om fem år är du {future_age} år gammal")
```

Tänk på

Lämna tomma rader mellan delar av programmet
Använd konsekvent stil
Undvik onödigt komplex kod

Sammanfattning

Att dela upp program och skriva läsbar kod gör att

program blir tydligare
det blir enklare att hitta fel
det blir lättare att utveckla vidare

Övning

Förbättra koden så att den blir mer läsbar

```
x=input("namn")
y=int(input("tal"))
z=y*2
print("hej",x)
print(z)
```

11.5 Övningar

I dessa övningar ska du träna på

namngivning
kommentarer
struktur
läsbar kod

Övning 1

Förbättra namnen i koden

```
a = 5
b = 10
c = a + b
print(c)
```

Övning 2

Lägg till kommentarer som förklarar vad koden gör

```
name = input("Vad heter du? ")
age = int(input("Hur gammal är du? "))
future_age = age + 5
```

```
print(f"Hej {name}")
print(f"Om fem år är du {future_age} år gammal")
```

Övning 3

Gör koden mer läsbar

```
n=input("namn")
x=int(input("tal"))
y=x*2
print("hej",n)
print(y)
```

Övning 4

Dela upp programmet i funktioner

```
name = input("Vad heter du? ")
number = int(input("Skriv ett tal: "))
result = number * 2
print(f"Hej {name}")
print(f"Ditt tal gånger två är {result}")
```

Övning 5

Skriv ett eget program som

frågar användaren efter namn och ålder
räknar ut hur gammal personen är om 10 år
använder funktioner
har tydliga variabelnamn
innehåller minst två kommentarer

Programmet ska vara lätt att läsa och förstå.

11.6 Reflektionsfrågor

1 - Varför är det viktigt att använda tydliga variabelnamn?

2 - Vad kan hända om man använder otydliga namn i sin kod?

3 - När är det bra att använda kommentarer i ett program?

4 - När är det onödigt att använda kommentarer?

5 - Vad innebär det att strukturera kod?

6 - Varför är det bra att dela upp program i funktioner?

7 - Vad kännetecknar läsbar kod?

8 - Hur kan man göra ett program lättare att förstå för andra?

9 - Varför är det viktigt att skriva kod som man själv förstår i efterhand?

10 - Vad kan du göra för att förbättra din egen kod i framtiden?

Gör nu inlämningsuppgifter till kapitel 1-11

12 - Användargränssnitt

12.1 Program och användare

Ett program används nästan alltid av en **användare**.
Därför är det viktigt att programmet kan **kommunicera tydligt** med den som använder det.

Detta kallas **användargränssnitt**.

Ett användargränssnitt är det som användaren ser och interagerar med.

Olika typer av användargränssnitt

Det finns flera typer av gränssnitt

kommandorad
grafiska gränssnitt
webbgränssnitt

I denna kurs börjar vi med **kommandorad**, där användaren skriver in text.

Exempel

```
name = input("Vad heter du? ")
print(f"Hej {name}")
```

Här sker en enkel kommunikation

programmet frågar
användaren svarar
programmet ger ett resultat

Vad är ett bra användargränssnitt

Ett bra användargränssnitt är

tydligt
enkelt att förstå
lätt att använda

Exempel på tydligt program

```
name = input("Skriv ditt namn: ")
age = int(input("Skriv din ålder: "))

print(f"Hej {name}")
print(f"Du är {age} år gammal")
```

Exempel på otydligt program

```
n = input("x: ")
a = int(input("y: "))

print(n)
print(a)
```

Det är svårt att förstå vad programmet vill.

Tänk på

Skriv tydliga frågor
Skriv tydliga svar
Ge instruktioner till användaren

Sammanfattning

Program och användare kommunicerar genom gränssnittet.
Ett bra gränssnitt gör programmet enklare att använda.

12.2 Textbaserade menyer

En vanlig typ av användargränssnitt är **textbaserade menyer**.

En meny låter användaren välja mellan olika alternativ i ett program.

Det gör programmet mer strukturerat och lättare att använda.

Exempel på meny

```
print("1. Hälsa")
print("2. Räkna")
print("3. Avsluta")

choice = input("Välj ett alternativ: ")
```

Använda val med if

```
if choice == "1":
     print("Hej!")
elif choice == "2":
     number = int(input("Skriv ett tal: "))
     print(number * 2)
elif choice == "3":
     print("Programmet avslutas")
else:
     print("Ogiltigt val")
```

Göra menyn återkommande

Ofta vill man att menyn ska visas flera gånger.
Det kan göras med en loop.

```
while True:
     print("1. Hälsa")
     print("2. Räkna")
     print("3. Avsluta")
```

```
choice = input("Välj ett alternativ: ")

if choice == "1":
      print("Hej!")

elif choice == "2":
      number = int(input("Skriv ett tal: "))
      print(number * 2)

elif choice == "3":
      print("Programmet avslutas")
      Break

else:
print("Ogiltigt val")
```

Varför använda menyer

Menyer gör att

användaren vet vad som går att göra
programmet blir tydligare
det blir enklare att använda flera funktioner

Tänk på

Ge tydliga alternativ
hantera felaktiga val
använd loop så att programmet fortsätter

Övning

Skapa en meny med följande alternativ

1 Skriv ut ditt namn
2 Räkna ut ett tal gånger 3
3 Avsluta programmet

Programmet ska fortsätta tills användaren väljer att avsluta.

12.3 Introduktion till Tkinter

Hittills har du arbetat med **textbaserade program** i terminalen.
Men många program använder istället **grafiska användargränssnitt**.

I Python kan man skapa enkla grafiska program med biblioteket **Tkinter**.

Tkinter följer oftast med Python och används för att skapa fönster, knappar och text.

Vad är Tkinter

Tkinter är ett bibliotek i Python som används för att skapa **grafiska program**.

Med Tkinter kan man till exempel skapa

fönster
text
knappar
inmatningsfält

Skapa ett enkelt fönster

Först måste man importera Tkinter.

```
import tkinter as tk
```

Sedan kan man skapa ett fönster.

```
window = tk.Tk()
window.title("Mitt första program")
```

Till sist måste programmet hållas igång.

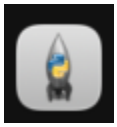
```
window.mainloop()
```

Detta program öppnar ett enkelt fönster med titeln **Mitt första program**.

Exempel: hela programmet

```
import tkinter as tk
window = tk.Tk()
window.title("Mitt första program")
window.mainloop()
```

Ett fönster som nedan dyker upp på din skärm och denna ikonen kummer upp i ditt appfält



Troligen läggs fönstert längst uppe till vänster på din skärm

Lägga till text

Man kan också lägga till text i fönstret.

```
import tkinter as tk

window = tk.Tk()
window.title("Exempel")

label = tk.Label(window, text="Hej världen")
label.pack()

window.mainloop()
```

Lägga till en knapp

Man kan även skapa en knapp.

```
import tkinter as tk
```

```
def say_hello():
     print("Hej!")

window = tk.Tk()

button = tk.Button(window, text="Klicka här", command=say_hello)
button.pack()

window.mainloop()
```

När användaren klickar på knappen körs funktionen. Så titta i din terminal när du klickar på knappen

Viktiga delar

`import tkinter as tk` importerar biblioteket
`tk.Tk()` skapar fönstret
`Label` visar text
`Button` skapar en knapp
`pack()` placerar objekt i fönstret
`mainloop()` håller fönstret öppet

Sammanfattning

Tkinter gör det möjligt att skapa enkla grafiska program i Python.

Det är ett första steg från textbaserade program till program med **grafiskt användargränssnitt**.

Övning

Skapa ett program med Tkinter som

visar ett fönster
har titeln **Mitt GUI-program**
innehåller en knapp
skriver ut **Hej** i terminalen när användaren klickar på knappen

12.4 TKinter input och uppdatera text

I detta kapitel bygger vi vidare på Tkinter och skapar mer användbara program.
Nu ska vi inte bara visa text och knappar, utan även låta användaren skriva in data och få ett resultat i fönstret.

Ta emot input i GUI

För att användaren ska kunna skriva något i programmet använder vi `Entry`.

```
import tkinter as tk

window = tk.Tk()
window.title("Input exempel")

entry = tk.Entry(window)
entry.pack()

window.mainloop()
```

Här kan användaren skriva text i ett fält.

Hämta värdet från inputfältet

För att använda det som användaren skriver använder vi `.get()`.

```
import tkinter as tk

def show_text():
     text = entry.get()
     print(text)

window = tk.Tk()

entry = tk.Entry(window)
entry.pack()

button = tk.Button(window, text="Visa text", command=show_text)
button.pack()

window.mainloop()
```

När knappen klickas hämtas texten från inputfältet.

Uppdatera text i fönstret

Istället för att skriva ut i terminalen kan vi visa resultat direkt i GUI.

```
import tkinter as tk

def greet():
      name = entry.get()
      label.config(text=f"Hej {name}")

window = tk.Tk()
window.title("GUI-program")

entry = tk.Entry(window)
entry.pack()

button = tk.Button(window, text="Säg hej", command=greet)
button.pack()

label = tk.Label(window, text="")
label.pack()

window.mainloop()
```

Programmet visar nu resultat direkt i fönstret.

Sammanfattning

I detta kapitel har du lärt dig att

- använda inputfält (Entry)
- hämta värden från användaren
- uppdatera text i GUI
- koppla knappar till funktioner

Detta gör att du kan skapa riktiga program med användarinteraktion.

Övningar

1 Skapa ett program där användaren skriver sitt namn och får en hälsning i fönstret.

2 Skapa ett program där användaren skriver ett tal och programmet visar talet gånger 2.

3 Utmaning
Skapa ett program där användaren skriver sin ålder och programmet visar hur gammal personen är om 5 år.

12.5 Tkinter Input och output, widgets, ändra utseende

I detta kapitel bygger vi vidare på Tkinter och lär oss fler funktioner så att vi kan skapa mer avancerade grafiska program.

Input och output i GUI

Du har redan sett hur man använder `Entry` och `Label`.
Detta är grunden i nästan alla GUI-program.

```
import tkinter as tk

def greet():
      name = entry.get()
      label.config(text=f"Hej {name}")

window = tk.Tk()

entry = tk.Entry(window)
entry.pack()

button = tk.Button(window, text="Säg hej", command=greet)
button.pack()

label = tk.Label(window, text="")
label.pack()

window.mainloop()
```

Flera widgets i samma program

Man kan ha flera knappar och funktioner i samma program.

```
import tkinter as tk
```

```
def double():
      value = int(entry.get())
      label.config(text=f"{value * 2}")

def triple():
      value = int(entry.get())
      label.config(text=f"{value * 3}")

window = tk.Tk()

entry = tk.Entry(window)
      entry.pack()

btn1 = tk.Button(window, text="x2", command=double)
btn1.pack()

btn2 = tk.Button(window, text="x3", command=triple)
btn2.pack()

label = tk.Label(window, text="")
label.pack()

window.mainloop()
```

Ändra utseende

Man kan ändra färg och storlek på text.

```
label = tk.Label(window, text="Hej", fg="blue", font=("Arial", 16))
label.pack()
```

Rensa inputfält

Det är vanligt att man vill tömma fältet efter användning.

```
def clear():
      entry.delete(0, tk.END)

button = tk.Button(window, text="Rensa", command=clear)
      button.pack()
```

Kontrollera input (enkel validering)

Man kan undvika att programmet kraschar.

```
def calculate():
      try:
            value = int(entry.get())
            label.config(text=f"{value * 2}")
      except:
            label.config(text="Skriv ett tal")
```

Placering med grid istället för pack

`pack()` staplar saker.
`grid()` ger mer kontroll.

```
entry = tk.Entry(window)
entry.grid(row=0, column=0)

button = tk.Button(window, text="OK")
button.grid(row=0, column=1)
```

Sammanfattning

Nu kan du

skapa flera knappar
hantera input och output
ändra utseende
hantera fel
placera element mer exakt

Detta räcker för att skapa riktiga små applikationer.

Övningar

1
Skapa ett program med två knappar
en som dubblar ett tal
en som tredubblar ett tal

2
Skapa ett program som
tar emot ett tal
visar resultatet gånger 5
hanterar felaktig input

12.6 Kompletta GUI-program med widgets

I detta kapitel ska vi använda Tkinter för att skapa **mer kompletta program**.
Nu kombinerar vi flera delar:

Målet är att bygga små appar som faktiskt känns användbara.

För att skapa mer avancerade program behöver vi fler delar än bara `Entry`, `Button` och `Label`.

Här är några vanliga komponenter som används i riktiga program.

Radioknappar (Radiobutton)

Radioknappar används när användaren ska välja **ett alternativ av flera**.

```
choice = tk.IntVar()
choice.set(1)

radio1 = tk.Radiobutton(window, text="Alternativ 1", variable=choice,
value=1)
radio1.pack()

radio2 = tk.Radiobutton(window, text="Alternativ 2", variable=choice,
value=2)
radio2.pack()
```

Man hämtar värdet med

```
choice.get()
```

Dropdown (OptionMenu)

Används när användaren ska välja från en lista.

```
options = ["Röd", "Blå", "Grön"]
```

```
selected = tk.StringVar()
selected.set(options[0])

dropdown = tk.OptionMenu(window, selected, *options)
dropdown.pack()
```

Hämta värdet:

```
selected.get()
```

Kryssruta (Checkbutton)

Används när användaren kan välja **ja/nej**.

```
checked = tk.BooleanVar()

checkbox = tk.Checkbutton(window, text="Jag godkänner",
variable=checked)
checkbox.pack()
```

Hämta värde:

```
checked.get()
```

Textfält (flera rader)

För längre text.

```
text = tk.Text(window, height=5, width=30)
text.pack()
```

Hämta text:

```
text.get("1.0", tk.END)
```

Sammanfattning

Nu har du fler verktyg för att bygga riktiga GUI-program:

radioknappar → välja ett alternativ
dropdown → välja från lista
kryssruta → ja eller nej
textfält → längre input

Exempel 1 Filmapp

Programmet låter användaren skriva in en film och hur lång den är.
Sedan visas en sammanfattning i fönstret.

```
import tkinter as tk

def add_movie():
      name = entry_name.get()
      try:
            length = int(entry_length.get())
            result_label.config(text=f"Filmen {name} är {length} minuter
lång")
```
OBS samma rad som ovan
```
      except:
            result_label.config(text="Skriv en giltig längd")

window = tk.Tk()
window.title("Filmlista")

label_name = tk.Label(window, text="Filmens namn:")
label_name.pack()

entry_name = tk.Entry(window)
entry_name.pack()

label_length = tk.Label(window, text="Längd i minuter:")
label_length.pack()

entry_length = tk.Entry(window)
entry_length.pack()

button = tk.Button(window, text="Lägg till film", command=add_movie)
button.pack()

result_label = tk.Label(window, text="")
result_label.pack()
```

```
window.mainloop()
```

Detta är ett komplett program där användaren interagerar och får direkt feedback.

Exempel 2 Temperaturomvandling

```
import tkinter as tk

def convert():
      try:
            temp = float(entry.get())
            if choice.get() == 1:
                  result = (temp * 9/5) + 32
                  result_label.config(text=f"{temp}°C = {result:.1f}°F")
```

OBS samma rad som ovan

```
            else:
                  result = (temp - 32) * 5/9
                  result_label.config(text=f"{temp}°F = {result:.1f}°C")
```

OBS samma rad som ovan

```
      except:
            result_label.config(text="Skriv ett giltigt tal")

window = tk.Tk()
window.title("Temperaturomvandlare")

label = tk.Label(window, text="Ange temperatur:")
label.pack()

entry = tk.Entry(window)
entry.pack()

choice = tk.IntVar()
choice.set(1)

radio1 = tk.Radiobutton(window, text="Celsius till Fahrenheit",
variable=choice, value=1)
radio1.pack()
```

```
radio2 = tk.Radiobutton(window, text="Fahrenheit till Celsius",
variable=choice, value=2)
radio2.pack()

button = tk.Button(window, text="Omvandla", command=convert)
button.pack()

result_label = tk.Label(window, text="")
result_label.pack()

window.mainloop()
```

Exempel 3 Enkel budgetapp

Användaren skriver in en kostnad och får en enkel summering.

```
import tkinter as tk

total = 0

def add_cost():
     global total
     try:
          cost = int(entry.get())
          total = total + cost
          result_label.config(text=f"Totalt: {total} kr")
     except:
          result_label.config(text="Fel inmatning")

window = tk.Tk()
window.title("Budget")

label = tk.Label(window, text="Ange kostnad i kronor:")
label.pack()

entry = tk.Entry(window)
entry.pack()

button = tk.Button(window, text="Lägg till kostnad", command=add_cost)
button.pack()
```

```
result_label = tk.Label(window, text="Totalt: 0 kr")
result_label.pack()

window.mainloop()
```

Vad är viktigt att förstå

Etiketter används för att visa resultat
Entry används för input
Funktioner kopplas till knappar
Programmet reagerar på användaren

Detta är grunden för riktiga appar.

12.7 Reflektionsfrågor

1 - Vad är ett användargränssnitt och varför är det viktigt?

2 - Vad är skillnaden mellan ett textbaserat program och ett grafiskt program?

3 - Varför är det viktigt att använda etiketter (Label) i GUI-program?

4 - När är det lämpligt att använda radioknappar?

5 - När passar det bättre att använda en dropdown istället för radioknappar?

6 - Vad används en kryssruta (Checkbutton) till?

7 - Hur skiljer sig Entry och Text från varandra?

8 - Vad händer om man inte hanterar felaktig input i ett GUI-program?

9 - Hur kan man göra ett GUI-program mer användarvänligt?

10 - Vilka delar behövs för att skapa ett komplett GUI-program i Tkinter?

11 - Vad var svårast med att arbeta med Tkinter och varför?

12 - Hur skulle du kunna förbättra ett GUI-program du har gjort?

Kapitel 13 Kompletta GUI appar/program

I detta kapitel ska vi använda Tkinter för att skapa mer kompletta program.

Nu kombinerar vi:

- input från användaren
- val (olika alternativ)
- struktur (hur saker placeras)
- visning av resultat

Målet är att bygga små appar som faktiskt känns användbara.

Widgets du behöver kunna

Tk (huvudfönster)

Startar programmet.

```
window = tk.Tk()
window.mainloop()
```

Label

Visar text.

```
tk.Label(window, text="Hej").pack()
```

Entry

Inmatningsfält.

```
entry = tk.Entry(window)
entry.pack()
```

```
entry.get()
```

Button

Startar en funktion.

```
tk.Button(window, text="Klicka", command=func).pack()
```

Frame

Grupperar widgets.

```
frame = tk.Frame(window)
frame.pack()
```

Input & val

Checkbutton (ja/nej)

```
checked = tk.BooleanVar()
tk.Checkbutton(window, text="OK", variable=checked).pack()
```

Radiobutton (välja ett alternativ)

```
choice = tk.IntVar()
tk.Radiobutton(window, text="A", variable=choice, value=1).pack()
tk.Radiobutton(window, text="B", variable=choice, value=2).pack()
```

Text (flera rader)

```
text = tk.Text(window, height=5)
text.pack()
text.get("1.0", tk.END)
```

Spinbox (välja tal)

```
spin = tk.Spinbox(window, from_=0, to=10)
spin.pack()
```

Listor & dropdown

Listbox

```
listbox = tk.Listbox(window)
listbox.pack()
listbox.insert(tk.END, "Item")
```

ttk.Combobox (dropdown)

```
from tkinter import ttk

combo = ttk.Combobox(window, values=["A", "B"])
combo.pack()
```

Layout & struktur

LabelFrame

```
frame = tk.LabelFrame(window, text="Inställningar")
frame.pack()
```

Scrollbar

```
scroll = tk.Scrollbar(window)
scroll.pack(side=tk.RIGHT, fill=tk.Y)
```

Extra

Toplevel (nytt fönster)

```
def open_window():
      new = tk.Toplevel(window)
      tk.Label(new, text="Nytt fönster").pack()
```

Menu

```
menu = tk.Menu(window)
window.config(menu=menu)
```

ttk.Notebook (flikar)

```
notebook = ttk.Notebook(window)
```

ttk.Treeview (tabell)

```
tree = ttk.Treeview(window)
```

Exempel 1 – Filmapp

```
import tkinter as tk
from tkinter import ttk

filmer = []

def add_movie():
   name = entry_name.get().strip()
   length = entry_length.get().strip()
   genre = combo.get().strip()

   try:
       length = int(length)

       if name == "":
           result_label.config(text="Du måste skriva filmens namn")
           return

       filmer.append(length)
       listbox.insert(tk.END, f"{name} ({length} min) - {genre}")

       total_min = sum(filmer)
       timmar = total_min // 60
       minuter = total_min % 60
       medel = total_min / len(filmer)
```

```
        summary_label.config(
            text=f"Total tid: {total_min} min ({timmar} h {minuter}
min)\nMedellängd: {medel:.1f} min"
        )

        result_label.config(text="Film tillagd")

        entry_name.delete(0, tk.END)
        entry_length.delete(0, tk.END)

    except ValueError:
        result_label.config(text="Fel inmatning - ange längd i
siffror")

window = tk.Tk()
window.title("Filmlista")

tk.Label(window, text="Film").pack()
entry_name = tk.Entry(window)
entry_name.pack()

tk.Label(window, text="Längd i minuter").pack()
entry_length = tk.Entry(window)
entry_length.pack()

combo = ttk.Combobox(window, values=["Action", "Drama", "Komedi"])
combo.pack()
combo.set("Action")

tk.Button(window, text="Lägg till", command=add_movie).pack()

listbox = tk.Listbox(window, width=40)
listbox.pack()

result_label = tk.Label(window, text="")
result_label.pack()
```

```
summary_label = tk.Label(window, text="Total tid: 0 min (0 h 0
min)\nMedellängd: 0 min")
summary_label.pack()

window.mainloop()
```

Exempel 2 – Temperaturomvandling

```
import tkinter as tk

def convert():
     try:
          temp = float(entry.get())
          if choice.get() == 1:
          result = (temp * 9/5) + 32
          result_label.config(text=f"{temp}°C = {result:.1f}°F")
     else:
          result = (temp - 32) * 5/9
          result_label.config(text=f"{temp}°F = {result:.1f}°C")
     Except:
          result_label.config(text="Skriv ett giltigt tal")

window = tk.Tk()
window.title("Temperaturomvandlare")

tk.Label(window, text="Ange temperatur:").pack()
entry = tk.Entry(window)
entry.pack()

choice = tk.IntVar()
choice.set(1)

tk.Radiobutton(window, text="Celsius till Fahrenheit",
variable=choice, value=1).pack()
tk.Radiobutton(window, text="Fahrenheit till Celsius",
variable=choice, value=2).pack()

tk.Button(window, text="Omvandla", command=convert).pack()
```

```
result_label = tk.Label(window, text="")
result_label.pack()

window.mainloop()
```

Exempel 3 – Budgetapp

```
import tkinter as tk

total = 0

def add_cost():
     global total
     try:
          cost = int(entry.get())
          total += cost
     result_label.config(text=f"Totalt: {total} kr")
     entry.delete(0, tk.END)
     except:
          result_label.config(text="Fel inmatning")

window = tk.Tk()

tk.Label(window, text="Kostnad").pack()
entry = tk.Entry(window)
entry.pack()

tk.Button(window, text="Lägg till", command=add_cost).pack()

result_label = tk.Label(window, text="Totalt: 0 kr")
result_label.pack()

window.mainloop()
```

Exempel 4 – Budgetapp

```
import tkinter as tk
from tkinter import ttk
```

```
def save_note():
   content = text.get("1.0", tk.END).strip()
   category = combo.get()

   if agree.get():
       tree.insert("", tk.END, values=(category, content[:20]))
       text.delete("1.0", tk.END)
       result_label.config(text="Anteckning sparad")
   else:
       result_label.config(text="Du måste godkänna")

def open_window():
   new = tk.Toplevel(window)
   tk.Label(new, text="Hjälpfönster").pack()

window = tk.Tk()
window.title("Anteckningsapp")

# MENU
menu = tk.Menu(window)
window.config(menu=menu)

file_menu = tk.Menu(menu)
menu.add_cascade(label="Arkiv", menu=file_menu)
file_menu.add_command(label="Nytt fönster", command=open_window)
file_menu.add_command(label="Avsluta", command=window.quit)

# NOTEBOOK
notebook = ttk.Notebook(window)
tab1 = tk.Frame(notebook)
tab2 = tk.Frame(notebook)

notebook.add(tab1, text="Skriv")
notebook.add(tab2, text="Lista")
notebook.pack()

# TAB 1
```

```
frame = tk.LabelFrame(tab1, text="Ny anteckning")
frame.pack(padx=10, pady=10)

scroll = tk.Scrollbar(frame)
scroll.pack(side=tk.RIGHT, fill=tk.Y)

text = tk.Text(frame, height=5, yscrollcommand=scroll.set)
text.pack()

scroll.config(command=text.yview)

combo = ttk.Combobox(tab1, values=["Skola", "Jobb", "Privat"])
combo.pack()

agree = tk.BooleanVar()
tk.Checkbutton(tab1, text="Spara anteckning", variable=agree).pack()

tk.Button(tab1, text="Spara", command=save_note).pack()

result_label = tk.Label(tab1)
result_label.pack()

# TAB 2 (TREEVIEW)
tree = ttk.Treeview(tab2, columns=("Kategori", "Text"),
show="headings")
tree.heading("Kategori", text="Kategori")
tree.heading("Text", text="Text")

tree.pack()

window.mainloop()
```

Övning – Bygg ett filmregister

Du ska bygga ett program där användaren kan:

- skriva in en film
- ange längd
- välja genre
- markera favorit
- spara filmen
- se alla filmer i en lista (tabell)

13 - Programmering, användare och framtid

13.1 Vanliga programspråk

Det finns många olika programspråk, och alla har utvecklats för att lösa olika typer av problem. Alla språk används för att ge instruktioner till datorer, men de är inte byggda på exakt samma sätt och passar inte lika bra till alla uppgifter. Därför väljer man ofta programspråk utifrån vad man vill skapa.

Vissa språk är gjorda för att vara enkla att läsa och skriva. De passar därför bra i undervisning och för den som är ny inom programmering. Andra språk används när man vill bygga stora system, arbeta nära datorns hårdvara eller skapa snabba program där prestanda är extra viktigt. Det finns också språk som är vanliga inom webbutveckling, apputveckling, spelutveckling och tekniska system.

I den här boken använder vi Python. Python är ett mycket vanligt språk och används både i undervisning och i verkliga projekt. En stor fördel med Python är att språket har en tydlig och lättläst syntax. Det gör att du snabbare kan fokusera på själva programmeringen och problemlösningen, i stället för att fastna i onödigt komplicerad kod. Python används bland annat för automation, dataanalys, AI, skript och enklare program med grafiskt gränssnitt.

Ett annat vanligt språk är JavaScript. Det används framför allt inom webbutveckling och körs ofta direkt i webbläsaren. Med JavaScript kan man göra webbsidor mer interaktiva, till exempel genom att reagera på knapptryckningar, ändra innehåll på sidan eller hämta information från internet. Medan Python ofta används i fristående program och scripts, används JavaScript ofta för sådant som händer direkt på webben.

C# är ett språk som ofta används när man vill bygga större program med tydlig struktur. Det är vanligt inom spelutveckling, särskilt i Unity, men används också till desktopprogram, appar och webbutveckling. För den som senare vill arbeta mer med spel eller större system är C# därför ett viktigt språk att känna till.

Java är också ett mycket vanligt språk. Det har använts länge och förekommer fortfarande i många stora system. Java har varit vanligt inom företag, undervisning och apputveckling. En styrka med Java är att det är byggt för att fungera på många olika plattformar, vilket har gjort språket användbart i många sammanhang.

C och C++ är språk som ofta används när man vill ha hög kontroll över datorns resurser. De används till exempel i operativsystem, spelmotorer, tekniska program och inbyggda system. Dessa språk kan ofta ge mycket hög prestanda, men de är också mer krävande att arbeta med än till exempel Python. Därför passar de inte alltid lika bra som första språk, även om de är mycket viktiga inom många tekniska områden.

Det finns alltså inget programspråk som är bäst till allt. I stället kan man tänka att programspråk är olika verktyg. Om du ska bygga en webbsida med interaktivitet är ett språk lämpligt. Om du ska skapa ett enkelt nybörjarprogram kan ett annat vara bättre. Om du ska styra tekniska system eller bygga något som måste vara mycket snabbt kan ett tredje språk passa bäst. Valet av språk handlar därför ofta om syfte, användningsområde och krav.

Programspråk förändras också över tid. Vissa språk har funnits länge men används fortfarande, medan andra är nyare och har skapats för att möta nya behov. Utvecklingen påverkas bland annat av nya typer av program, nya enheter, ökade krav på säkerhet och önskemål om att koden ska bli tydligare och lättare att underhålla. Därför är programmering inte något helt stillastående område, utan något som hela tiden utvecklas.

I den här kursen använder vi Python eftersom det passar bra för att lära sig grunderna i programmering. När du lär dig Python tränar du på sådant som också finns i många andra språk, till exempel variabler, villkor, loopar, funktioner, inmatning, utskrift och felhantering. Det betyder att kunskaper i Python också ger en bra grund för att senare förstå andra programspråk.

13.2 Olika sätt att bygga program

Program kan byggas på olika sätt. Det betyder inte att ett sätt alltid är rätt och alla andra fel. I stället handlar det om att olika sätt att bygga program passar olika bra beroende på hur stort programmet är, vad det ska göra och hur tydlig strukturen behöver vara.

När man talar om olika sätt att bygga program använder man ibland ordet programmeringsparadigm. Det är ett ord för olika arbetssätt eller olika sätt att tänka när man programmerar. På den här nivån behöver du inte kunna avancerad teori, men det är bra att förstå att samma problem kan lösas på olika sätt.

Ett vanligt sätt är att skriva program steg för steg i den ordning de ska köras. Det kallas ofta procedurell programmering. Då bygger man programmet som en serie instruktioner där datorn först gör en sak, sedan nästa och sedan nästa. Man kan använda variabler, villkor, loopar och funktioner för att strukturera programmet, men grunden är fortfarande att programmet följer en tydlig ordning.

Ett enkelt exempel på procedurell programmering är ett program som frågar efter ett namn, sparar svaret och sedan skriver ut en hälsning. Programmet börjar överst och arbetar sig nedåt rad för rad. Många av de program du har skrivit hittills i boken fungerar på det sättet. Det gör procedurell programmering till ett bra sätt att börja, eftersom strukturen ofta blir tydlig och lätt att följa.

Ett annat sätt är att dela upp programmet i funktioner. Det är fortfarande nära procedurell programmering, men koden blir bättre organiserad. I stället för att skriva allt i en lång följd kan man skapa funktioner som ansvarar för olika delar, till exempel inmatning, beräkning och utskrift. Då blir programmet lättare att läsa, testa och förbättra.

När program blir större kan man också bygga dem med objektorientering. Då tänker man mer i form av objekt, alltså saker i programmet som har data och funktioner som hör ihop. Om man till exempel skapar ett filmprogram skulle ett objekt kunna vara en film. Filmen kan ha egenskaper som namn och längd, och den kan också ha funktioner som hör till just den filmen.

Objektorientering behöver inte vara komplicerat, men det är bra att känna till att det finns som ett vanligt sätt att bygga program. Många större program och många vanliga språk använder objektorienterade idéer. I vissa språk, till exempel C# och Java, är detta extra vanligt. I Python går det också att arbeta objektorienterat, men i den här kursen ligger fokus främst på enklare och tydligare programstruktur.

Det finns också program som är händelsestyrda. Det betyder att programmet väntar på att något ska hända, till exempel att en användare klickar på en knapp. Detta arbetssätt är vanligt i grafiska gränssnitt. När du arbetar med Tkinter märker du att programmet inte bara kör allt uppifrån och ned och sedan avslutas. I stället ligger programmet och väntar på händelser från användaren, till exempel knapptryckningar eller inmatning i ett fält.

Det betyder att du redan har mött mer än ett sätt att bygga program. När du skriver ett vanligt Pythonprogram med inmatning och utskrift arbetar du ofta procedurellt. När du delar upp koden i funktioner får du bättre struktur. När du arbetar med Tkinter använder du också händelsestyrd programmering. Senare kan man också lära sig mer om objektorientering.

Det viktiga i Programmering 1 är inte att kunna alla dessa arbetssätt på djupet. Det viktiga är att förstå att program kan byggas på olika sätt och att valet av struktur påverkar hur lätt programmet blir att förstå, använda och vidareutveckla.

Ett litet program kan fungera bra med en enkel struktur. Ett större program behöver ofta delas upp tydligare. Därför är det viktigt att redan tidigt träna på att skriva kod som är läsbar, logisk och lätt att följa.

13.3 Anpassning till användare och tillgänglighet

När man programmerar räcker det inte att programmet fungerar rent tekniskt. Ett program kan ge rätt resultat och ändå vara svårt att använda. Därför är det viktigt att tänka på användaren. Den som använder programmet ska förstå vad som ska göras, kunna mata in rätt information och lätt kunna tolka resultatet som visas. Ett program som är tydligt och lätt att använda blir ofta mycket bättre än ett program som bara fungerar i teorin.

Att anpassa ett program till användaren betyder att man funderar på vem programmet är till för och hur det ska användas. En van användare kan ibland förstå ett mer avancerat program, men en nybörjare behöver ofta tydligare vägledning. I Programmering 1 handlar detta ofta om enkla saker som gör stor skillnad. Det kan vara tydliga texter, bra etiketter, enkla knappar och hjälpsamma felmeddelanden. Det handlar alltså inte bara om design, utan också om att göra programmet begripligt.

Ett vanligt problem i nybörjarprogram är att instruktionerna är för otydliga. Om ett program bara visar texten “Skriv här” eller “Mata in värde” kan användaren bli osäker på vad som egentligen ska skrivas in. Det är mycket bättre att vara konkret. Om användaren ska skriva filmens längd är det tydligare att skriva “Skriv filmens längd i minuter”. Då vet användaren både vad som ska matas in och i vilken form. Ju tydligare instruktioner programmet har, desto mindre risk är det att användaren gör fel.

Felmeddelanden är också en viktig del av anpassningen till användaren. Om något går fel ska programmet inte bara visa att det blev fel, utan också hjälpa användaren vidare. Ett felmeddelande som bara säger “Fel” hjälper nästan inte alls. Ett bättre meddelande kan vara “Fel: skriv ett heltal för längden”. Då förstår användaren både vad som blev fel och hur problemet kan lösas. Det gör programmet mer användarvänligt och minskar frustrationen.

Läsbarhet är en annan viktig del. Texten i programmet ska vara lätt att läsa och lätt att förstå. I grafiska program är det bra om etiketter står nära rätt inmatningsfält och om resultatet visas tydligt. Om texten är rörig eller placerad på ett ologiskt sätt blir programmet svårare att använda. Ett enkelt och tydligt gränssnitt är därför ofta bättre än ett gränssnitt som försöker göra för mycket på en gång.

Det är också viktigt att programmet kontrollerar inmatning på ett rimligt sätt. Detta kallas validering. Om ett program frågar efter ett tal är det bra att kontrollera att användaren verkligen skriver ett tal. Om programmet frågar efter ålder eller längd är det också rimligt att kontrollera att värdet inte är negativt eller orimligt. Validering gör att programmet fungerar bättre och att användaren snabbare förstår vad som förväntas. Det minskar också risken för krascher och konstiga resultat.

I grafiska program spelar även ordningen stor roll. Användaren ska enkelt kunna förstå vad som ska göras först, vad som kommer sedan och hur resultatet visas. Om ett Tkinterprogram till exempel har en etikett för filmnamn, en etikett för längd, två inmatningsfält och en knapp för att beräkna resultatet, blir arbetsflödet tydligt. Användaren skriver först in namn, sedan längd och klickar därefter på knappen. När ordningen är logisk känns programmet mer naturligt att använda.

Man kan också tänka på tillgänglighet. Tillgänglighet betyder att så många som möjligt ska kunna använda programmet. I den här kursen behöver det inte handla om avancerade regler, men det är bra att förstå grundtanken. Texten ska vara tydlig, instruktionerna enkla och programmet ska inte kräva onödigt krångliga steg. Färg och kontrast kan också spela roll. Om text syns dåligt mot bakgrunden blir programmet svårare att använda. Därför är det bra att välja tydliga färger och en enkel layout när man bygger grafiska gränssnitt.

Användarvänlighet handlar ofta om små detaljer som tillsammans gör stor skillnad. Ett program blir mer användarvänligt om det har tydliga namn, enkel struktur, bra instruktioner och felmeddelanden som hjälper. Det behöver inte vara avancerat för att vara bra. Tvärtom är det ofta de enklaste lösningarna som fungerar bäst för användaren.

Om man ser tillbaka på de Tkinterprogram som du har arbetat med i boken märks detta tydligt. När varje inmatningsfält har en Label före, när knappen har en tydlig text som “Beräkna”, när resultatet visas på flera rader och när felmeddelanden förklarar vad användaren ska göra, då blir programmet mycket bättre. Samma program hade varit betydligt svårare att använda om det saknade etiketter, hade otydliga knappar eller bara visade korta felmeddelanden utan förklaring.

Att tänka på användare och tillgänglighet är därför en viktig del av programmering. Det räcker inte att skriva kod som fungerar. Man behöver också skriva program som passar sitt syfte och som går att använda på ett tydligt och rimligt sätt. Detta är viktigt både i små övningsprogram och i större projekt.

13.4 Datorsimulering och programmerade system

Programmering används inte bara för att skriva små övningsprogram eller bygga enkla appar. Den används också för att simulera verkligheten och för att styra system som påverkar maskiner, utrustning och beslut. Det gör att programmering får stor betydelse både i vardagen, i arbetslivet och i samhället.

En datorsimulering är en modell av något verkligt som körs i ett program. I stället för att testa något direkt i verkligheten kan man låta datorn räkna ut vad som skulle kunna hända. Det kan handla om att simulera trafik, väder, ekonomi, spridning av sjukdomar eller rörelser i ett spel. Simuleringen bygger på regler som programmeraren har skrivit in. Datorn följer sedan dessa regler och visar ett resultat.

Fördelen med simuleringar är att man kan prova många olika situationer snabbt och säkert. Om man till exempel vill undersöka hur bilar rör sig i en korsning kan det vara enklare att först göra en simulering än att bygga om en riktig väg. På samma sätt kan man i spelutveckling simulera rörelse, kollisioner och poängsystem innan ett spel är helt färdigt. Programmering blir då ett verktyg för att testa idéer, undersöka problem och förutse resultat.

Samtidigt är en simulering aldrig exakt samma sak som verkligheten. Den bygger alltid på antaganden och förenklingar. Om reglerna i programmet är fel eller för enkla kan simuleringen ge ett missvisande resultat. Därför är det viktigt att förstå att en simulering är en modell av verkligheten, inte verkligheten själv. Den kan vara mycket användbar, men den måste tolkas med eftertanke.

Programmerade system används också för att styra maskiner och teknisk utrustning. Inom industrin kan program styra robotar, produktionslinjer och sensorer. I hemmet finns programmerade system i till exempel tvättmaskiner, mikrovågsugnar och värmesystem. I bilar används programmering för att hantera bromssystem, motorstyrning, varningar och olika hjälpfunktioner. I vården används programmerade system i medicinsk utrustning, journalsystem och bokningssystem.

När programmerade system styr maskiner blir det extra viktigt att programmen fungerar korrekt. Ett litet fel i ett vanligt textprogram kanske bara ger ett felaktigt resultat på skärmen. Men ett fel i ett system som styr en maskin kan få mycket större konsekvenser. Därför behöver sådana system testas noggrant och byggas på ett genomtänkt sätt. Programmering handlar alltså inte bara om att få kod att köra, utan också om att skapa lösningar som är säkra och pålitliga.

Programmerade system påverkar inte bara maskiner utan också beslutsfattande. Många digitala system används för att sortera information, föreslå alternativ eller stödja människor i olika beslut. Ett system kan till exempel hjälpa till att rekommendera filmer, sortera sökresultat, föreslå vägar i en GPS eller analysera data i en verksamhet. I sådana fall är det fortfarande människor som ofta har det yttersta ansvaret, men programmet påverkar ändå vilket underlag som visas och vilka val som verkar mest rimliga.

Det betyder att programmering kan få stor påverkan även när ingen fysisk maskin styrs direkt. Om ett system prioriterar viss information framför annan kan det påverka vad människor ser, väljer och tror. Därför är det viktigt att förstå att programmerade system inte är neutrala bara för att de är digitala. De bygger på regler, val och prioriteringar som människor har skapat.

I den här kursen kanske du inte bygger avancerade simuleringssystem eller styrsystem, men principen är ändå densamma som i enklare program. Du skriver instruktioner, programmet följer dem och resultatet påverkar det som händer. Ett litet Pythonprogram som räknar ut kostnader, visar rätt svar eller reagerar på användarens val bygger på samma grundidé som större tekniska system. Skillnaden är främst hur omfattande och viktiga systemen är.

Därför är det värdefullt att redan i Programmering 1 förstå att kod kan användas till mycket mer än bara utskrifter och enkla beräkningar. Programmering kan användas för att modellera verkligheten, styra utrustning och påverka beslut. Det gör ämnet större och mer betydelsefullt än vad det först kan verka.

13.5 Programmering i förändring

Programmering är inte något som står stilla. Språk, verktyg och arbetssätt förändras hela tiden. Nya tekniska behov leder till nya lösningar, och därför utvecklas också programmeringen. Det betyder att den som lär sig programmera inte bara lär sig ett visst språk, utan också får en grund för att förstå och anpassa sig till framtida förändringar.

Tidigare användes datorer främst till beräkningar och enklare administrativa uppgifter. I dag används programmering inom nästan alla delar av samhället. Program finns i mobiltelefoner, spel, webbplatser, bilar, sjukhus, fabriker och smarta hem. När användningen förändras behöver också programmeringsspråk och utvecklingsmiljöer förändras. Det gör att vissa språk blir vanligare inom vissa områden, medan andra språk används mindre än tidigare eller får nya roller.

En viktig förändring är att programmering har blivit mer tillgängligt. Förr arbetade många direkt närmare datorns tekniska nivå, medan man i dag oftare använder språk och verktyg som gör utvecklingen snabbare och lättare att förstå. Python är ett tydligt exempel på detta. Språket har blivit mycket populärt eftersom det har en relativt tydlig syntax och kan användas till många olika saker, till exempel utbildning, webb, dataanalys och AI. Det betyder inte att äldre eller mer tekniskt avancerade språk har försvunnit, utan att olika språk används där de passar bäst.

Programmering förändras också genom att nya arbetssätt blir vanligare. I dag är det till exempel vanligt att arbeta steg för steg, testa ofta och förbättra program successivt. Versionshantering, samarbete via plattformar som GitHub och användning av färdiga bibliotek har blivit en naturlig del av hur många programmerare arbetar. Det innebär att programmering inte bara handlar om att själv skriva varje rad från början, utan också om att förstå, anpassa och vidareutveckla befintliga lösningar.

En annan tydlig förändring är att programmering i allt högre grad kopplas till data, automatisering och artificiell intelligens. Program används inte bara för att följa fasta instruktioner, utan också för att analysera stora mängder information och ge förslag eller förutsägelser. Det gör att programmering får nya användningsområden och att kraven på programmerare förändras. Det räcker ofta inte bara att kunna skriva kod. Man behöver också kunna testa, granska, förstå användare och tänka på säkerhet, ansvar och kvalitet.

Samtidigt finns det sådant som inte förändras lika mycket. Grundläggande idéer som variabler, villkor, loopar, funktioner och problemlösning är fortfarande centrala. Det betyder att de kunskaper du lär dig i början av programmering är viktiga även om tekniken utvecklas. Om du förstår grunderna blir det lättare att lära dig nya språk, nya verktyg och nya arbetssätt senare.

Det är också svårt att veta exakt hur framtiden kommer att se ut. Vissa språk och verktyg som är vanliga i dag kanske blir mindre vanliga i framtiden, medan nya lösningar tar plats. Historiskt har programmeringen hela tiden förändrats när nya behov har uppstått. Därför är det klokt att se programmering som något levande. Man lär sig inte bara ett ämne med färdiga svar, utan ett område som fortsätter att utvecklas.

För den som studerar programmering betyder detta att det är viktigt att vara nyfiken och beredd att fortsätta lära sig. Målet är inte bara att kunna använda ett visst språk i en viss situation, utan också att förstå principerna bakom programmering. Då blir det lättare att följa med när tekniken förändras.

13.6 Refektionsfrågor

1. Varför finns det flera olika programspråk i stället för ett enda språk som används till allt?
2. Varför kan Python vara ett bra språk för nybörjare, och i vilka situationer kan andra språk vara bättre?
3. Vad är skillnaden mellan att bygga program på ett procedurellt sätt och att bygga dem med objektorienterade idéer?
4. Varför är det viktigt att anpassa program till användaren och inte bara fokusera på att programmet fungerar tekniskt?
5. Ge exempel på hur ett program kan bli svårt att använda om man inte tänker på tillgänglighet.
6. På vilket sätt kan tydliga instruktioner, felmeddelanden och enkel navigation göra ett program bättre?

7. Vad är en datorsimulering, och varför kan simuleringar vara användbara i teknik och samhälle?
8. Ge exempel på ett programmerat system som påverkar maskiner, utrustning eller beslut i vardagen.
9. Hur kan programmering påverka samhället på ett positivt sätt?
10. Hur kan programmering också skapa problem eller risker om system utformas på ett dåligt sätt?
11. Varför förändras programmering över tid?
12. Varför är det viktigt att lära sig grunderna i programmering även om språk och verktyg förändras?
13. Hur tror du att framtidens programmering kommer att skilja sig från dagens?
14. Vilka kunskaper tror du blir särskilt viktiga i framtiden för den som arbetar med programmering?

14 - Projekt

14.1 Planera ett program

Innan du börjar koda måste du planera ditt program.

Många nybörjare hoppar direkt till kod – men då fastnar man ofta.

Planering hjälper dig att

förstå problemet
strukturera lösningen
undvika onödiga fel

Vad innebär planering

Att planera ett program betyder att du bestämmer

vad programmet ska göra
vem som ska använda det
hur det ska fungera

Exempel

Du vill skapa en filmapp.

Då kan planeringen vara:

Programmet ska
 låta användaren skriva in filmer
 spara längd på filmer
 visa information om filmer

En enkel plan

Du kan skriva din plan så här:

Program: Filmlista

Programmet ska
 ta emot filmens namn
 ta emot filmens längd
 visa information om filmen

Viktiga frågor att ställa

Vad är syftet med programmet
 Vad ska användaren kunna göra
 Vilken information behövs
 Hur ska resultatet visas

Tips

Börja enkelt
 Gör inte för stort program direkt
 Bygg vidare steg för steg

Koppling till kursen

När du planerar tränar du

problemlösning
 struktur
 förståelse för hur program byggs

Detta är en viktig del av programmering och används i verkliga projekt.

Övning

Skriv en plan för ett eget program.

Det kan till exempel vara

en budgetapp
en träningsapp
en filmlista
ett enkelt spel

Skriv

vad programmet ska göra
vad användaren ska kunna göra
vilken information som behövs

Planen ska vara tydlig och enkel att förstå.

14.2 Kravspecifikation (VAD)

Efter att du har planerat ditt program är nästa steg att skriva en kravspecifikation.

En kravspecifikation beskriver exakt vad programmet ska göra.

Den gör det tydligt vad som ska byggas innan du börjar koda.

Varför behövs en kravspecifikation

En kravspecifikation hjälper dig att

ha tydliga mål
undvika missförstånd
bygga rätt funktioner från början

I större projekt är detta extra viktigt.

Vad innehåller en kravspecifikation

En enkel kravspecifikation kan innehålla

funktionella krav
krav på användaren
krav på hur programmet ska bete sig

Funktionella krav

Funktionella krav beskriver vad programmet ska kunna göra.

Exempel:

Programmet ska
 ta emot användarens namn
 ta emot användarens ålder
 beräkna år kvar till pension
 visa resultatet

Krav på användaren

Här beskriver du vad användaren behöver göra.

Exempel:

Användaren ska
 kunna skriva in text
 kunna skriva in siffror
 förstå vad som ska matas in

Krav på beteende

Här beskriver du hur programmet ska reagera.

Exempel:

Programmet ska
 visa felmeddelande vid felaktig inmatning
 inte krascha
 visa tydliga instruktioner

Exempel

Program: Filmlista

Funktionella krav
 Programmet ska kunna lägga till en film
 Programmet ska kunna visa alla filmer
 Programmet ska kunna visa filmens längd

Användarkrav
 Användaren ska kunna skriva in filmens namn
 Användaren ska kunna skriva in filmens längd

Beteende
 Programmet ska visa ett tydligt fel om längden inte är ett tal
 Programmet ska visa filmer i en lista

Tips

Skriv tydligt och konkret
Undvik otydliga ord som “bra” och “snabb”
Tänk på användaren

Koppling till kursen

När du skriver kravspecifikation tränar du

problemlösning
struktur
att skapa program med anpassning till användare

Detta är viktigt för att nå högre betyg.

Övning

Skriv en kravspecifikation för ditt program från föregående övning.

Dela upp i

funktionella krav
användarkrav
beteende

Skriv tydligt vad programmet ska göra.

14.3 Design (HUR)

När du har en plan och en kravspecifikation är nästa steg design.

Design handlar om hur programmet ska byggas.

Du bestämmer struktur, uppdelning och hur olika delar hänger ihop.

Vad innebär design

Att designa ett program betyder att du bestämmer

vilka delar programmet ska bestå av
hur dessa delar ska samarbeta
hur koden ska organiseras

Dela upp programmet

Ett bra program delas upp i mindre delar.

Det gör koden lättare att förstå och arbeta med.

Exempel:

Istället för att skriva allt i en lång kod kan du dela upp i steg

ta emot inmatning
 bearbeta data
 visa resultat

Exempel

Program: Filmlista

Design:

Del 1 – Inmatning
 Användaren skriver in filmens namn
 Användaren skriver in filmens längd

Del 2 – Bearbetning
 Programmet sparar informationen

Del 3 – Utskrift
 Programmet visar information om filmen

Pseudokod

Ett sätt att designa är att skriva pseudokod.

Pseudokod är en beskrivning av programmet med vanliga ord.

Exempel:

Starta program
 Fråga efter filmens namn
 Fråga efter filmens längd
 Spara informationen
 Visa filmen

En enkel struktur

Så här kan strukturen se ut:

Start
 Inmatning
 Bearbetning
 Utskrift
 Slut

Tänk på detta

Håll designen enkel
 Dela upp i tydliga steg
 Skriv så att någon annan förstår

Koppling till kursen

När du arbetar med design tränar du

strukturerad problemlösning
 planering av program
 förmåga att bygga större program

Detta är viktigt för att nå högre betyg.

Övning

Skriv en design för ditt program.

Använd pseudokod.

Dela upp i steg som

inmatning
 bearbetning
 utskrift

Din design ska vara tydlig och lätt att följa.

14.4 Implementering

Nu är det dags att börja koda.

Implementering betyder att du skriver programmet utifrån din design.

Du går från plan och struktur till fungerande kod.

Vad innebär implementering

Att implementera ett program betyder att du

skriver koden
 följer din design
 testar under tiden

Arbeta steg för steg

Skriv inte hela programmet direkt.

Bygg i små delar och testa ofta.

Exempel:

Börja med att ta emot inmatning
 Testa att det fungerar
 Lägg till nästa del

Exempel

Program: Filmlista

```
def hamta_film():
      namn = input("Skriv filmens namn: ")
      return namn

def hamta_langd():
      while True:
      try:
            langd = int(input("Skriv filmens längd: "))
            return langd
      except ValueError:
      print("Fel: skriv ett heltal")

def visa_film(namn, langd):
      print(f"Film: {namn}")
      print(f"Längd: {langd} minuter")
      film = hamta_film()
```

```
    langd = hamta_langd()
    visa_film(film, langd)
```

Testa under tiden

Kör programmet ofta.

Testa olika saker:

skriv rätt värden
skriv fel värden
testa gränsfall

Vanliga fel

syntaxfel (felstavningar i koden)
logiska fel (programmet gör fel sak)
felaktig inmatning

Tips

Skriv tydliga variabelnamn
Använd funktioner
Kommentera vid behov
Testa ofta

Koppling till kursen

När du implementerar tränar du

programmering i Python
kontrollstrukturer
felhantering
struktur i kod

Detta är centralt i kursen.

Övning

Implementera ditt program utifrån din design.

Ditt program ska

ta emot inmatning
bearbeta information
visa resultat

Övning

Skapa ett program med ett grafiskt gränssnitt (GUI) med Tkinter.

Programmet ska hantera information om en film.

Programmet ska

ta emot filmens namn
ta emot filmens längd i minuter
beräkna längden i timmar
visa resultatet i fönstret

Krav

Använd etiketter (Label) före varje inmatningsfält

Använd inmatningsfält (Entry) för användarens input

Använd en knapp (Button) som startar beräkningen

Visa resultatet med en Label

Programmet ska visa ett felmeddelande om längden inte är ett heltal

Tips

Använd en funktion som körs när knappen klickas

Använd `get()` för att läsa in data från Entry

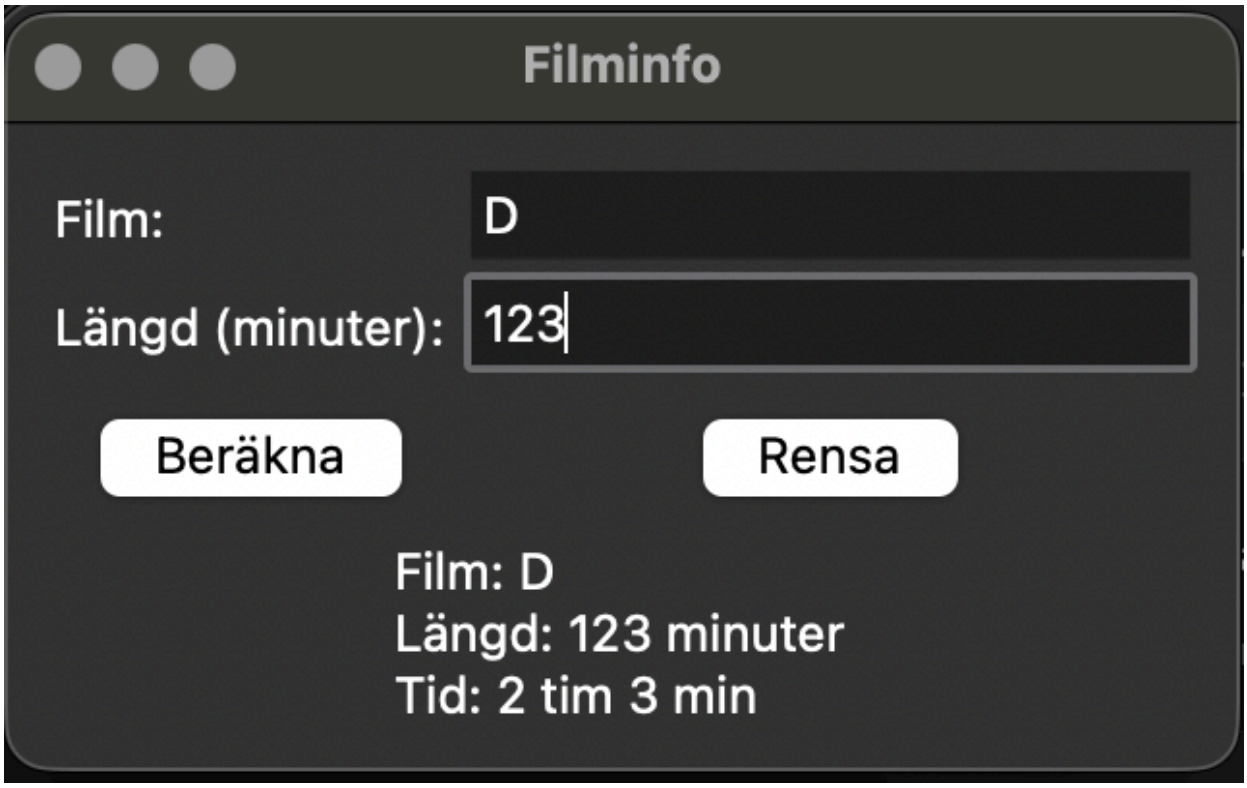
Använd `config()` för att uppdatera text i Label

Extra utmaning

Gör så att resultatet visas på flera rader

Formatera så det står tex 2 tim och 36 min om användaren matar in 123 min

Testa att skriva fel värden och se vad som händer



14.5 Testning

När programmet är färdigt behöver du testa det.

Testning betyder att du kontrollerar att programmet fungerar som det ska.

Du testar både vanliga fall och situationer där något kan bli fel.

Varför är testning viktigt

Testning hjälper dig att

hitta fel
förbättra programmet
se om kraven är uppfyllda

Även små program behöver testas.

Vad ska man testa

Du ska testa att

programmet startar
 inmatning fungerar
 beräkningar blir rätt
 resultatet visas korrekt
 fel hanteras på rätt sätt

Exempel

Du har gjort ett filmprogram.

Då kan du testa detta:

Skriv in ett filmnamn
 Skriv in ett vanligt heltal som längd
 Kontrollera att rätt resultat visas

Testa sedan felaktiga värden:

lämna filmnamnet tomt
 skriv bokstäver i längdfältet
 skriv ett negativt tal
 skriv 0

Testfall

Ett testfall är en planerad test.

Du bestämmer

vad du skriver in
 vad som borde hända
 vad som faktiskt händer

Exempel på testfall

Test 1
 Inmatning: Interstellar, 169
 Förväntat resultat: Programmet visar filmens namn och tiden 2 tim 49 min

Test 2
 Inmatning: Tomt namn, 120
 Förväntat resultat: Felmeddelande om att filmnamn saknas

Test 3
 Inmatning: Avatar, abc
 Förväntat resultat: Felmeddelande om att längden måste vara ett heltal

Test 4
Inmatning: Dune, -5
Förväntat resultat: Felmeddelande om att längden måste vara större än 0

Dokumentera testningen

Det är bra att skriva ner dina tester.

Då blir det lättare att se

vad du har testat
vilka fel du hittat
vad du behöver ändra

Tips

Testa både rätt och fel värden
Testa flera olika situationer
Ändra en sak i taget om något går fel

Koppling till kursen

När du testar tränar du

att hitta syntaxfel
att hitta logiska fel
att förebygga fel
att förbättra programkvalitet

Detta är en viktig del av programmering.

Övning

Testa ditt program med minst fyra olika testfall.

Skriv för varje testfall

vilken inmatning du använder
vad du förväntar dig
vad som faktiskt händer

Fundera också på om du behöver ändra något i programmet efter testningen.

14.6 Dokumentation

När programmet är klart behöver det dokumenteras.

Dokumentation betyder att du beskriver hur programmet fungerar.

Det gör det lättare för dig själv och andra att förstå koden.

Varför behövs dokumentation

Dokumentation hjälper dig att

förklara programmet
visa hur det fungerar
göra koden lättare att förstå senare

Det är extra viktigt om någon annan ska läsa eller använda programmet.

Vad kan dokumenteras

Du kan dokumentera

vad programmet gör
hur programmet används
hur koden är uppbyggd
vilka delar som är viktiga

Kommentarer i kod

Ett vanligt sätt att dokumentera är att skriva kommentarer i koden.

Kommentarer förklarar vad olika delar gör.

Exempel:

```
# Hämtar filmens namn från användaren

namn = entry_namn.get()
```

Du ska inte kommentera varje rad i onödan.

Skriv kommentarer där de hjälper läsaren att förstå.

Beskrivning av programmet

Det är också bra att skriva en kort text om programmet.

Exempel:

Programmet tar emot ett filmnamn och en längd i minuter.
Sedan räknar programmet om tiden till timmar och minuter.
Resultatet visas i programmets fönster.

Beskrivning av viktiga delar

Du kan också förklara viktiga delar, till exempel

vad en funktion gör
hur fel hanteras
hur resultat visas

Exempel:

Funktionen `berakna()` läser in användarens värden, kontrollerar att längden är korrekt och visar resultatet i fönstret.

Vad bra dokumentation kännetecknas av

Bra dokumentation är

tydlig
kortfattad
relevant
lätt att förstå

Tips

Skriv så att en annan elev förstår
Förklara viktiga delar
Håll dokumentationen enkel och tydlig

Koppling till kursen

När du dokumenterar tränar du

god struktur
tydlig kommunikation
förståelse för kodens uppbyggnad

Detta är en viktig del av professionell programmering.

Övning

Dokumentera ditt program.

Skriv

en kort beskrivning av vad programmet gör
kommentarer till viktiga delar i koden
en förklaring av hur användaren använder programmet

Dokumentationen ska vara tydlig och lätt att förstå.

14.7 Redovisning

När ett program är klart ska det ofta redovisas.

Redovisning betyder att du visar upp ditt arbete och förklarar hur du har tänkt.

Det handlar inte bara om att visa att programmet fungerar.

Du ska också kunna beskriva hur du planerade, byggde, testade och förbättrade programmet.

Varför redovisning är viktigt

En redovisning hjälper dig att

visa vad du har gjort
förklara hur programmet fungerar
visa hur du har löst problem
visa att du förstår din egen kod

I större projekt är detta en viktig del av arbetet.

Vad en redovisning kan innehålla

En redovisning kan innehålla

vad programmet gör
vem programmet är gjort för
vilka krav du utgick från
hur du designade programmet
hur du implementerade det
hur du testade det

vilka problem du stötte på
hur du löste dem

Beskriv projektet

Börja med att berätta vad ditt program är.

Exempel:

Jag har gjort ett filmprogram i Tkinter.
Användaren kan skriva in en films namn och dess längd i minuter.
Programmet räknar om tiden till timmar och minuter och visar resultatet i fönstret.

Beskriv din planering

Berätta hur du tänkte innan du började koda.

Exempel:

Först planerade jag vad programmet skulle göra.
Sedan skrev jag en kravspecifikation.
Efter det gjorde jag en enkel design där jag bestämde vilka delar programmet skulle ha.

Beskriv hur programmet fungerar

Förklara de viktigaste delarna.

Exempel:

Programmet har två inmatningsfält och två knappar.
En knapp används för att räkna ut resultatet.
Den andra används för att rensa fälten.
Programmet kontrollerar också om användaren skriver fel.

Beskriv problem och lösningar

I en redovisning är det bra att visa att du har löst problem.

Exempel:

Ett problem var att användaren kunde skriva bokstäver i längdfältet.
Det löste jag med `try/except`.

Ett annat problem var att användaren kunde lämna filmnamnet tomt.
Det löste jag med en kontroll som visar ett felmeddelande.

Beskriv testningen

Berätta hur du testade programmet.

Exempel:

Jag testade både korrekta och felaktiga värden.
Jag testade till exempel ett vanligt tal, ett negativt tal, text i längdfältet och tomt namn.

Visa förbättringar

Det är också bra att säga vad du skulle kunna utveckla vidare.

Exempel:

Om jag skulle fortsätta utveckla programmet skulle jag kunna lägga till en lista med flera filmer.
Jag skulle också kunna spara filmer i en fil.

Tips inför redovisning

Försök att använda tydliga ord
Visa programmet steg för steg
Förklara varför du gjort vissa val
Var beredd att beskriva hur koden fungerar

Koppling till kursen

När du redovisar tränar du

att beskriva ett projektarbete
att visa problemlösning
att förklara hur större program byggs

Detta är viktigt när du arbetar med mer omfattande programmeringsuppgifter.

Övning

Förbered en kort redovisning av ditt program.

Ta med

vad programmet gör
hur du planerade arbetet
hur programmet är uppbyggt
vilka problem du löste

hur du testade programmet
vad du skulle vilja förbättra vidare

Redovisningen ska vara tydlig och lätt att följa.

15 - Programmering i samhället

Programmering påverkar många delar av samhället.

Den används inte bara av programmerare, utan också i många yrken och system som människor använder varje dag.

Programmering finns till exempel inom

webbplatser
spel
appar
sjukvård
banker
industri
transport

Programmering och arbetsliv

Kunskaper i programmering kan vara användbara i många yrken.

En del arbetar direkt med att skapa program, till exempel webbutvecklare och spelutvecklare.

Andra använder programmering för att automatisera uppgifter, analysera data eller styra tekniska system.

När man arbetar med programmering är det viktigt att kunna

lösa problem
arbeta strukturerat
testa och förbättra
samarbeta med andra

Automatisering

Automatisering betyder att en uppgift utförs automatiskt med hjälp av teknik.

Det kan till exempel vara

ett program som räknar ut totalpris
en robot som sorterar varor
ett system som skickar mejl automatiskt

Automatisering används för att

spara tid
minska fel
göra arbetet snabbare

Men automatisering kan också skapa problem om programmet innehåller fel.

Därför måste automatiserade system testas noggrant.

Programmering i samhället

Programmering används för att lösa verkliga problem.

Den kan hjälpa företag, skolor, sjukhus och andra verksamheter att arbeta effektivare.

Samtidigt påverkar programmering hur människor lever, arbetar och använder teknik.

Det är därför viktigt att förstå att programmering inte bara handlar om kod, utan också om hur tekniken används i samhället.

Koppling till kursen

När du lär dig programmering tränar du inte bara på att skriva kod.

Du tränar också på

problemlösning
logiskt tänkande
struktur
förståelse för hur programmering används i vardagen och arbetslivet

15.1 AI och programmering

AI är ett område inom programmering där datorer tränas att lösa uppgifter som annars ofta kräver mänskligt tänkande.

Det kan till exempel handla om att känna igen bilder, föreslå texter, översätta språk eller analysera stora mängder data.

AI bygger på programmering, men skiljer sig från vanliga program på ett viktigt sätt.

Ett vanligt program följer ofta tydliga regler som en människa har skrivit.

AI kan i stället tränas på data och sedan hitta mönster själv.

AI finns redan i samhället

AI används redan i många system som människor möter varje dag.

Exempel:

sökmotorer
rekommendationer i appar
chattbottar
bildigenkänning
översättning
självkörande funktioner i fordon

Det betyder att AI och programmering redan påverkar både vardag och arbetsliv.

AI och framtidens IT-yrken

Det är troligt att AI kommer att förändra många IT-yrken.

Men det är också viktigt att säga att det är svårt att veta exakt hur framtiden blir.

Teknik utvecklas snabbt, företag arbetar på olika sätt och nya yrken kan uppstå samtidigt som andra förändras.

Flera stora rapporter pekar ändå på att AI, informationsbehandling, digitalisering och automation kommer att fortsätta förändra arbetsmarknaden de närmaste åren.

Hur IT-yrken kan förändras

En möjlig utveckling är att vissa enklare arbetsuppgifter blir mer automatiserade.

Det kan till exempel handla om att AI hjälper till att skriva kod, hitta buggar, skapa dokumentation eller testa program.

Då kan vissa arbetsuppgifter gå snabbare än tidigare.

Samtidigt betyder det inte säkert att programmerare behövs mindre.

Det kan i stället betyda att arbetet förändras.

Människor kommer fortfarande att behövas för att

planera system
förstå användarnas behov
granska resultat
testa kvalitet
ta ansvar för säkerhet och etik
lösa problem som inte är enkla eller tydliga

Det är därför rimligt att tro att framtidens IT-yrken kan innehålla mindre rutinjobb och mer arbete med analys, kvalitet, samarbete och ansvar.

Yrken som kan växa

Rapporter från World Economic Forum pekar på att roller inom AI, maskininlärning, big data och digital teknik hör till de yrken som väntas växa snabbt fram till 2030. Samtidigt betonas att många arbeten förändras snarare än försvinner helt. Källa

EU lyfter också återkommande fram att Europa behöver fler med avancerade digitala färdigheter och fler personer som kan utveckla, använda och förstå ny digital teknik. Källa

Osäkerhet och försiktighet

Det går inte att säga exakt vilka IT-yrken som kommer att finnas kvar, förändras eller försvinna.

Historiskt har ny teknik både tagit bort vissa arbetsuppgifter och skapat helt nya roller.

Därför är det klokt att vara försiktig med tvärsäkra påståenden.

Det mest välunderbyggda man kan säga är att IT-yrken sannolikt kommer att förändras, och att digital kompetens, problemlösning och förmåga att samarbeta med AI blir allt viktigare. Källa

Vad som kan bli viktigt framöver

I framtiden kan det bli extra viktigt att kunna

förstå kod
granska AI-genererat innehåll
testa och kvalitetssäkra system
arbeta med data
förstå säkerhet och integritet
förklara tekniska lösningar för andra

Det betyder att programmering fortfarande är viktigt, men också att mänskligt omdöme kan bli ännu viktigare när AI används mer.

Koppling till kursen

När du lär dig programmering får du inte bara träna på att skriva kod.

Du får också bättre förståelse för hur digital teknik fungerar och hur AI kan påverka samhället och arbetslivet.

Det gör det lättare att delta i en värld där programmering och AI blir allt vanligare.

15.2 Etiska frågor

När programmering används i samhället uppstår också etiska frågor.

Etik handlar om vad som är rätt och fel, och om hur teknik påverkar människor.

Ett program kan fungera tekniskt bra men ändå skapa problem.

Därför är det viktigt att inte bara fråga om något går att bygga, utan också om det bör byggas och hur det bör användas.

Varför etik är viktigt

Program och digitala system påverkar ofta många människor.

De kan samla in information, fatta beslut eller styra vad användaren ser.

Om ett program är dåligt utformat kan det leda till orättvisa, felaktiga beslut eller problem med integritet.

Därför behöver programmerare och andra som arbetar med teknik tänka på konsekvenserna.

Integritet

Många program samlar in data om användare.

Det kan vara namn, plats, sökningar, köp, bilder eller annan information.

Då uppstår frågor som

Vilken information samlas in
Varför samlas den in

Hur länge sparas den
Vem får tillgång till den

Om människor inte förstår vad som samlas in kan det bli ett problem.

Därför är det viktigt att program är tydliga och att användarens integritet respekteras.

Rättvisa och bias

Ett program kan behandla människor olika.

Det kan bero på hur programmet är byggt eller vilken data som används.

Om ett system tränas på sned eller ofullständig data kan resultatet bli orättvist.

Detta kallas ofta bias.

Ett AI-system som hjälper till att sortera jobbansökningar kan till exempel ge sämre resultat för vissa grupper om träningsdatan är obalanserad.

Det betyder att tekniska system inte alltid är neutrala.

Ansvar

När ett digitalt system gör fel uppstår frågan om ansvar.

Vem är ansvarig om ett program ger fel beslut

programmeraren
företaget
användaren
den som beställde systemet

Det finns inte alltid ett enkelt svar.

Därför är det viktigt att system testas, granskas och används på ett ansvarsfullt sätt.

Säkerhet

Program måste också vara säkra.

Om ett system har svag säkerhet kan information läcka eller missbrukas.

Det kan skada både enskilda personer och hela organisationer.

Därför är säkerhet en viktig etisk fråga, inte bara en teknisk fråga.

AI och etik

När AI används blir etiska frågor ofta ännu tydligare.

AI kan fatta eller påverka beslut inom till exempel skola, vård, arbetsliv och sociala medier.

Då blir frågor om rättvisa, insyn och ansvar extra viktiga.

Samtidigt är det ofta svårt att förstå exakt hur ett AI-system har kommit fram till ett resultat.

Det gör att människor behöver vara försiktiga och kritiska.

Det är inte alltid enkelt

Etiska frågor har ofta inga enkla svar.

Ett system kan vara användbart men samtidigt skapa risker.

Till exempel kan ansiktsigenkänning öka säkerheten på vissa platser, men också hota människors integritet.

Därför behöver man väga fördelar mot nackdelar.

Exempel

Tänk dig en app som samlar in elevers resultat och föreslår vilka elever som behöver extra stöd.

Det kan vara hjälpsamt.

Men det väcker också frågor:

Är informationen säker
Kan systemet göra fel
Riskerar vissa elever att bedömas orättvist
Förstår användarna hur systemet fungerar

Koppling till kursen

När du arbetar med etiska frågor tränar du inte bara programmering.

Du tränar också förståelse för hur digital teknik påverkar människor och samhälle.

Det är en viktig del av att använda programmering på ett ansvarsfullt sätt.

16 - Versionshantering och GitHub

När man programmerar ändras koden ofta många gånger.

Man testar nya lösningar, rättar fel och bygger vidare steg för steg.

Därför är det viktigt att kunna spara olika versioner av sitt program.

I det här kapitlet får du lära dig vad versionshantering är, hur GitHub kan användas och hur du laddar upp Pythonfiler till ett repository.

16.1 Vad är versionshantering

Ja, **det är bäst att skriva det i boken** så att eleverna får en tydlig regel.

Du kan ändra den delen i kapitel 15 till att commit-kommentarer **alltid skrivs på engelska**.

Här är en bra version att stoppa in:

Commit-kommentarer på engelska

I den här kursen ska commit-kommentarer skrivas på engelska.

Det gör att du vänjer dig vid ett arbetssätt som är vanligt inom programmering.

Många verktyg, guider och projekt i GitHub använder engelska.

Därför är det bra att redan från början träna på att skriva tydliga kommentarer på engelska.

Exempel på bra commit-kommentarer

```
Create first version of the program

Add menu for user choices

Fix input error for numbers

Add function for calculating average

Improve error handling with try-except
```

Viktigt att tänka på

En commit-kommentar ska vara

kort
tydlig
konkret

Den ska berätta vad du har ändrat.

Undvik sådana kommentarer

```
stuff
```

```
update
```

```
new
```

```
fix
```

De är för otydliga och berättar inte vad som faktiskt ändrades.

En enkel modell

Du kan skriva:

Verb + what you changed

Exempel:

```
Add menu
```

```
Fix loop condition
```

```
Create function for input
```

```
Improve print output
```

16.2 Ladda upp Pythonfiler till GitHub

När du har skapat ett repository på GitHub kan du börja ladda upp dina Pythonfiler.

I den här kursen räcker det att du använder GitHub direkt i webbläsaren.

Du behöver alltså inte börja med terminalkommandon.

Vad du behöver

Innan du börjar behöver du

ett GitHub-konto
ett repository
en eller flera Pythonfiler på datorn

Vanliga filer i den här kursen är till exempel

`main.py`

`uppgift1.py`

`spel.py`

Skapa ett repository

Ett repository är projektets mapp på GitHub.

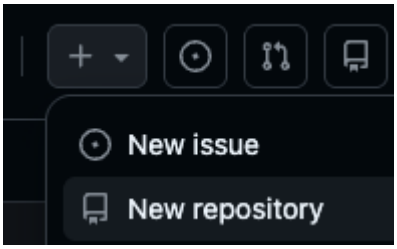
Så här gör du:

Gå till GitHub

Logga in på ditt konto

Klicka på `New`

Eller välj



Skriv ett namn på projektet, till exempel `python-uppgifter`

Välj om projektet ska vara `Public` eller `Private`

Klicka på `Create repository`

Ladda upp en Pythonfil

När ditt repository är skapat kan du ladda upp dina filer.

Så här gör du:

Öppna ditt repository

Klicka på `Add file`

Välj `Upload files`

Dra in din `.py`-fil eller klicka på `choose your files`

Kontrollera att rätt fil har valts

Skriv en commit-kommentar på engelska

Klicka på `Commit changes`

Nu finns filen sparad på GitHub.

Exempel

Du har gjort ett program som heter `main.py`.

Då kan du ladda upp filen till ditt repository.

En bra commit-kommentar kan vara:

```
Add first version of main.py
```

Ladda upp en ny version av samma fil

VIKTIGT: Du måste ladda upp en nyversion varje programmeringstillfälle hellre en gång för mycket än en gång för lite. Minst varje dag du kodar.

Om du har ändrat i filen på din dator kan du ladda upp den igen.

GitHub sparar då den nya versionen och visar att en förändring har gjorts.

Skriv alltid en ny commit-kommentar som förklarar vad du ändrat.

Exempel:

```
Add input for user name
```

```
Fix error in while loop
```

```
Improve print output
```

Redigera filer direkt på GitHub

Du kan också ändra en fil direkt på GitHub.

Det passar bäst för små ändringar.

Så här gör du:

Öppna filen i ditt repository

Klicka på pennan för att redigera filen

Ändra i koden

Scrolla ner på sidan

Skriv en commit-kommentar på engelska

Klicka på `Commit changes`

Vad du ska ladda upp

I den här kursen räcker det oftast att du laddar upp

`.py`-filer
eventuellt en `README.md`
andra filer som hör till projektet

Det viktigaste är att din Pythonkod finns sparad i repositoryt.

Arbeta steg för steg

Det är bättre att ladda upp koden ofta än att vänta tills allt är klart.

Ett bra arbetssätt är:

skriv en del av programmet

testa att det fungerar

ladda upp filen

skriv en tydlig commit-kommentar

fortsätt sedan med nästa del

På det sättet blir det lättare att följa hur programmet utvecklas.

Vanliga misstag

Vanliga misstag är att

ladda upp fel fil
glömma att skriva en tydlig commit-kommentar
vänta för länge mellan uppladdningarna
skriva över fungerande kod utan att ha sparat en tidigare version

Därför är det bra att arbeta i små steg.

README.md

I många projekt på GitHub finns en fil som heter `README.md`.

Det är ofta den första filen man ser när man öppnar ett repository.

README-filen används för att förklara vad projektet är och hur det fungerar.

Vad betyder README

Namnet `README` betyder ungefär “läs detta först”.

Det är alltså en fil som hjälper andra att snabbt förstå projektet.

Vad betyder `.md`

Filändelsen `.md` står för Markdown.

Markdown är ett enkelt sätt att formatera text.

Du kan till exempel skriva rubriker, listor och länkar på ett tydligt sätt.

Varför README är viktig

En README gör projektet lättare att förstå.

Den hjälper andra att se

vad programmet gör
vem projektet är till för
hur man startar programmet
vad filerna används till

Även om projektet är enkelt är det bra att vänja sig vid att ha en README.

README ska skrivas på engelska

I den här kursen ska README-filen skrivas på engelska.

Det beror på att GitHub ofta används i internationella sammanhang.

Många projekt, guider och utvecklare använder engelska.

Därför är det bra att träna på att skriva enkel och tydlig projektinformation på engelska.

Vad kan stå i en enkel README

I Programmering 1 behöver README-filen inte vara lång eller avancerad.

Det räcker med enkel information som hjälper någon att förstå projektet.

Du kan till exempel ha med:

project title
short description
how to run the program
files in the project
author

Exempel på innehåll

Det här kan vara en enkel struktur:

```
# Movie Program

This program lets the user enter the name of a movie and its length in
minutes.

The program then shows the movie title and converts the time into
hours and minutes.

## How to run

Run the file main.py in Python.

## Files

main.py - the main program

README.md - information about the project
```

```
## Author

Paul
```

Vad som passar i den här kursen

För elever i Programmering 1 räcker det ofta att README-filen innehåller:

en rubrik med projektets namn
en kort beskrivning av programmet
hur man startar programmet
vilka filer som finns
författarens namn

Tips

Skriv kort och tydligt.

Använd enkel engelska.

Det viktigaste är att någon annan kan förstå vad projektet handlar om.

17 - Gör en Tkinter-app till en Mac-app med PyInstaller

1. Kontrollera att programmet fungerar

Börja med att testa din Pythonfil som vanligt.

Exempel:

```
python3 main.py
```

Programmet ska fungera innan du försöker göra en app.

2. Öppna Terminal

Öppna Terminal på din Mac.

Gå till mappen där din Pythonfil finns.

Exempel:
`cd ~/Desktop/minapp`

Står du i mappen `Programmering 1` ska du skriva

`cd "kap 16"`
Med `ls` kan du kontrollera att du är i rätt mapp

```
paul@MacBook-Pro Programmering 1 % cd "kap 16"
paul@MacBook-Pro kap 16 % ls
16-start.py
paul@MacBook-Pro kap 16 %
```

3. Installera PyInstaller

Skriv:

```
pip3 install pyinstaller
```

Om det inte fungerar kan du prova:

```
python3 -m pip install pyinstaller
```

4. Kontrollera att PyInstaller är installerat

Skriv:

```
python3 -m PyInstaller --version
```

Om det inte fungerar skriv

```
pyinstaller --version
```

Om du får ett versionsnummer är allt klart.

5. Bygg appen

Om din fil heter `main.py`, skriv:
`python3 -m PyInstaller --windowed --onedir main.py`

Detta betyder:

`--windowed` = appen öppnas utan terminalfönster

`--onedir` = appen byggs som en mapp med allt den behöver

På macOS skapas då både en körbar fil och en `.app` i `dist`-mappen.

6. Hitta appen

När byggandet är klart får du normalt dessa mappar:

```
build
```

```
dist
```

och en spec-fil, till exempel:

```
main.spec
```

Gå in i `dist`-mappen.

Där ska du hitta något i stil med:

```
main.app
```

Det är din Mac-app.

7. Starta appen

Dubbelklicka på `main.app` i Finder.

Om macOS varnar för att appen kommer från en okänd utvecklare kan du behöva:

högerklicka på appen

välja `Öppna`

sedan välja `Öppna` igen

8. Bygg med eget namn

Om du vill att appen ska heta något snyggare kan du skriva:

```
pyinstaller --windowed --onedir --name Filminfo main.py
```

Då blir appen:

```
Filminfo.app
```

9. Lägg till ikon

Om du har en ikonfil i `.icns`-format kan du använda:

```
pyinstaller --windowed --onedir --name Filminfo --icon ikon.icns
main.py
```

10. Viktigt för macOS

Bygg appen på en Mac.

En Mac-app ska helst byggas på samma typ av system som den ska köras på.

PyInstaller-dokumentationen säger också att appar blir kompatibla med den macOS-version där de byggs och senare versioner, inte bakåt hur som helst.

11. Använd inte detta som första val på Mac

Undvik helst:

```
pyinstaller --onefile --windowed main.py
```

Det fungerar ibland, men PyInstaller själva rekommenderar inte den kombinationen för macOS-appbundles eftersom den är mindre effektiv och packar upp innehåll varje gång appen startas.

12. Om appen inte fungerar

Vanliga problem är:

fel Python-miljö

saknade filer som programmet använder

fel sökvägar till bilder eller andra resurser

att man byggt innan programmet fungerade färdigt

Om appen kraschar direkt, testa först att köra `.py`-filen normalt igen.

13. Om programmet använder bilder eller andra filer

Om din Tkinter-app använder till exempel:

bilder

textfiler

json-filer

andra resurser

så behöver de ibland läggas till särskilt i bygget. Då kan man behöva använda extra PyInstaller-inställningar eller en `.spec`-fil. PyInstaller använder spec-filer för mer avancerade byggen.

14. Enkel standardmodell

För de flesta elevprojekt med Tkinter räcker detta:

```
python3 -m pip install pyinstaller
```

```
cd sökväg/till/mappen
```

```
python3 -m PyInstaller --windowed --onedir main.py
```

15. Sammanfattning

Gör så här:

testa att Pythonprogrammet fungerar

installera PyInstaller

öppna Terminal i rätt mapp

kör `pyinstaller --windowed --onedir main.py`

öppna `.app`-filen i `dist`

Exempel

Om du har filen `filmapp.py`, kan du skriva:

```
python3 -m PyInstaller --windowed --onedir main.py
```

Då får du:

```
dist/FilmApp.app
```

Bra att veta

PyInstaller analyserar ditt program och samlar med Python-tolk och beroenden i paketet. Det är därför mottagaren oftast inte behöver installera Python separat för att köra appen.

18 - Arbetssätt i projekt kopplat till Teknik 2

När du arbetar med ett större programmeringsprojekt tränar du inte bara på att skriva kod.

Du tränar också på att planera, lösa problem, testa lösningar, dokumentera ditt arbete och redovisa resultat.

Detta liknar hur man arbetar i Teknik 2.

Där är projektarbete, metodik och problemlösning viktiga delar.

Programmering som en del av ett teknikprojekt
I ett teknikprojekt börjar man ofta med ett problem eller ett behov.

Sedan undersöker man hur problemet kan lösas.

I den här processen kan programmering användas som ett verktyg.

Det betyder att du inte bara programmerar för att öva kod.

Du programmerar för att skapa en lösning som fyller en funktion.

Exempel:

ett program som räknar ut energiförbrukning
ett bokningssystem
en budgetapp
ett program för att hålla ordning på lager
ett planeringsverktyg för skolarbete

Alla dessa exempel är program som kan lösa ett verkligt problem.

Arbeta i steg
I både Programmering 1 och Teknik 2 är det viktigt att arbeta steg för steg.

Ett bra projektarbete kan delas upp så här:

identifiera ett problem
beskriva behov och användare
planera lösningen
skriva kravspecifikation
göra design
implementera programmet
testa programmet
förbättra programmet
dokumentera arbetet
redovisa resultatet

Det gör arbetet mer strukturerat och lättare att följa.

Problem och behov
Ett projekt börjar ofta med att du ser ett behov.

Det kan vara något som tar tid, något som ofta blir fel eller något som kan göras enklare med hjälp av ett program.

Exempel:

En elev vill hålla ordning på sina läxor.
Då kan ett planeringsprogram vara en möjlig lösning.

En butik vill räkna ut totalsummor snabbare.
Då kan ett kassaprogram vara en möjlig lösning.

När du formulerar problemet blir det lättare att förstå vad programmet ska göra.

Krav och design
När du vet vilket problem du vill lösa behöver du beskriva vad programmet ska kunna göra.

Då skriver du en kravspecifikation.

Sedan planerar du hur programmet ska byggas.

Detta är designfasen.

Här kan du använda pseudokod, skisser eller flödesscheman.

Det är ett arbetssätt som passar både programmering och teknikprojekt.

Implementering
Efter planering och design börjar du bygga lösningen.

Det är då du implementerar programmet.

Här använder du det du lärt dig om variabler, villkor, loopar, funktioner, felhantering och användargränssnitt.

När du kodar utifrån en tydlig plan blir arbetet ofta enklare.

Testning och förbättring
I ett projekt räcker det inte att programmet nästan fungerar.

Du behöver testa om lösningen verkligen uppfyller sitt syfte.

Det är också viktigt att fundera på vad som kan gå fel.

Du kan till exempel testa:

om användaren skriver rätt värden
om användaren skriver fel värden
om resultatet blir rimligt
om programmet är lätt att förstå

När du hittar fel eller svagheter förbättrar du programmet.

Detta är en viktig del av både programmering och teknikutveckling.

Risker och rimlighet
I Teknik 2 är det viktigt att tänka på risker och kvalitet.

Det kan du också göra i programmeringsprojekt.

Frågor att fundera på kan vara:

Vad händer om användaren skriver fel
Vad händer om programmet räknar fel
Finns det risk att information visas eller sparas på fel sätt
Verkar resultatet rimligt

Ett program kan fungera tekniskt men ändå ge dåliga resultat om det inte är ordentligt testat.

Därför behöver du göra rimlighetsbedömningar och tänka kritiskt.

Användare och tillgänglighet
En teknisk lösning ska fungera för den som ska använda den.

Därför behöver du fundera på användaren.

Frågor att tänka på:

Är programmet lätt att förstå
Är instruktionerna tydliga
Är knappar och texter lätta att använda
Kan användaren förstå felmeddelanden

Detta är viktigt både i Programmering 1 och i Teknik 2.

Dokumentation och redovisning
När projektet är klart behöver arbetet dokumenteras.

Du behöver kunna beskriva

vad problemet var
hur du tänkte
hur programmet är uppbyggt
hur du testade det
vilka förbättringar du gjorde

Sedan redovisar du arbetet.

Det visar inte bara vad programmet gör, utan också hur du har arbetat.

Det är en viktig del av projektmetodik.

Koppling till Teknik 2
När du arbetar så här tränar du på flera delar som är viktiga i Teknik 2.

Du tränar på

metodik och arbetsformer i projekt
tillämpning av programmering för problemlösning
testning och kvalitetsarbete
dokumentation och presentation
att värdera tekniska lösningar

Det gör att programmering kan användas som en del av ett större teknikarbete.

Sammanfattning
Ett programmeringsprojekt kan också vara ett teknikprojekt.

När du arbetar strukturerat med planering, krav, design, kod, testning, dokumentation och redovisning tränar du arbetssätt som passar både Programmering 1 och Teknik 2.

Programmering blir då inte bara kod, utan ett sätt att lösa verkliga problem.

Förslag på app / program

Innan projektet

Du ska

- välja ett problem eller behov som ditt program ska lösa
- bestämma vem programmet är till för
- beskriva syftet med programmet
- skriva vad programmet ska kunna göra
- skriva en enkel kravspecifikation
- fundera på vilken information programmet behöver
- planera hur programmet ska fungera steg för steg
- skriva pseudokod eller göra en enkel skiss
- bestämma hur programmet ska delas upp i delar eller funktioner
- välja tydliga namn på filer, variabler och funktioner
- skapa en projektmapp på datorn
- skapa ett repository på GitHub
- skriva en enkel README.md på engelska på GitHub
- ladda upp en första version av projektet till GitHub

Under projektet

Du ska

- bygga programmet steg för steg
- börja med en enkel version som fungerar
- testa varje del innan du går vidare
- skriva tydlig och strukturerad kod
- använda bra namn på variabler och funktioner
- kommentera viktiga delar i koden när det behövs
- använda felhantering där det behövs
- kontrollera att programmet fungerar med rätt inmatning
- kontrollera att programmet hanterar felaktig inmatning
- göra rimlighetsbedömningar av resultatet
- förbättra programmet när du hittar fel
- spara nya versioner ofta
- ladda upp dina `.py`-filer till GitHub regelbundet

VIKTIGT: Du måste ladda upp en nyversion varje programmeringstillfälle hellre en gång för mycket än en gång för lite. Minst varje dag du kodar.

- skriva tydliga commit-kommentarer på engelska

- uppdatera README.md om projektet förändras
- skriva ner problem som uppstår och hur du löser dem

Efter projektet

Du ska
- testa hela programmet med flera testfall
- skriva ner vilken inmatning du testat
- skriva vad du förväntade dig skulle hända
- skriva vad som faktiskt hände
- rätta fel som finns kvar
- kontrollera att programmet uppfyller kraven
- kontrollera att programmet är tydligt för användaren
- städa koden så att den blir lätt att läsa
- kontrollera att filnamn och kodstruktur är tydliga
- göra färdigt README.md på engelska
- se till att slutversionen finns uppladdad på GitHub
- dokumentera hur programmet fungerar
- beskriva hur du planerade, byggde och testade programmet
- fundera på vad som gick bra
- fundera på vad som var svårt
- skriva vad du skulle vilja förbättra vidare
- förbereda en redovisning av projektet

Kort checklista

Innan
- problem
- syfte
- användare
- krav
- planering
- pseudokod
- GitHub

Under
- koda steg för steg
- testa ofta
- spara versioner
- skriva commits
- förbättra

Efter
- sluttesta
- dokumentera
- rätta fel
- ladda upp slutversion
- redovisa

Appendix - förslag till projekt

E-nivå = en funktion, få inmatningar, tydligt resultat
C-nivå = flera funktioner, bättre struktur, listor eller filer
A-nivå = större helhet, tydlig användaranpassning, robustare kod, fler delar

E-nivå
- Hälsningsapp
- Åldersräknare
- Temperatur-omvandlare
- BMI-räknare
- Enkel miniräknare
- Tärningskastare
- Myntkast
- Par eller udda
- Multiplikationstabell
- Klickräknare
- Filmapp med namn och längd
- Enkel quizapp med några frågor
- Gissa talet med GUI
- Rabattkalkylator
- Rektangelräknare

C-nivå
- Att-göra-lista
- Inköpslista
- Kontaktlista
- Filmlista med flera filmer
- Boklista
- Glosförhör
- Mattequiz med poäng
- Budgetapp
- Studieplanerare
- Vanetracker
- Cafébeställning

- Enkel kassaapparat
- Bokningsapp
- Biblioteksapp i enkel form
- Anteckningsapp med spara-funktion

A-nivå

- Väderapp med API
- Quizapp med flera kategorier och poängsystem
- Biblioteksapp med utlån och återlämning
- Bokningssystem med flera val
- Inventarieapp med sökfunktion
- Träningsapp med loggning och sparning
- Receptapp med flera recept och kategorier
- Studieapp med flera vyer
- Budgetapp med sparade utgifter
- Felanmälningssystem
- Medlemsregister
- Enkel hotellbokning
- Turneringhanterare
- Vanetracker med statistik
- Projektplanerare